



УНИВЕРЗИТЕТ
У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Алекса Величковић

**Пројектовање и развој система за реализацију
видео конференција у реалном времену
заснованог на
WebRTC технологији**

ЗАВРШНИ РАД

Основне струковне студије

Нови Сад, година 2026.



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	Монографска документација
Тип записа, ТЗ :	Текстуални штампани документ
Врста рада, ВР :	Завршни рад
Аутор, АУ :	Алекса Величковић
Ментор, МН :	доц. Др Сениша Николић, ФТН Нови Сад
Наслов рада, НР :	Пројектовање и развој система за реализацију видео конференција у реалном времену заснованог на WebRTC технологији
Језик публикације, ЈП :	Српски
Језик извода, ЈИ :	Српски/Енглески
Земља публикавања, ЗП :	Србија
Уже географско подручје, УГП :	Војводина
Година, ГО :	2026
Издавач, ИЗ :	Ауторски репринт
Место и адреса, МА :	Факултет техничких наука, Трг Доситеја Обрадовића 6, Нови Сад
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	Бр. Поглавља 9/страница 111/цитата 62/табела 4/слика 30/листинга 23
Научна област, НО :	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :	Софтверско инжењерство
Предметна одредница/Кључне речи, ПО :	Видео конференције, WebRTC, LiveKit, LiveKit Egress, MinIO, S3, Keycloak, Angular, Spring Boot, Docker
УДК	
Чува се, ЧУ :	Библиотека ФТН, Трг Доситеја Обрадовића 6, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	У раду је приказано пројектовање и развој само-хостоване платформе за видео састанке у реалном времену. Систем користи WebRTC медијску комуникацију преко LiveKit SFU сервера, LiveKit Egress за снимање, MinIO као S3-компатибилно објектно складиште, Spring Boot backend и Angular клијентску апликацију. Посебно су анализирани једноставан приступ без обавезне регистрације, безбедност заснована на Keycloak-у, API кључевима и JWT токенима, као и могућност Google регистрације преко IdentityProvider-a.
Датум прихватања теме, ДП :	
Датум одбране, ДО :	
Чланови комисије, КО :	Председни к: др Мирослав Зарић, проф., ФТН Нови Сад
	Члан: др Драган Ивановић, проф., ФТН Нови Сад
	Члан, ментор др Сениша Николић, доц., ФТН Нови Сад
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual material		
Contents code, CC :	Bachelor thesis		
Author, AU :	Aleksa Veličković		
Mentor, MN :	Siniša Nikolić, PhD		
Title, TI :	Design and Development of a Real-Time Video Conferencing System Based on WebRTC Technology		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian/English		
Country of publication, CP :	Serbia		
Locality of publication, LP :	Vojvodina		
Publication year, PY :	2026		
Publisher, PB :	Author's reprint		
Publication place, PP :	Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	No. of Chapters 9/Pages 111/references 62/tables 4/pictures 30/listings 23		
Scientific field, SF :	Electrical and computer engineering		
Scientific discipline, SD :	Software Engineering		
Subject/Key words, S/KW :	video conferencing, WebRTC, LiveKit, LiveKit Egress, MinIO, S3, Keycloak, Angular, Spring Boot, Docker		
UC			
Holding data, HD :			
Note, N :			
Abstract, AB :	This thesis presents the design and development of a self-hosted real-time video meeting platform. The system uses WebRTC media communication through a LiveKit SFU server, LiveKit Egress for recording, MinIO as an S3-compatible object store, a Spring Boot backend and an Angular frontend. The work analyses guest access without mandatory registration, Keycloak-based security, API keys and JWT tokens, as well as optional Google registration through an identity provider.		
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended Board, DB :	President:	Miroslav Zarić, PhD, prof., FTN Novi Sad	
	Member:	Dragan Ivanović, PhD, prof., FTN Novi Sad	Menthor's sign
	Member, Mentor:	Siniša Nikolić, PhD, assist. prof., FTN Novi Sad	



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА

21000 НОВИ САД, Трг Доситеја Обрадовића 6

ЗАДАТАК ЗА ЗАВРШНИ РАД

Број:

Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверске и информационе технологије		
Студент:	Алекса Величковић	Број индекса:	SR 44/2023
Степен и врста студија:	Основне струковне студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Др. Синиша Николић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна:			

НАСЛОВ ЗАВРШНОГ РАДА:

Пројектовање и развој система за реализацију видео конференција у реалном времену заснованог на WebRTC технологији

ТЕКСТ ЗАДАТКА:

Аналиzirати постојећа решења за реализацију видео конференција и састанака у реалном времену, са посебним освртом на примену WebRTC технологије и SFU архитектуре. Описати принципе реализације комуникације у реалном времену, технологије коришћене за посредовање и снимање медијског садржаја, као и механизме аутентификације, ауторизације и контроле приступа. Извршити спецификацију и опис архитектуре информационог система за реализацију видео састанака заснованог на WebRTC технологији. Имплементирати систем који омогућава креирање и приступ видео састанцима без обавезне регистрације, учешће више корисника у видео позиву, пренос аудио и видео садржаја у реалном времену, дељење екрана и снимање састанака. Омогућити управљање корисничким налозима, аутентификацију и ауторизацију, различите нивое приступа састанцима, као и чување и контролисано преузимање снимљеног садржаја. Документовати спецификацију, архитектуру и имплементацију развијеног решења.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора



ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1.	Увод	15
2.	Стање у области	17
2.1.	Веб комуникација у реалном времену	17
2.2.	Комерцијална решења	17
2.2.1.	Whereby	17
2.2.2.	Zoom	18
2.2.3.	Google Meet	20
2.2.4.	Microsoft Teams	21
2.2.5.	Jitsi Meet	21
2.2.6.	Cisco Webex	22
2.2.7.	Пресек анализираних решења	23
2.3.	Само-хостована решења	24
2.4.	Снимање видео састанака	25
2.5.	Идентитет и приступ	25
2.6.	Изведени принципи за сопствено решење	25
3.	Коришћене технике и технологије	27
3.1.	WebRTC	27
3.2.	SFU архитектура и LiveKit	28
3.3.	Снимање видео састанака и објектно складиштење	30
3.4.	Аутентификација, ауторизација и управљање токенима ³²	
3.4.1.	JWT токени и раздвајање права приступа	33
3.5.	Серверске технологије и перзистенција података	34
3.5.1.	REST и HTTP интерфејс	35
3.5.2.	JPA и Hibernate	35
3.5.3.	PostgreSQL	36
3.5.4.	Redis	36
3.6.	Клијентске технологије	36
3.6.1.	RxJS и Angular Signals	37
3.6.2.	Tailwind CSS и Lucide	37
3.7.	Контејнеризација и инфраструктурне технологије	38
3.8.	Mailpit и SMTP	38
4.	Спецификација и модел система	41
4.1.	Актери и случајеви коришћења	41
4.2.	Доменски модел	43

4.3.	Секвенца креирања састанка	44
4.4.	Статуси и животни циклус.....	45
4.5	Преглед Docker Compose сервиса	46
4.6	API модел	48
4.7	Безбедносни модел.....	48
4.8	Детаљна анализа ентитета Meeting.....	48
4.9	Детаљна анализа ентитета Participant	49
4.10	Детаљна анализа ентитета ApiKey.....	49
4.11	Доменски инваријанти	49
4.12	Модел грешака.....	50
4.13	Модел претњи и границе поверења	50
4.14	Поузданост снимања и складиштења	51
5	Имплементација система	53
5.1	Backend структура	53
5.2	Сигнализација и медијска веза.....	53
5.3	Креирање састанка	53
5.4	LiveKit, токени и дозволе	55
5.5	Композитно и participant снимање	57
5.6	Webhook обрада	58
5.7	PostgreSQL и JSONB	60
5.8	Завршетак и scheduler-и.....	60
5.9	Email обрада.....	60
5.10	API и Keycloak безбедност	60
5.11	Google Identity Provider	63
5.12	Angular OAuth и API interceptor	64
5.13	LiveKit frontend сервис.....	66
5.14	Nginx и runtime конфигурација	67
5.15	Управљање временом и зонама.....	67
5.16	Валидација	68
5.17	Идемпотентност webhook обраде	71
5.18	Понашање при прекиду мреже	71
5.19	Руковање медијским уређајима	72
5.20	Управљање белешкама и rich text садржајем	73

5.21	<i>Email испорука и Mailpit.....</i>	74
5.22	<i>API уговор и frontend модели</i>	75
5.23	<i>Продукциона употреба и оперативни ризици.....</i>	77
5.24	<i>Ручна провера отказних сценарија</i>	78
5.25	<i>Приватност података.....</i>	79
6	Приказ имплементираног система.....	81
6.1	<i>Ручна провера функционалности</i>	101
7	Закључак	105
8	Литература	107
9	Биографија	111

1. Увод

У овом поглављу представљен је проблем реализације видео састанака у реалном времену, описани су циљеви рада и образложен је избор приступа заснованог на само-хостованој веб платформи. Посебна пажња посвећена је једноставности употребе: корисник не мора да креира налог, да инсталира посебан програм или да пролази кроз сложен поступак подешавања пре него што се придружи састанку.

Видео комуникација у реалном времену обухвата истовремени пренос звука, слике и контролних порука између више корисника. За разлику од класичних веб-страница које функционишу по принципу захтева и одговора, системи за видео конференције морају у реалном времену да реагују на различите промене стања. Ове промене обухватају прикључивање или напуштање учесника, промене у статусу медијских уређаја (укључивање и искључивање камере или микрофона), као и осцилације у квалитету мрежне везе. Због чињенице да ове промене представљају догађаје који утичу на све остале учеснике, развој оваквих система захтева специфичан приступ архитектури и управљању подацима. [1, 2]

Постојећа комерцијална решења за видео састанке често нуде велики број функција, што може непотребно повећати сложеност коришћења за једноставне и брзе синхронизације. Основни проблеми који се јављају код оваквих система су обавеза креирања корисничког налога пре приступа састанку, неопходност преузимања десктоп или мобилних апликација, као и ослањање на спољашњу инфраструктуру. Последња ставка представља посебан изазов јер ограничава контролу над подацима, медијским токовима и самим снимцима састанака.

Из наведених проблема произилази мотивација за овај рад: потреба за развојем система који ће омогућити једноставнији и бржи приступ видео састанку, без обавезне регистрације за госте, уз потпуну контролу над инфраструктуром и генерисаним подацима. Систем треба да буде погодан за једнократне састанке, позиве са спољним сарадницима и ситуације у којима није оправдано захтевати регистрацију корисника, а да притом сам процес придруживања захтева мали број једноставних корисничких корака.

Предмет овог рада је пројектовање и развој само-хостоване платформе за видео конференције у реалном времену, засноване на WebRTC технологији [1]. Уместо посматрања само појединачних компоненти, рад анализира целокупан системски приступ – од креирања састанка, преко медијске комуникације, до обраде и безбедног чувања снимака унутар сопствене инфраструктуре.

Циљ је да се реализује систем који омогућава креирање састанка без обавезне регистрације, придруживање једним линком, контролу медијских уређаја, снимање и безбедно преузимање резултујућих фајлова, као и да се прикаже целокупан животни циклус једног видео састанка кроз реализацију комплетног софтверског решења које стриктно раздваја јавни приступ од аутентификованог управљања. Како би се овај циљ остварио, систем мора да испуни следеће кључне функционалне и архитектонске захтеве:

- омогућавање једноставног приступа састанку директно из веб-прегледача, без инсталације додатног софтвера;
- обезбеђивање поуздане медијске комуникације у реалном времену коришћењем архитектуре која ефикасно рутира медијске токове без преоптерећења клијената;
- подршка за аутоматско снимање састанака и чување датотека у оквиру сопственог објектног складишта;

- имплементација флексибилног система аутентификације који дозвољава приступ гостима, док регистрованим корисницима пружа напредне опције (управљање профилем, креирање и отказивање састанака), уз могућност пријаве преко спољних пружаоца идентитета;

2. Стање у области

У овом поглављу анализирани су приступи који се користе за реализацију видео комуникације, снимање сесија и управљање идентитетима. Намера није да се прави потпуно функционално поређење свих производа, већ да се издвоје принципи релевантни за пројектовање система који треба да буде једноставан за приступ и под контролом власника инфраструктуре.

2.1. Веб комуникација у реалном времену

Класичан HTTP модел је заснован на захтеву и одговору, што је погодно за креирање састанка, читање података и измену конфигурације. Са друге стране, медијска комуникација има другачије захтеве јер се подаци генеришу непрекидно и морају да се обрађују без чекања на нови кориснички захтев. Да би се овај проблем превазишао, користи се WebRTC (Web Real-Time Communication), пројекат отвореног кода и скуп стандарда који омогућава комуникацију у реалном времену директно кроз веб-прегледаче, без потребе за инсталацијом додатног софтвера. WebRTC комбинује процес сигнализације са директним медијским каналима и користи мрежне механизме који су специјално прилагођени интерактивном преносу звука и слике са минималним кашњењем [1, 2, 48].

2.2. Комерцијална решења

Комерцијалне платформе обично нуде веома широку функционалност: календарске интеграције, управљање корисницима, транскрипцију, напредне модераторске функције и клијентске апликације. Такав приступ може бити оправдан у великим организацијама, али за једноставан позив ствара додатне кораке. При избору решења за овај рад предност је дата минималном току: креирање, дељење линка и улазак у собу. Да би се постојећа решења објективно упоредила са циљевима овог рада, уведено је пет кључних критеријума за евалуацију:

1. Приступ без регистрације: Да ли гост може да приступи састанку искључиво путем линка, без обавезе креирања корисничког налога?
2. Рад у прегледачу: Да ли је за неометано коришћење неопходна инсталација десктоп/мобилне апликације, или се комуникација одвија изворно у веб-прегледачу?
3. Само-хостовање: Да ли решење нуди могућност инсталације инфраструктуре на сопственим серверима ради потпуне контроле система?
4. Снимање: Да ли платформа подржава снимање сесија и на који начин корисник задржава контролу над тим видео-записима?
5. API / embed интеграција: Колико је систем прилагођен уграђивању у спољашње веб-апликације путем API интерфејса или embed компоненти?

Поређење комерцијалних производа показује да је за овај рад важно преместити сложеност у backend, док кориснику остају само креирање састанка, дељење линка и улазак у собу. Такође, поређење кроз горенаведене критеријуме показаће да постојећи системи често жртвују једноставност и контролу инфраструктуре зарад масовних функционалности.

2.2.1. Whereby

Whereby је платформа за видео састанке која се издваја једноставним приступом преко веб прегледача [42]. Кориснику није потребна инсталација додатног софтвера, а за приступ састанку обично је довољан линк собе. Концепт персоналне собе, односно трајнијег линка који организатор

дели са другим учесницима, смањује број корака потребних за започињање разговора. Ова особина је посебно значајна за кориснике који не желе да пролазе кроз сложен поступак подешавања налога и клијентске апликације.

Важна карактеристика Whereby платформе јесте подршка за уграђивање видео састанка у спољашње веб апликације. Састанак се на тај начин може приказати у оквиру постојећег корисничког интерфејса, без преусмеравања корисника на други сервис. Платформа подржава укључивање и искључивање камере и микрофона, дељење екрана, текстуални чет и основну контролу учесника. За планирани систем Whereby представља најдиректнији узор јер показује како се видео комуникација може понудити као једна функционална компонента већег производа, а не као изолована апликација. На слици 2.1 приказан је екран састанка у оквиру платформе Whereby.



Слика 2.1 — Приказ састанка на Whereby платформи

Извор: Whereby [42]

Анализа по критеријумима:

- Приступ без регистрације: Подржан за госте, довољан је само линк.
- Рад у прегледачу: Изузетно добро подржан, без потребе за клијентским апликацијама.
- Само-хостовање: Није подржано, у питању је искључиво SaaS решење [57].
- Снимање: Омогућено, али се подаци чувају на спољним серверима провајдера.
- API / embed интеграција: Највећа предност ове платформе, нуди моћан iFrame API за лако уграђивање.

2.2.2. Zoom

Zoom је једно од најпознатијих решења за видео комуникацију и онлајн састанке. Подржава састанке са већим бројем учесника, дељење екрана, текстуалну комуникацију, снимање и управљање

учесницима преко контрола организатора [37]. Почетни кориснички ток јасно раздваја заказивање, покретање и придруживање састанку, што олакшава сналажење корисника.

Zoom решава потребе великих састанака и догађаја кроз чекаонице, управљање дозволама и издвајање учесника у посебне просторије. Снимање је важно за накнадни преглед садржаја и документовање договора. Недостатак оваквог широког приступа може бити сложенији интерфејс и чињеница да комплетан екосистем нуди више могућности него што је потребно мањем пројекту који жели само уграђени видео модул [43]. Због тога Zoom представља узор за напредне функције и управљање састанком, али не и за минималан број корака. На слици 2.2 приказан је екран састанка у оквиру платформе Zoom.



Слика 2.2 — Приказ састанка на Zoom платформи

Извор: Zoom [37]

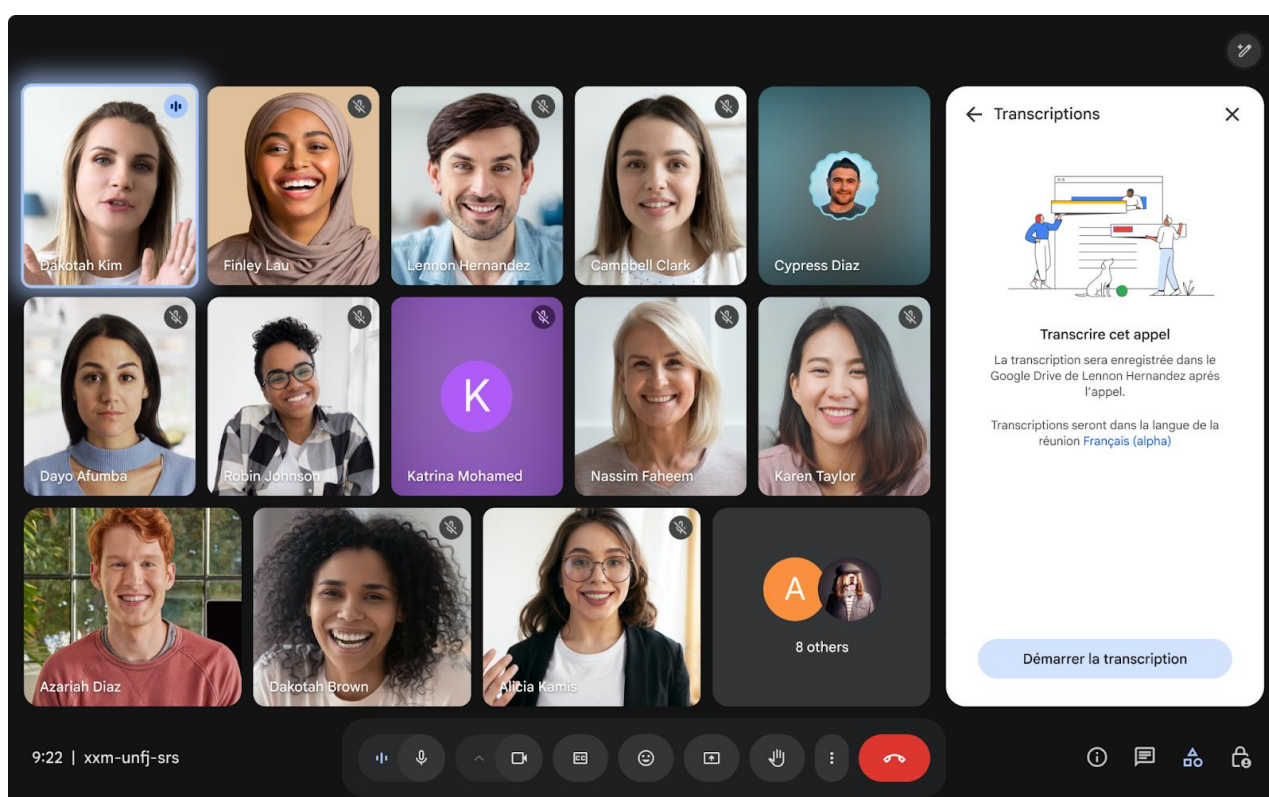
Анализа по критеријумима:

- Приступ без регистрације: Могућ за госте, али интерфејс често инсистира на пријави.
- Рад у прегледачу: Подржан, али са значајно деградираним перформансама у односу на препоручену десктоп апликацију.
- Само-хостовање: Није подржано, у питању је искључиво SaaS решење.
- Снимање: Подржано локално и у облаку, али без директне интеграције са сопственим складиштима попут S3.
- API / embed интеграција: Постоје обимни SDK алати, али су сложени за брзо уграђивање у лаке веб-клијенте.

2.2.3. Google Meet

Google Meet је сервис који се издваја једноставним приступом и интеграцијом са Google екосистемом [38]. Када се користи са Google Calendar-ом и Gmail-ом, заказивање и слање позива учесницима обављају се без великог броја ручних корака. Корисник се у састанак може укључити преко линка, а сервис је прилагођен раду са различитих уређаја.

Подршка за титлове, дељење екрана и једноставан кориснички ток показују како се смањује техничка препрека за улазак у видео комуникацију. Google Meet је, међутим, пре свега намењен крајњим корисницима и организацијама унутар Google окружења. За разлику од планиране платформе, његова основна вредност није обезбеђивање API-ја и embeddable компоненте за произвољну спољашњу апликацију. Зато је користан узор за приступачност и интеграцију календара, али не и потпуна замена за самостални API слој [44]. На слици 2.3 приказан је екран састанка у оквиру платформе Google Meet.



Слика 2.3 — Приказ састанка на Google Meet платформи

Извор: Google [38]

Анализа по критеријумима:

- Приступ без регистрације: Често захтева Google налог, зависно од подешавања организатора.
- Рад у прегледачу: Одлично оптимизован, не захтева инсталацију на десктоп рачунарима.
- Само-хостовање: Није подржано.
- Снимање: Интегрисано директно у Google Drive, без могућности независног хостовања снимака.
- API / embed интеграција: Веома ограничена када је у питању уграђивање (embed) у корисничке веб-апликације изван Google екосистема.

2.2.4. Microsoft Teams

Microsoft Teams је интегрисана платформа за комуникацију и сарадњу у организацијама [39]. Поред видео састанака укључује чет, канале, дељење датотека, календар и повезивање са другим Microsoft 365 алатима. Такав приступ решава шири проблем тимског рада, јер корисници на једном месту добијају комуникацију, материјале и организацију посла.

У погледу видео састанака Teams подржава заказивање, контролу приступа, дељење екрана, снимање и сарадњу током позива. Предност је највећа у организацијама које већ користе Microsoft окружење, јер се смањује потреба за преласком између више сервиса. Са друге стране, корисник који жели само брз видео разговор може наићи на више функција и сложенији ток него што му је потребно. Ово решење зато показује да широка функционалност није увек исто што и једноставност.

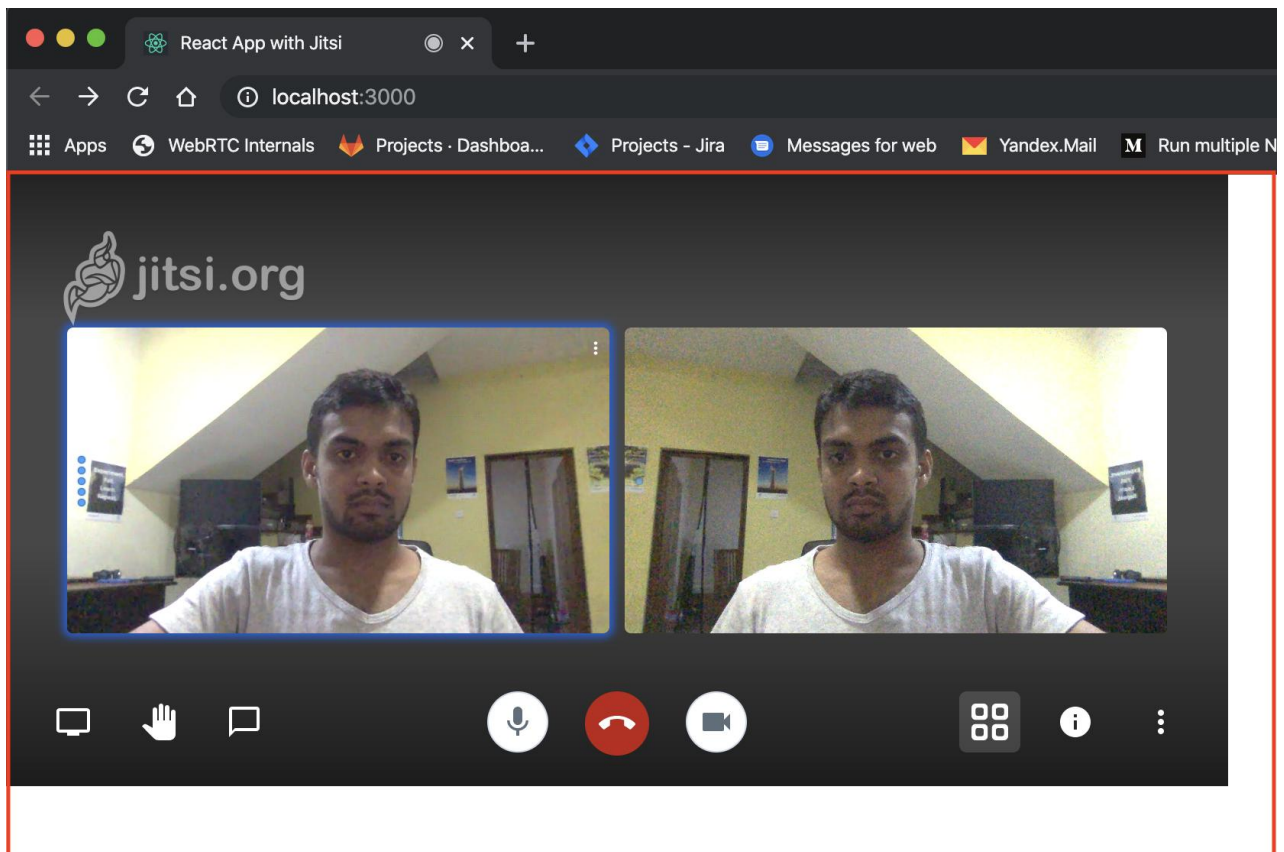
Анализа по критеријумима:

- Приступ без регистрације: Гостима је омогућен приступ преко линка, али је процес често успорен безбедносним чекаоницама и captcha проверама.
- Рад у прегледачу: Могућ, мада платформа снажно форсира преузимање апликације ради стабилности.
- Само-хостовање: Није подржано.
- Снимање: Аутоматски се чува у OneDrive системима провајдера.
- API / embed интеграција: Ослања се на сложени Graph API, што отежава брзу embed интеграцију у мање системе [58].

2.2.5. Jitsi Meet

Jitsi Meet је флексибилно решење отвореног кода које може да се користи без регистрације и, по потреби, самостално хостује [40, 45]. Ово је значајно за системе код којих су приватност, контрола инфраструктуре и независност од комерцијалног провајдера важнији од потпуно управљаног сервиса. Доступне су основне функције као што су дељење екрана, текстуални чет, укључивање и искључивање микрофона и камере и управљање учесницима.

Jitsi Meet показује да видео састанак може бити доступан преко линка и у моделу који омогућава самостално управљање инфраструктуром. За пројекат који се ослања на сопствени embeddable frontend и API, ипак је потребно посебно решити начин уклапања корисничког интерфејса и контрола приступа у постојећи систем. Зато Jitsi представља узор за отвореност и само-хостовање, док планирана платформа исте принципе комбинује са сопственом пословном логиком и моделом података. На слици 2.4 приказан је екран састанка у оквиру платформе Jitsi Meet.



Слика 2.4 — Приказ састанка на Jitsi Meet платформи

Извор: Jitsi Meet [40]

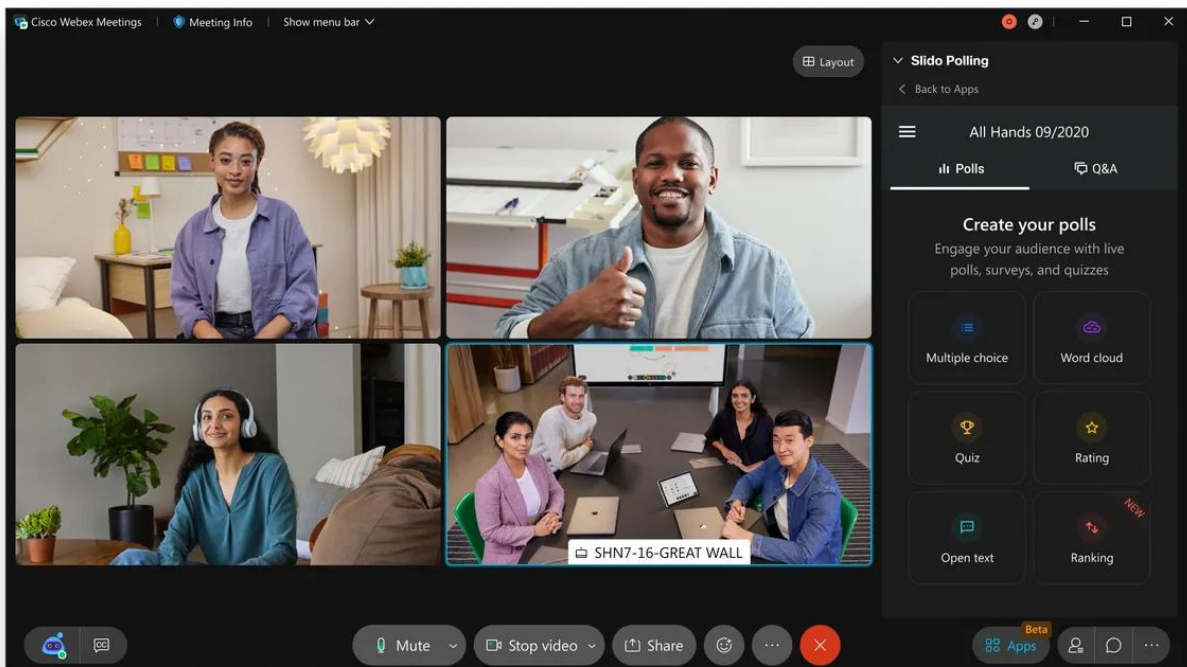
Анализа по критеријумима:

- Приступ без регистрације: Потпуно подржано и веома приступачно.
- Рад у прегледачу: Изворно дизајнирано за рад у прегледачу помоћу WebRTC технологије.
- Само-хостовање: Потпуно подржано; комплетна архитектура се може покренути на сопственом серверу.
- Снимање: Могуће уз помоћ додатне компоненте, али захтева сложенију конфигурацију сервера.
- API / embed интеграција: Нуди изузетан iFrame API за лако уграђивање.

2.2.6. Cisco Webex

Cisco Webex је пословно оријентисана платформа за видео комуникацију и сарадњу [41]. Поред видео састанака подржава дељење садржаја, презентације, вебинар режим и управљање учесницима. Посебан нагласак ставља се на стабилност, безбедност и административну контролу, што је важно организацијама са строгим захтевима у погледу заштите података и управљања корисницима [46].

Webex показује како се видео састанак може уклопити у професионални процес сарадње, али и како број функција утиче на сложеност производа. За планирани систем корисно је преузети принципе поузданости и контроле приступа, али ограничити обим корисничког интерфејса на функције потребне за брзо креирање, придруживање и управљање састанком. На слици 2.5 приказан је екран састанка у оквиру платформе Cisco Webex.



Слика 2.5 — Приказ састанка на Cisco Webex платформи

Извор: Cisco [41]

Анализа по критеријумима:

- Приступ без регистрације: Могућ путем веб-апликације за госте.
- Рад у прегледачу: Подржан, али се десктоп апликација сматра стандардом за пословне састанке.
- Само-хостовање: Примарно cloud решење, уз опцију хибридних чворова, али не и потпуног само-хостовања за мање пројекте.
- Снимање: Напредно снимање са транскрипцијама које се чува на Cisco серверима.
- API / embed интеграција: Подржано путем API-ја, али конципирано за велике пословне интеграције.

2.2.7. Пресек анализираних решења

На основу анализе може се закључити да све посматране платформе решавају основни проблем повезивања корисника путем видео састанка, најчешће кроз линк и веб прегледач. То смањује потребу за инсталацијом посебног софтвера и омогућава приступ са различитих уређаја. Уочава се и заједнички скуп функција: укључивање и искључивање камере и микрофона, дељење екрана, текстуални чет и управљање учесницима.

Напреднија решења, као што су Zoom, Microsoft Teams и Cisco Webex, додају снимање, чекаонице, групне просторије и интеграције са другим алатима. То је погодно за формалне и велике састанке, али истовремено повећава број опција и сложеност корисничког тока. Google Meet показује значај интеграције са постојећим екосистемом и приступачности, док Jitsi Meet показује предности отвореног кода и само-хостовања.

Whereby се највише издваја у контексту планираног пројекта јер наглашава једноставност и могућност уграђивања видео састанка у другу апликацију. Тај приступ директно подржава циљ да корисник не мора да напусти систем у коме већ ради.

На основу свих анализираних производа изводи се закључак да сопствено решење треба да задржи једноставност Whereby платформе, да преузме добре праксе Google Meet-а у погледу корисничког тока, да примени безбедносне и административне принципе Zoom-а и Webex-а и да уважи флексибилност коју показује Jitsi Meet.

Постојећи производи на тржишту нуде висок ниво функционалности, при чему су велике комерцијалне платформе превасходно оптимизоване за свеобухватне корпоративне екосистеме. Са друге стране, иако решења отвореног кода попут Jitsi Meet-а испуњавају кључне захтеве као што су само-хостовање и приступ без регистрације, њихово проширење напредним опцијама, попут независног снимања састанака, захтева инсталацију, конфигурацију и одржавање комплексних екстерних сервиса.

Основни циљ је академски и инжењерски: практична демонстрација пројектовања софтверске архитектуре и интеграције савремених технологија. Развој наменског система овде служи као доказ концепта који показује како се може конструисати ефикасна платформа са јасним API-јем, прегледним корисничким интерфејсом и потпуном, независном контролом над медијском инфраструктуром и подацима.

2.3. Само-хостована решења

У контексту видео-конференција, само-хостован приступ подразумева покретање и управљање целокупном комуникационом инфраструктуром на сопственим серверима, уместо ослањања на екстерне cloud провајдере. То значи да систем самостално мора да обезбеди кључне мрежне компоненте за WebRTC комуникацију.

Основа ове мрежне архитектуре су сервери за сигнализацију, медијски сервер (SFU – Selective Forwarding Unit) за ефикасно рутирање видео токова, као и мрежни транслатори: STUN и TURN сервери [36, 49]. Обзиром да се корисници најчешће налазе иза локалних мрежа, STUN (Session Traversal Utilities for NAT) сервер има задатак да открије јавну IP адресу клијента, омогућавајући му директно повезивање са SFU сервером. Међутим, у корпоративним окружењима са строгим заштитним зидовима или симетричним NAT-ом, директна конекција често није могућа [60]. У таквим ситуацијама посредује TURN (Traversal Using Relays around NAT) сервер, који комплетан медијски саобраћај релејно преусмерава кроз себе. Само-хостовање TURN сервера представља посебан оперативни изазов јер, за разлику од STUN-а који шаље само мале пакете за откривање адресе, TURN захтева изузетно велики мрежни пропусни опсег.

Када је у питању безбедност, главна предност само-хостованог приступа лежи у потпуној контроли над инфраструктуром и подацима. Медијски саобраћај, метаподаци и снимци састанака не пролазе кроз сервере трећих лица. Заштита медијских токова се у WebRTC стандарду обезбеђује кроз DTLS и SRTP протоколе (криптографска заштита саобраћаја у транзиту), док се видео-снимци директно складиште у приватним објектним складиштима (нпр. приватни S3 сервиси) [59]. Ово омогућава већу контролу над начином чувања и обраде података, што може олакшати испуњавање одређених захтева у погледу заштите података.

Са друге стране, елиминисањем cloud провајдера, губи се једноставност потпуно управљаних решења, а власник преузима целокупну оперативну одговорност и одржавање. Одржавање система овакве сложености подразумева континуирани надзор ресурса сервера, управљање тајнама, редовно обнављање TLS сертификата и примену софтверских закрпа како би се спречили безбедносни пропусти. Посебан инжењерски изазов представља скалирање; за разлику од глобалних платформи које аутоматски распоређују ресурсе, само-хостован систем захтева пажљиво праћење оптерећења и успостављање механизма за динамичко додавање нових SFU и TURN инстанци када број истовремених учесника порасте.

Дакле, код само-хостованог приступа, корисничко искуство може остати минималистичко – где је за улазак у собу довољан само једноставан линк – искључиво зато што позадински систем засебно

и транспарентно решава сву инжењерску сложеност мрежног рутирања, безбедног складиштења података и непрекидног одржавања сервиса.

2.4. Снимање видео састанака

Снимање видео-конференција представља сложен инжењерски изазов који значајно превазилази једноставно копирање медијског тока у датотеку. Да би се вишеструки видео и аудио токови различитих учесника претворили у јединствен видео-запис, систем мора да обави процесе compositing-а и транскодовања у реалном времену. Ово подразумева одређивање просторног распореда учесника на екрану, миксовање звука, одабир одговарајућих кодека, као и коначно чување великих датотека у објектна складишта.

Због изузетно високе захтевности ових операција у погледу процесорских ресурса и радне меморије, савремене архитектуре захтевају да се процес снимања логички и физички одвоји од главних сервера за сигнализацију и медијско рутирање. У пракси се за ову намену користе наменски позадински сервиси чији је једини задатак интензивна обрада медија. Оваква децентрализација архитектуре смањује ризик од преоптерећења система, омогућава независно скалирање ресурса за снимање и обезбеђује стабилност самих састанака, док главни систем задржава улогу оркестрације – управљајући искључиво асинхроним догађајима о почетку и завршетку снимања.

2.5. Идентитет и приступ

Системи за видео састанке сусрећу се са специфичним изазовом управљања идентитетима, јер морају да подрже два потпуно различита корисничка тока. Са једне стране, организаторима састанака су потребни трајни кориснички налози, профили и одговарајуће привилегије за креирање и управљање собама. Ово захтева робустне механизме аутентификације, који се данас најчешће ослањају на протоколе отвореног стандарда (попут OAuth 2.0 и OpenID Connect) и екстерне провајдере идентитета. Овакав приступ повећава безбедност јер се избегава директно излагање и руковање осетљивим креденцијалима унутар саме клијентске апликације.

Са друге стране, систем мора да омогући гостујућим корисницима да приступе састанку са минималним препрекама, најчешће искључиво путем дељеног линка, без обавезе креирања налога. За управљање идентитетом регистрованих корисника често се користе системи за управљање идентитетима и приступом (IAM – Identity and Access Management), док се приступ гостујућих корисника може реализовати посебним временски ограниченим токенима везаним за конкретан састанак

2.6. Изведени принципи за сопствено решење

На основу претходне анализе постојећих комерцијалних решења и уочених недостатака, дефинисан је скуп основних принципа којима ће се водити развој сопственог система. Ови принципи представљају архитектуралне и функционалне захтеве усмерене ка једноставности коришћења, флексибилној интеграцији и потпуној контроли над инфраструктуром:

- **Приступ без регистрације и рад у прегледачу:** Улазак у видео-собу за гостујуће кориснике мора бити омогућен искључиво путем дељеног линка, без неопходности креирања корисничког налога или преузимања клијентских апликација (рад изворно у веб-прегледачу).
- **Флексибилно управљање идентитетима:** Постојање корисничког налога треба да буде намењено искључиво организаторима ради креирања састанака, док се управљање

идентитетима мора ослањати на стандардизоване протоколе и екстерне провајдере, логички одвојене од главне апликације.

- **Декапсулација медијске обраде:** Због високе захтевности процеса, рутирање медијског саобраћаја и генерисање видео-снимака морају бити издвојени од главне пословне логике (API-ја) у засебне позадинске сервисе.
- **Независност инфраструктуре (само-хостовање):** Све компоненте система морају бити дизајниране тако да се могу покренути и одржавати унутар приватне мреже, потпуно независно од спољних *cloud* провајдера.
- **Контрола над подацима и снимцима:** Систем мора омогућити директно чување видео-записа у приватном објектном складишту под контролом власника, уз механизме за безбедан и временски ограничен приступ тим датотекама.
- **Спремност за интеграцију:** Клијентски интерфејс и позадински систем треба да буду пројектовани тако да омогућавају лако уграђивање у постојеће спољашње системе.

Приказани принципи не значе да је само-хостована платформа универзално бољи избор у свим сценаријима. Њена предност долази до изражаја када су примарни захтеви контрола над подацима, дубока интеграција са специфичним пословним системима и могућност прилагођавања корисничког тока. Уколико су, са друге стране, неопходни глобална доступност, уговорени нивои подршке или велики број напредних функција за корпоративну сарадњу, комерцијална услуга представља логичнији избор који смањује оперативно оптерећење. За потребе овог рада, одабран је приступ сопственог развоја и само-хостовања, јер је он у потпуности усклађен са циљем технолошке независности, безбедног руковања подацима и минималистичког корисничког искуства.

3. Коришћене технике и технологије

У овом поглављу описане су технике и технологије које представљају основу развијене платформе за реализацију видео састанака у реалном времену. Посебна пажња посвећена је технологијама које непосредно одређују начин успостављања и одржавања видео комуникације, посредовању медијског садржаја, снимању састанака, објектном складиштењу, као и механизмима аутентификације и ауторизације. Поред наведених технологија, описане су и серверске, клијентске и инфраструктурне технологије које обезбеђују развој, извршавање и повезивање појединачних делова система.

3.1. WebRTC

WebRTC (Web Real-Time Communication) представља скуп веб технологија, програмских интерфејса и комуникационих механизма намењених реализацији аудио, видео и податковне комуникације у реалном времену између учесника, при чему је значајан део функционалности доступан директно у савременом веб прегледачу [1, 2]. Основна предност ове технологије за системе видео конференција огледа се у томе што омогућава приступ медијским уређајима и успостављање комуникације без потребе за инсталацијом посебног клијентског софтвера. На тај начин веб прегледач постаје непосредно извршно окружење за медијску комуникацију.

У оквиру WebRTC комуникације преносе се различите врсте медијских садржаја, пре свега звук и слика, док су подаци могући путем податковних канала. Медијски токови настају на страни учесника, обрађују се у реалном времену и затим преносе другим учесницима. WebRTC омогућава успостављање медијске комуникације између крајњих тачака, при чему се, у зависности од архитектуре система, медијски токови могу размењивати непосредно између учесника или посредством медијског сервера [1, 2].

Један од важних аспеката WebRTC технологије јесте чињеница да успостављање медијске везе није ограничено на један конкретан тип мрежне инфраструктуре. Учесници могу бити смештени иза различитих облика мрежног превођења адреса и заштитних механизма, због чега се користе механизми за откривање доступних мрежних путања и избор одговарајућег начина комуникације. Избор путање зависи од услова у мрежи и расположивих комуникационих параметара. У практичним системима видео комуникације то значи да се поузданост везе не може посматрати само са становишта апликације, већ и са становишта мрежних услова под којима се медијски ток успоставља.

WebRTC такође уводи безбедносне механизме који су значајни за систем који обрађује аудио и видео садржај. Приступ камери и микрофону контролише веб прегледач, а кориснику се мора омогућити да експлицитно одобри приступ овим уређајима. Овај механизам представља важну безбедносну границу, јер апликација не би требало да буде у могућности да не приметно активира медијске уређаје корисника [1, 2]. Истовремено, доступност медијских уређаја не значи аутоматски и право на објављивање њиховог садржаја другим учесницима. Право на приступ соби и право на објављивање и пријем медијских токова морају бити контролисани на нивоу комуникационог система.

Посебан значај има раздвајање сигнализације од медијског преноса. Сигнализација служи за размену контролних информација неопходних за успостављање сесије, док се стварни аудио и видео садржај преноси медијским каналом. Сигнализацијом се, између осталог, могу преносити информације о идентитету учесника, расположивим медијским токовима, статусу везе и другим

параметрима потребним за управљање сесијом. Сам медијски садржај не мора пролазити кроз исти механизам комуникације. Ово раздвајање представља један од основних концепата система за видео конференције, јер омогућава да се контролна логика и медијска комуникација организују као различити, али повезани механизми [1, 2].

У систему видео састанака додатно је важно праћење промена стања комуникационе везе. Прекиди у мрежи не морају нужно означавати да је учесник напустио састанак. Привремени губитак везе може бити последица промене мреже, краткотрајног пада квалитета или других пролазних услова. Због тога је значајно разликовати стање поновног повезивања од трајног прекида сесије. Такво раздвајање омогућава да кориснички интерфејс одговори на привремене проблеме без непосредног уклањања учесника и ослобађања свих ресурса повезаних са активном комуникацијом.

Квалитет WebRTC комуникације зависи од више међусобно повезаних фактора. На њега утичу доступни мрежни пропусни опсег, кашњење, губитак пакета, варијације кашњења, могућности медијских уређаја и тренутно оптерећење система. Због тога WebRTC комуникација није ограничена на једноставно „слање видеа“, већ представља континуалан процес праћења стања мреже и прилагођавања медијског преноса. Промена мрежних услова може захтевати промену квалитета или карактеристика медијског тока како би се очувала употребљивост комуникације.

WebRTC сам по себи не дефинише комплетан модел управљања собама, учесницима, снимањем и пословним правилима. У системима видео-конференција са више учесника за дистрибуцију медијских токова могу се користити додатне серверске компоненте, као што су SFU медијски сервери.

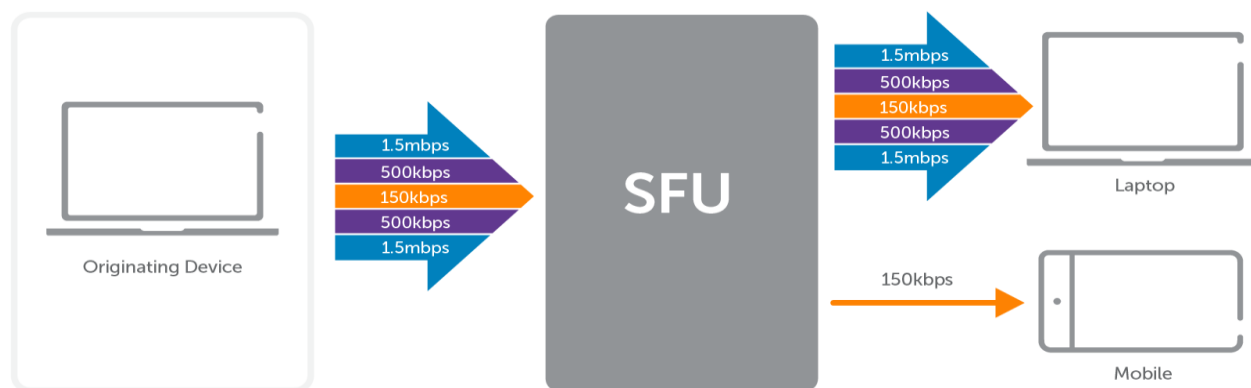
3.2. SFU архитектура и LiveKit

При реализацији видео конференција са више учесника комуникациона архитектура мора да реши проблем дистрибуције медијских токова. У најједноставнијем моделу сваки учесник би морао да успостави посебну везу са сваким другим учесником, што број комуникационих токова повећава са порастом броја учесника. Такав приступ може довести до раста оптерећења на клијентским уређајима и мрежи, због чега се у системима са више учесника често примењује архитектура SFU.

SFU (Selective Forwarding Unit) представља тип медијског сервера који прима медијске токове од учесника и прослеђује их другим учесницима без потребе да сваки долазни ток поново кодира за сваког примаоца. На тај начин медијски сервер преузима улогу посредника у дистрибуцији садржаја, док се значајан део медијске обраде задржава на клијентској страни. Овај приступ представља компромис између директне комуникације учесника и модела у коме сервер обрађује и поново кодира комплетан медијски садржај.

Основна предност SFU модела огледа се у томе што учесник не мора да шаље засебан медијски ток сваком другом учеснику. Уместо тога, сопствени ток се шаље медијском серверу, а сервер га селективно прослеђује онима којима је потребан. Оваква организација значајно смањује број неопходних одлазних веза на страни клијента. Истовремено, сервер добија увид у активне учеснике и њихове медијске токове, што омогућава организовано управљање комуникационом собом.

На слици 3.1 приказан је поједностављени однос између WebRTC клијената и SFU медијског сервера. Приказана архитектура илуструје да се медијска комуникација одвија преко централне тачке за прослеђивање, док су учесници и даље непосредно укључени у WebRTC комуникацију.



Слика 3.1 — SFU посредовање

Извор: WebRTC SFU: the complete Guide [49].

LiveKit представља серверску платформу за реализацију комуникације засноване на WebRTC технологији и SFU архитектури [9, 10]. Уместо да апликација самостално реализује све механизме за управљање медијским сесијама, LiveKit обезбеђује готов слој за рад са собама, учесницима и медијским токовима. Тиме се комплексност ниског нивоа комуникације изолује од пословне логике апликације.

Један од основних концепата LiveKit-а јесте соба као логичка целина унутар које се одвија медијска комуникација. У соби се налазе учесници, а сваки учесник може објављивати и примати различите врсте медијских токова. Сервер управља дистрибуцијом тих токова и води рачуна о њиховој доступности другим учесницима у складу са дозвољеним правима [9, 10].

Издвајање медијске комуникације у засебан SFU сервис има и архитектонски значај. Пословни део система може бити задужен за креирање и управљање састанцима, док медијски сервер обрађује питања која су непосредно повезана са реалним временом. На овај начин раздвајају се различити типови оптерећења: захтеви за управљање подацима и пословном логиком и континуалан пренос аудио и видео садржаја. Ово раздвајање олакшава независно подешавање ресурса и праћење понашања различитих делова инфраструктуре.

LiveKit такође обезбеђује серверске механизме за генерисање и проверу приступних токена, управљање учесницима и објављеним токовима, као и достављање догађаја о променама стања сесије [9, 10]. На тај начин је комуникација између апликационог сервера и медијског слоја заснована на јасном уговору: апликациона страна одређује ко може да приступи одређеној соби и које дозволе треба да добије, док LiveKit извршава та правила унутар медијске сесије.

Посебну пажњу захтевају дозволе за објављивање и пријем медијских токова. Чињеница да је учеснику дозвољен приступ соби не подразумева да он мора имати исте привилегије као власник или организатор састанка. Раздвајање ових права омогућава да се медијска комуникација прилагоди различитим улогама. Учесник може, на пример, имати право да прими видео и звук, а да нема исте могућности управљања сесијом као организатор.

Улога LiveKit-а није ограничена само на прослеђивање пакета. Систем мора да прати присуство учесника, стање њихових веза и доступне медијске токове. Промене у овим елементима представљају догађаје који могу бити од значаја за кориснички интерфејс и пословну логику. На тај

начин медијски слој постаје извор динамичког стања које се мора правилно обрадити и приказати корисницима.

Избор SFU приступа у систему видео конференција повезан је и са ограничењем броја учесника у једној соби. За мање и средње састанке SFU модел омогућава добар однос између сложености инфраструктуре и потребних перформанси. Са порастом броја учесника расту и захтеви према медијском серверу, па се архитектура мора посматрати у односу на очекивани број истовремених учесника, квалитет медијског садржаја и расположиве ресурсе.

Поред основне функције преноса медијских токова, LiveKit обезбеђује инфраструктуру погодну за различите сценарије комуникације у реалном времену, укључујући динамичко управљање учесницима, промену доступних медијских извора и обраду догађаја током трајања сесије [9, 10]. Подршка за издвајање серверских функција, интеграцију са спољним сервисима и различите механизме обраде медијског садржаја омогућава да се платформа за видео комуникацију проширује без промене основног модела комуникације. Посебно је значајна могућност издвајања медијског садржаја из активне сесије ради његовог даљег чувања и обраде, чиме се отвара простор за примену механизма за снимање видео састанака.

3.3. Снимање видео састанака и објектно складиштење

Поред комуникације у реалном времену, имплементиран систем за видео конференције има потребу за трајним чувањем садржаја реализованих састанака. Снимање медијске сесије представља посебан проблем јер се континуални ток аудио и видео података мора претворити у датотеку погодну за касније чување и преузимање. Једно од решења намењених снимању и обради медијских сесија у LiveKit окружењу јесте LiveKit Egress

LiveKit Egress представља сервис намењен извозу медијског садржаја из LiveKit окружења ка датотеци или другом одредишту за дистрибуцију и складиштење [10, 11]. За разлику од основне функције SFU сервера, чији је задатак прослеђивање активних медијских токова, Egress се користи онда када је потребно медијску сесију претворити у трајни резултат. На тај начин снимање има сопствени животни циклус, ресурсе и могућности обраде грешака.

Приликом снимања потребно је дефинисати начин на који се више аудио и видео токова комбинује у коначни резултат. Композитно снимање подразумева стварање јединственог видео записа који одражава целокупну сесију, док је могуће издвојити и појединачне медијске токове. Избор између ових начина зависи од потреба система, али и од расположивих рачунарских ресурса. Композитно снимање захтева додатну обраду јер се више токова мора поставити у јединствени распоред и кодирати у излазни формат [10, 11].

Снимање у реалном времену представља ресурсно захтевнији поступак од самог прослеђивања медијских токова. Поред пријема садржаја, неопходне су операције декодирања, композиције и кодирања, а затим и упис резултујуће датотеке. Због тога систем за снимање не треба посматрати као пасивни додатак медијском серверу, већ као засебан тип радног оптерећења. Независност овог слоја омогућава да се његови ресурси посматрају и димензионишу одвојено од ресурса потребних за активну комуникацију.

У пракси је посебно значајно управљање животним циклусом снимања. Потребно је разликовати почетак снимања, активно трајање снимања и успешно или неуспешно завршавање процеса. Додатни проблем представљају привремени прекиди, откази медијских токова или недостатак

распоживих ресурса. Систем мора бити пројектован тако да не претпоставља да је захтев за снимање истовремено и доказ да је снимак успешно креиран и сачуван.

За трајно чување резултујућих видео датотека употребљен је модел објектног складиштења заснован на S3 API-ју. S3 представља стандардизовани програмски модел за рад са објектима у систему објектног складиштења, при чему се подаци организују кроз bucket-ове и идентификују путем кључева објеката [28, 29]. За разлику од класичног система датотека, објектно складиште је намењено раду са великим бројем независних објеката и омогућава приступ садржају преко програмског интерфејса.

У систему видео конференција овај модел је погодан јер се видео снимци по својој природи могу посматрати као велики независни објекти. Мета-подаци о снимку могу се чувати одвојено од његовог бинарног садржаја, док се сам видео запис смешта у објектно складиште. Ово раздвајање омогућава да релациона база буде задужена за пословне податке, док је објектно складиште прилагођено чувању великих бинарних датотека.

MinIO представља само-хостовану имплементацију компатибилну са S3 моделом и омогућава да се исти концепт објектног складиштења користи у локалном и развојном окружењу [28, 29, 30]. Његова улога у оваквој архитектури значајна је јер омогућава локално тестирање целокупног тока снимања, складиштења и преузимања без зависности од спољног провајдера. Истовремено, коришћење S3 компатибилног API-ја смањује зависност апликације од конкретног произвођача објектног складишта.

Посебно важан концепт код приступа снимцима представљају presigned URL адресе. Препотписан захтев омогућава да клијент добије привремено право приступа одређеном објекту без директног излагања credentials-а за складиште [28, 29, 34]. Сервер генерише потписани захтев за конкретан ресурс, а клијент затим користи добијену адресу за преузимање садржаја. На овај начин се ауторизација одвија пре издавања адресе, док само преузимање не захтева да клијент поседује главне приступне податке складишта.

Временско ограничење presigned URL адресе представља важну безбедносну меру. Адреса није трајна дозвола већ временски ограничено право приступа конкретном ресурсу. Истек адресе не уклања објекат из складишта, већ спречава даљу употребу тог конкретног начина приступа. Због тога се животни циклус приступних адреса и животни циклус самих објеката морају посматрати као два одвојена механизма [28, 30, 34].

Приступ објектном складишту мора бити ограничен и на нивоу сервиса. У развојном окружењу могу се користити једноставнија правила приступа ради лакшег подешавања, али таква правила не треба преносити у продукционо окружење. Јавни упис и читање објеката, као и дељење административних credentials-а, представљају непотребно проширивање површине напада. За продукциону употребу погоднији је модел у коме је bucket приватан, а приступ се одобрава само сервисима који за то имају оправдану потребу [28, 30].

Поред контроле приступа, трајно складиштење видео записа захтева и оперативне политике. Снимци могу заузети велики количину простора, због чега је потребно планирати резервне копије, задржавање података, евентуално аутоматско брисање након одређеног временског периода, као и праћење потрошње простора. У зависности од захтева система могу бити релевантни и versioning, шифровање и механизми опоравка од губитка података.

Укупна архитектура снимања тако се може посматрати као ланац више технологија: LiveKit обезбеђује активну медијску сесију, Egress преузима и обрађује медијски садржај, а S3 компатибилно

складиште обезбеђује трајно чување резултујуће датотеке [10, 11, 28, 29]. Овакво раздвајање омогућава да се свака од наведених функција скалира и контролише независно од осталих делова система.

3.4. Аутентификација, ауторизација и управљање токенима

Безбедност система видео конференција не може се свести на проверу идентитета корисника. Поред аутентификације, неопходна је ауторизација, односно одређивање шта конкретни корисник сме да ради након успешне пријаве. Додатни изазов представља чињеница да један систем може користити више различитих механизма контроле приступа. У системима за видео конференције могу се раздвојити идентитет корисника, приступ пословним ресурсима, приступ медијској соби и приступ спољном складишту.

За управљање идентитетима употребљен је Keycloak, систем за управљање идентитетима и приступом који подржава OAuth 2.0 и OpenID Connect [6, 7, 8, 27]. Његова улога је да централизује процесе регистрације, пријаве, управљања корисничким подацима, промене и ресетовања лозинке, као и издавања токена. Оваква архитектура омогућава да пословна апликација не мора самостално да имплементира целокупан механизам за управљање идентитетом.

OpenID Connect проширује OAuth 2.0 механизмима за идентификацију корисника и размену података о њиховом идентитету [6, 7, 8]. У овом моделу клијент не мора сам да обрађује корисничке лозинке. Аутентикација се одвија у систему за управљање идентитетима, а апликација након успешног процеса добија доказ о идентитету у облику токена. На тај начин се одговорност за управљање корисничким налозима одваја од пословне логике апликације.

Keycloak организује идентитете и приступ путем концепата као што су realm, client и identity provider [27]. Realm представља административни простор у коме се управља корисницима и политиком идентитета, док client представља апликацију која користи услуге система за аутентификацију. Identity provider механизам омогућава повезивање са спољним системима идентитета без промене основног модела аутентификације у апликацији.

За аутентификацију веб клијента употребљен је Authorization Code ток са PKCE механизмом. OAuth 2.0 дефинише модел у коме клијент посредством ауторизационог сервера добија приступни токен, при чему се сам ток ауторизације одвија одвојено од пословне логике апликације [5, 6]. PKCE (Proof Key for Code Exchange) додаје заштиту која је посебно значајна за јавне клијенте, као што су апликације које раде у веб прегледачу.

Основна идеја PKCE механизма јесте да клијент приликом започињања ауторизационог процеса генерише тајни код који не напушта клијентски контекст. Од њега се изводи вредност која се шаље ауторизационом серверу, док се оригинална вредност користи касније, током размене ауторизационог кода за токен [5, 6]. На овај начин ауторизациони код сам по себи није довољан за добијање токена ако нападач нема и одговарајући верификатор.

У оквиру истог механизма значај има и state вредност. Њена сврха је да повеже одговор ауторизационог сервера са конкретно започетим процесом аутентификације. Провера state вредности смањује ризик од прихватања неочекиваног callback-а и представља важан део заштите ауторизационог тока [5, 6].

Конфигурација OpenID Connect клијента захтева ограничење redirect URI адреса, web origins и scope-ова. Прешироко дефинисана правила могу омогућити злоупотребу callback механизма или повећати број места на којима се токени могу користити. Због тога се конфигурација клијента мора

посматрати као део безбедносног модела, а не само као технички услов за успостављање аутентификације [6, 8].

Поред локалне регистрације, OpenID Connect архитектура омогућава интеграцију спољних провајдера идентитета. Keycloak Identity Broker омогућава интеграцију спољних провајдера идентитета, укључујући могућност пријаве путем Google налога [27]. Код овог приступа Google није непосредни део пословне логике апликације. Keycloak преузима улогу посредника, реализује комуникацију са спољним провајдером и резултат пријаве представља у истом OpenID Connect моделу који се користи и за локалну аутентификацију.

Предност оваквог приступа огледа се у јединственом моделу обраде идентитета. Без обзира на то да ли је корисник аутентификован локалним credentials-има или путем спољног провајдера, серверска страна може да обрађује исти тип идентитетског токена. На тај начин се смањује потреба за увођењем засебних правила у пословне механизме приступа.

При повезивању спољног налога посебно је важно контролисати начин повезивања постојећих и нових идентитета. Само поклапање електронске адресе није довољно да би се безусловно извршило аутоматско повезивање налога, јер погрешно подешавање овог процеса може довести до преузимања идентитета. Због тога Keycloak омогућава контролу first broker login тока и правила по којима се спољни идентитет повезује са већ постојећим корисником [27].

3.4.1. JWT токени и раздвајање права приступа

JWT (JSON Web Token) представља формат за пренос потписаних података између страна које учествују у комуникацији. Иако је садржај JWT-а читљив уколико се само декодира, његова безбедносна вредност не потиче из тајности садржаја, већ из проверљивог потписа и правила по којима сервер прихвата токен [4, 7]. Због тога се JWT не сме третирати као обичан JSON објекат који клијент може самостално да прогласи валидним.

Токен издат у оквиру система за управљање идентитетима може садржати више claim-ова који описују корисника и услове важења токена. Међу њима се могу налазити subject, email, issuer и временски подаци као што су време издавања и време истека. Сервер мора проверити да ли је токен издала очекивана страна, да ли је потпис валидан и да ли су временска ограничења испуњена [4, 7].

У систему видео конференција посебно је важно да се не мешају различити типови токена. Идентитетски токен који доказује идентитет корисника пред апликационим сервером није исто што и токен за улазак у конкретну медијску собу. Слично томе, API кључ који служи машинској интеграцији не би требало изједначавати са токеном за корисничку аутентификацију. Раздвајање ових механизма представља важну безбедносну границу јер различити ресурси имају различите услове приступа и различит животни циклус [4, 7, 33].

LiveKit токен носи контекст медијске сесије, што може обухватати идентитет учесника, идентификатор собе и дозволе за објављивање и пријем медијских токова [9, 10]. Такав токен није намењен приступу пословним подацима апликације. Његова сврха ограничена је на медијску комуникацију за коју је издат.

Издвојеност медијског токена посебно је значајна у систему у коме постоји могућност приступа састанку без креирања корисничког налога. Гост може добити временски и функционално ограничену дозволу за учешће у конкретној соби, без стицања права да приступа административним или пословним ресурсима апликације. На тај начин могуће је истовремено подржати гостујући приступ и задржати контролу над осетљивим операцијама.

API кључ представља другачији модел ауторизације који је погодан за програмски приступ. За разлику од корисничког JWT-а, API кључ се везује за интеграцију или клијента који треба да приступа одређеним операцијама. Због тога се његов животни циклус мора посматрати кроз издавање, чување, опозив и ротацију. Оригинална вредност кључа не треба трајно да се чува у бази у облику у коме би касније могла директно да се употреби. Један од погодних приступа јесте чување хеша кључа и приказ оригиналне вредности само у тренутку креирања [53].

Приликом провере API кључа, примљена вредност се упоређује са сачуваном репрезентацијом, чиме се смањује последица евентуалног откривања садржаја базе. Овај механизам не треба мешати са шифровањем, јер хеш представља једносмерну трансформацију намењену провери вредности, а не њеном каснијем враћању.

Ауторизација се не завршава успешном провером токена. Након утврђивања идентитета неопходно је проверити да ли конкретни корисник има право приступа конкретном ресурсу и конкретној операцији. Поседовање валидног JWT-а, на пример, не значи аутоматски да корисник може да измени сваки постојећи састанак. Између идентитета и права приступа мора постојати пословна провера власништва или одговарајуће улоге [4, 7, 33].

3.5. Серверске технологије и перзистенција података

Серверски део система заснива се на технологијама које заједно обезбеђују извршавање пословне логике, комуникацију путем HTTP захтева, перзистенцију података и брзу размену привременог стања. Ове технологије нису непосредно одговорне за медијски пренос, али обезбеђују неопходан апликациони слој у коме се управља подацима о састанцима, корисницима, учесницима и другим пословним концептима.

Java представља статички типизирани објектно оријентисани програмски језик који се широко користи за развој серверских апликација [12]. У систему је коришћена Java 21 као верзија извршног окружења, што је усаглашено и са процесом изградње и паковања апликације [32]. Статичка типизација омогућава да се значајан број неусклађености открије већ током компајлирања, пре извршавања програма. Ово је посебно значајно на границама између различитих слојева апликације, где се преносе структурирани подаци.

Java обезбеђује богат стандардни модел типова и механизма рада са изузецима, идентификаторима, временом и конкурентним извршавањем [12]. Савремене верзије језика омогућавају коришћење компактнијих структура за пренос података и јасније изражавање доменских концепата. У систему видео састанака овакве карактеристике су значајне због потребе за истовременом обрадом HTTP захтева, догађаја повезаних са медијском сесијом и позадинских операција.

Spring Boot представља оквир за развој самосталних Spring апликација који значајно поједностављује конфигурисање и покретање серверског окружења [13, 50]. Његова аутоконфигурација омогућава да се велики број уобичајених компоненти повеже уз ограничену количину конфигурационог кода, док механизми dependency injection-а омогућавају раздвајање компоненти и њихових зависности.

Важна карактеристика Spring Boot-а јесте могућност груписања сродних функционалности кроз starter зависности. На тај начин серверска апликација може укључити веб слој, безбедносне механизме, рад са базом и слање електронске поште без потребе за ручним повезивањем већег броја

појединачних библиотека [13, 50]. Истовремено, конфигурационе вредности се могу издвојити из изворног кода, што је значајно за различита развојна и продукциона окружења.

3.5.1. REST и HTTP интерфејс

REST (Representational State Transfer) представља архитектурни стил за пројектовање мрежних система који је формализовао Roy Fielding у оквиру докторске дисертације из 2000. године [51]. REST није протокол нити готова библиотека, већ скуп ограничења којима се дефинише начин организације комуникације између клијента и сервера. Ова ограничења обухватају раздвајање клијента и сервера, stateless принцип, јасно дефинисан интерфејс и коришћење представљања ресурса.

У пракси се REST најчешће реализује над HTTP протоколом, при чему се ресурси представљају кроз URI адресе, а операције изражавају HTTP методама. Оваква организација омогућава јасно раздвајање одговорности између клијента и сервера и представља погодан модел за реализацију пословног API-ја [14, 51].

У систему видео конференција REST интерфејс погодан је за операције које не захтевају непрекидан медијски ток. Креирање и измена података о састанцима, приступ подацима о корисницима, административне операције и комуникација са другим сервисима могу бити организовани путем HTTP захтева. Овим се пословни API логички раздваја од WebRTC комуникације у реалном времену.

Један од значајних принципа REST приступа јесте коришћење одговарајућих HTTP статусних кодова. Сервер треба јасно да разликује успешан захтев, невалидне улазне податке, недостатак ауторизације, непостојећи ресурс и интерну грешку. На овај начин клијентска страна може да обради одговор без познавања интерне имплементације серверске апликације [14].

За размену података између клијента и сервера погодан је JSON формат. Одвајање јавног HTTP модела од интерног доменског модела омогућава да се структура API-ја мења независно од начина на који су подаци перзистирани. Ово представља важан аспект одрживости API-ја, јер јавни уговор не треба директно да зависи од структуре базе података [4, 14].

3.5.2. JPA и Hibernate

JPA (Java Persistence API) дефинише стандардизовани модел перзистенције Java објеката у релационим базама података [15, 16]. JPA одређује правила и интерфејсе за мапирање објектног модела на релациони модел, док Hibernate представља конкретну имплементацију тог стандарда [15, 16].

Основна предност оваквог приступа огледа се у томе што се рад са релационим подацима може организовати на начин који је ближи објектном моделу програмског језика. Мапирање ентитета, примарних и страних кључева и веза између објеката омогућава да апликација ради са доменским објектима, док Hibernate у позадини реализује одговарајуће SQL операције.

Истовремено, употреба ORM приступа не уклања потребу за разумевањем релационих база. Начин учитавања повезаних података, број генерисаних упита, индекси и трансакционе границе и даље директно утичу на перформансе. Посебно су значајни fetch стратегије, којима се одређује када ће повезани подаци бити учитани, као и cascade правила и orphanRemoval, који утичу на понашање приликом уписа и брисања повезаних ентитета [15, 16].

За системе видео састанака релациони модел погодан је за трајно чување података који представљају пословно стање. Информације о састанцима, учесницима, корисницима и приступним

механизмима могу бити моделоване кроз повезане табеле, док ORM слој смањује количину нискониво кода потребног за њихов приступ.

3.5.3. PostgreSQL

PostgreSQL је објектно-релациони систем за управљање базама података који подржава трансакције, индексе, различите типове података и JSONB структуре [17]. У контексту система видео конференција, PostgreSQL је погодан за податке који морају остати доступни и након завршетка појединачне медијске сесије.

Релациона база омогућава дефинисање ограничења интегритета и односа између података. На пример, везе између састанка и његових учесника могу бити формализоване страним кључевима, чиме се смањује могућност настанка неконзистентних записа. Транзакције додатно омогућавају да се више повезаних промена изврши као једна логичка целина [17].

Подршка за JSONB је значајна у ситуацијама у којима део података има полуструктурирану природу. На овај начин се могу комбиновати предности релационог модела са могућношћу чувања сложенијих JSON структура. Ипак, коришћење JSONB не треба да буде замена за правилно моделовање података који представљају стабилне и често претраживане пословне концепте.

Перформансе PostgreSQL базе зависе и од правилног избора индекса и начина извршавања упита. Индекси могу значајно убрзати претрагу над великим бројем записа, али истовремено повећавају цену уписа и заузеће простора. Због тога избор индекса мора бити повезан са стварним шаблонима приступа подацима, а не са механичким индексирањем свих колона [17].

3.5.4. Redis

Redis је in-memory систем за структуре података и брзу размену стања. [18] У систему га користе LiveKit и Egress за координацију сервиса, док је Spring Data Redis доступан backend-у. Compose обезбеђује Redis сервис, named volume и healthcheck. У продукцији је потребно ограничити приступ, конфигурисати аутентификацију и планирати persistence политику. Redis у овој архитектури служи као брза координациона тачка LiveKit екосистема, а не као замена за PostgreSQL записе који представљају трајно пословно стање.

3.6. Клијентске технологије

Клијентска страна система намењена је приказу корисничког интерфејса, комуникацији са серверским API-јем и непосредном управљању медијском сесијом. За њен развој користе се технологије које омогућавају компонентну организацију интерфејса, статичку типизацију, реактивну обраду асинхроних догађаја и прилагођавање различитим величинама екрана.

Angular представља развојни оквир заснован на компонентном моделу који обезбеђује механизме за организовање корисничког интерфејса, усмеравање, dependency injection и комуникацију са спољним сервисима [19]. Оваква архитектура погодна је за апликације које садрже више страница и стања, јер омогућава раздвајање визуелних елемената, навигације и функционалне логике.

Angular је погодан за развој корисничких интерфејса система за видео конференције, укључујући странице за приступ састанцима, управљање корисничким налозима и интеракцију током видео позива. Његова компонентна организација погодна је за издвајање различитих делова интерфејса,

док routing омогућава да се појединачни кориснички токови представе независним URL адресама [19].

Angular подржава standalone компоненте, што омогућава организацију апликације са мање посредних модула. Овај приступ је погодан за савремене Angular апликације јер се зависности компоненте могу дефинисати непосредно уз њену декларацију. Поред тога, Angular обезбеђује сопствени систем реактивности кроз Signals, који се може комбиновати са старијим и шире коришћеним реактивним моделом заснованим на RxJS-у [19, 21].

TypeScript представља типизирани надскуп JavaScript језика који уводи статичке типове, интерфејсе и друге механизме за контролу структуре програма [20]. У сложенијим клијентским апликацијама статичка типизација је значајна јер велики број грешака може бити откривен већ током изградње апликације. Ово је посебно важно код комуникације са REST API-јем, где неслагање назива или типова поља може довести до грешке у време извршавања ако није благовремено уочено.

Међутим, TypeScript типови не представљају безбедносну границу. Клијентски JavaScript код се извршава у окружењу које корисник контролише, па се типови и интерфејси не могу користити као замена за серверску валидацију. Сервер мора независно да проверава све улазне податке без обзира на то што су они претходно типизирани на страни клијента [20].

3.6.1. RxJS и Angular Signals

RxJS представља библиотеку за реактивно програмирање засновано на токовима података и асинхроним догађајима [21]. У оквиру Angular екосистема користи се за HTTP комуникацију, обраду асинхроних резултата и комбинацију више независних токова.

Посебна вредност реактивног приступа долази до изражаја у системима за видео конференције јер се стање комуникације непрекидно мења. Повезивање, реконекција, појава новог учесника, промена доступности медијског тока и завршетак снимања представљају догађаје који утичу на кориснички интерфејс. Реактивни модел омогућава да се такви догађаји обрађују без ручног синхронизовања великог броја међусобно зависних променљивих.

Angular Signals представљају механизам за реактивно локално стање у Angular апликацијама [19]. За разлику од RxJS-а, који је пре свега намењен токовима и асинхроним догађајима, Signals су погодни за представљање тренутне вредности коју треба аутоматски одразити у корисничком интерфејсу. У контексту видео позива то може обухватити тренутно стање повезаности, број учесника, статус снимања или стање медијских контрола.

Комбиновање RxJS-а и Signals омогућава раздвајање два различита проблема: асинхроног тока догађаја и тренутног стања приказа. Оваква подела је посебно значајна код реконекције, где долазних догађаја могу привремено мењати стање интерфејса, али корисник треба да види једну јасну и конзистентну слику тренутног стања комуникације [19, 21].

3.6.2. Tailwind CSS и Lucide

Tailwind CSS омогућава изградњу responsive корисничких интерфејса применом utility класа, док Lucide обезбеђује скуп векторских иконица погодних за интерактивне контроле [22, 23]. Визуелни приказ не сме бити једини носилац информације; контроле морају имати текстуалне или ARIA (Accessible Rich Internet Applications) описе. Tailwind CSS и Lucide омогућавају да контроле састанка остану визуелно доследне и прилагођене различитим ширинама екрана, уз услов да приступачност не зависи само од иконице.

3.7. Контејнеризација и инфраструктурне технологије

Docker представља технологију за паковање апликација и њихових зависности у изолована извршна окружења, односно контејнере [24]. Контејнер омогућава да се апликација покрене са унапред дефинисаним окружењем без потребе да се све њене зависности ручно инсталирају на хост оперативни систем.

За систем који се састоји од већег броја логички одвојених сервиса, коришћење контејнера поједностављује организацију развојног окружења. Серверска апликација, клијентска апликација, база података, систем за медијску комуникацију, систем за складиштење и остали пратећи сервиси могу се покретати као независни контејнери, са сопственим мрежним подешавањима и ресурсним ограничењима [24].

Docker Compose представља алат за декларативно дефинисање и покретање више повезаних контејнера путем једне YAML конфигурације [25]. Уместо да се сваки сервис покреће и конфигурише појединачно, Compose омогућава да се мрежа, зависности, volume-и, променљиве окружења и остали параметри опишу на једном месту.

У системима који користе више инфраструктурних компоненти овакав приступ је значајан због поновљивости. Ново развојно окружење може бити подигнуто на основу исте конфигурације, чиме се смањује зависност од локалних подешавања појединачног рачунара. Истовремено, Compose конфигурација представља документовани опис односа између сервиса.

Посебну пажњу у контејнеризованим системима захтевају тајне, јавни портови и верзије слика. Тајни подаци не би требало да буду уграђени у слике нити да се неконтролисано уписују у конфигурационе датотеке које се деле са другим корисницима. Јавне портове је потребно ограничити на оне који су стварно неопходни, док је препоручљиво користити прецизно одређене верзије слика уместо покретних ознака као што је latest [24, 25].

Контејнеризација је погодна за повезивање апликационих и инфраструктурних компоненти у системима који се састоје од већег броја сервиса. На тај начин се обезбеђује да серверска апликација, база података, као и остали сервиси могу радити у контролисаном и међусобно повезаном окружењу.

Nginx представља веб сервер и reverse проху који може преузети више различитих улога у веб архитектури [26]. У SPA апликацијама може се користити за сервирање статичких датотека клијентске апликације, док истовремено може прослеђивати API захтеве према серверској апликацији.

Reverse проху модел омогућава да спољни клијент види једну улазну тачку, док се унутар инфраструктуре захтеви прослеђују одговарајућим сервисима. Ово поједностављује конфигурацију клијентске стране јер се API и кориснички интерфејс могу излагати под истим доменским контекстом. Уједно се добија централизовано место за конфигурацију проху заглавља, ограничење доступних путања и TLS механизме [26].

Код SPA апликација важан је и fallback механизам за руте. Сервер мора бити у стању да за директан захтев ка одређеној клијентској путањи врати основни HTML документ апликације, након чега Angular Router преузима даљу обраду. Овај приступ омогућава правилно функционисање клијентских рута чак и када корисник директно отвори одређену URL адресу [26].

3.8. Mailpit и SMTP

SMTP је стандардни протокол намењен преносу електронске поште између клијентских и серверских система, као и између самих сервера за електронску пошту [31]. Његова основна улога је

прослеђивање порука од пошиљаоца ка одговарајућем mail серверу и даљи пренос до сервера примаоца. Поред самог протокола, системи за електронску пошту обухватају и механизме за аутентификацију, шифровање комуникације и контролу доставе, који доприносе поузданости и безбедности преноса порука.

Mailpit је развојни SMTP сервер намењен тестирању система који шаљу електронску пошту. За разлику од продукционих mail сервера, Mailpit не прослеђује поруке стварним примаоцима, већ их прихвата и чува у оквиру локалног окружења, где су доступне кроз веб интерфејс. На тај начин могуће је проверити садржај порука, заглавља и податке о примаоцу без њиховог стварног слања.

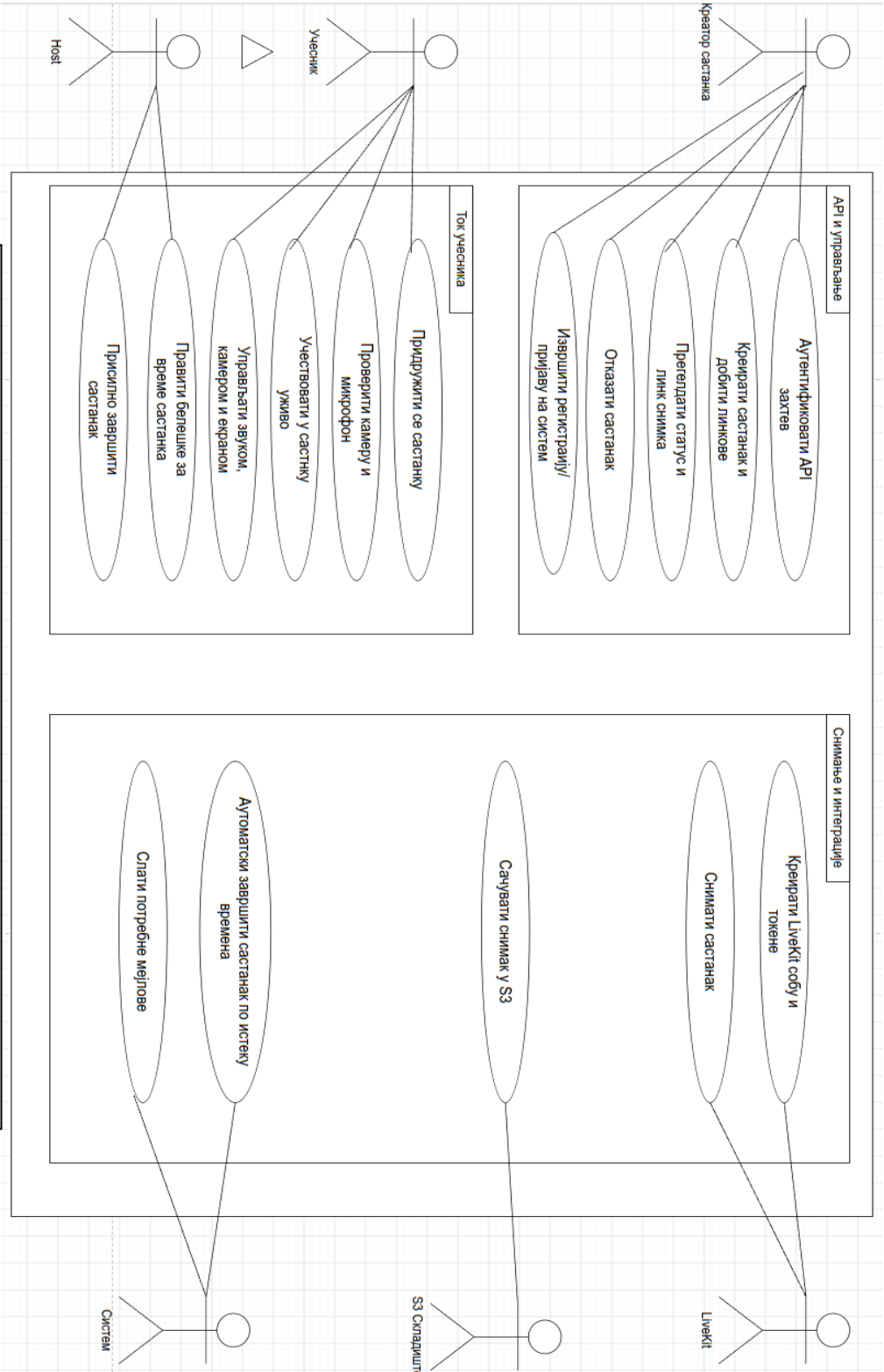
4. Спецификација и модел система

Ово поглавље описује систем на моделском нивоу. Приказују се актери, функције, подаци и токови који су неопходни за разумевање решења, док се конкретни Java и TypeScript детаљи остављају за наредно поглавље.

Систем је пројектован као веб платформа за реализацију видео састанака у реалном времену, са раздвојеним клијентским, апликационим и медијским слојем. Клијентски слој обезбеђује интеракцију са корисником, апликациони слој управља пословном логиком и подацима, док је комуникација у реалном времену издвојена у посебан медијски слој заснован на SFU архитектури. За трајно чување пословних података предвиђена је релациона база, док се медијски садржај издваја у објектно складиште. Додатне компоненте обезбеђују аутентификацију, координацију сервиса и електронску пошту, а сви делови система организовани су као међусобно повезани сервиси.

4.1. Актери и случајеви коришћења

На слици 4.1 приказан је дијаграм случајева коришћења система. Креатор састанка представља спољну особу или систем који може да креира и прати састанке. Учесник приступа соби преко линка и не мора да поседује налог. Host има проширену дозволу за завршетак састанка, док пријављени власник управља заказаним састанцима и профилем. LiveKit, S3 складиште, и системски сервис моделовани су као спољни системски актери.



Слика 4.1 — UML дијаграм случајева коришћења

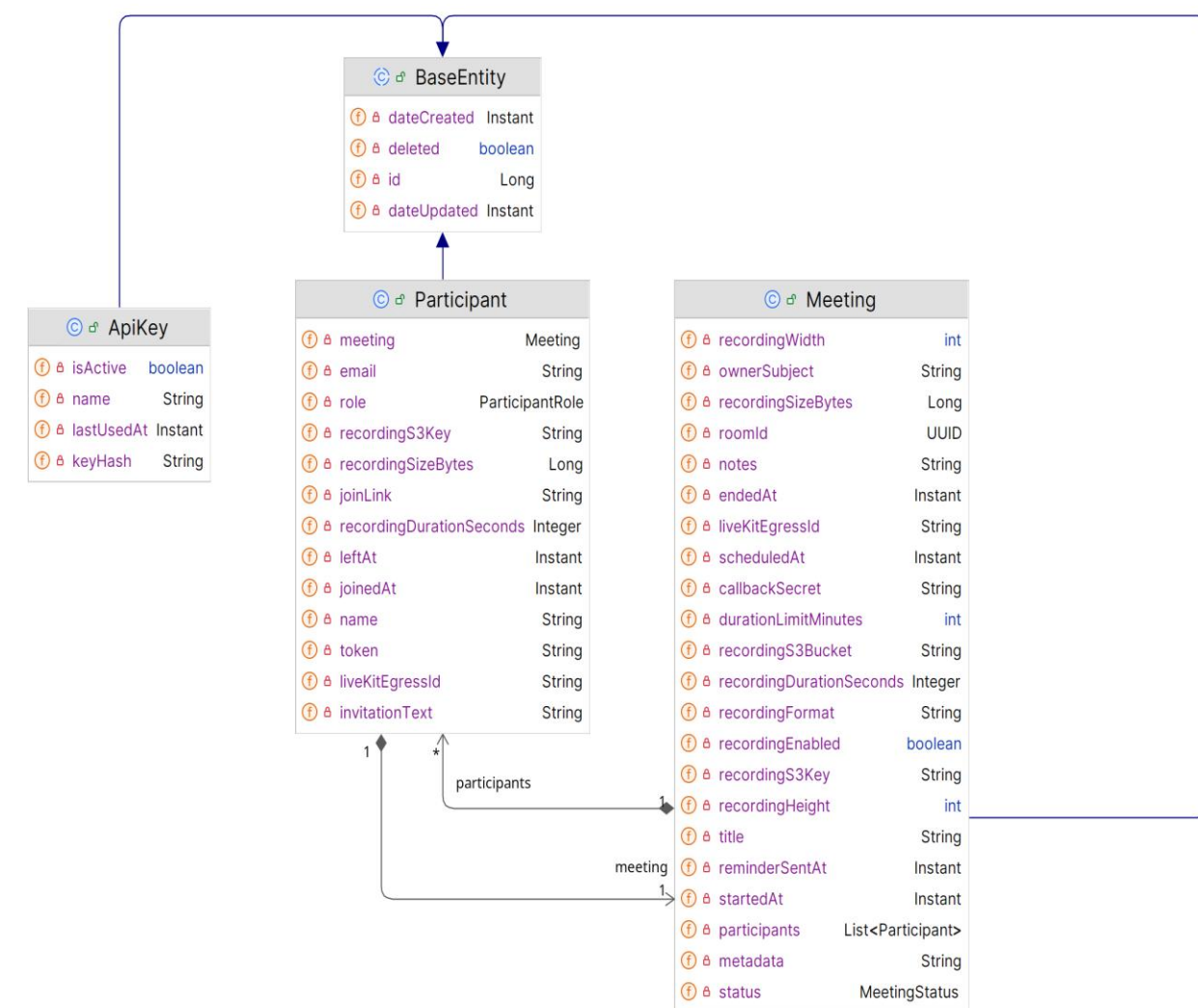
Извор: Ауторски UML дијаграм направљен помоћу DrawIO софтвера [54].

4.2. Доменски модел

Доменски модел се заснива на састанку као централном објекту. Састанак има више учесника, статус, временске ознаке, конфигурацију снимања и референце ка снимку. API кључ је одвојен ентитет који представља интеграцију спољног система. BaseEntity обезбеђује заједнички идентификатор, датуме и логичко брисање. Табела 4.1 представља преглед намене и најважнијих поља сваке класе доменског модела, док је на слици 4.2 приказан класни дијаграм система.

Тип	Мапирање	Кључни атрибути	Намена
BaseEntity	Mapped superclass	id, dateCreated, dateUpdated, deleted	Заједничка основа доменских ентитета која обезбеђује идентификатор, датуме креирања и измене и логичко брисање.
Meeting	JPA entity	roomId, title, status, schedule, recording, metadata, notes, ownerSubject	Централни ентитет који представља видео састанак и чува податке неопходне за његово управљање, заказивање и снимање.
Participant	JPA entity	meeting, name, email, role, token, joinLink, presence, recording	Ентитет који представља учесника састанка и његове податке о улози, приступу, присуству и снимању.
ApiKey	JPA entity	name, keyHash, isActive, lastUsedAt	Ентитет намењен управљању приступом спољних система путем API кључева, уз чување њихове безбедне репрезентације и статуса.

Табела 4.1 — Доменске класе система, њихови најважнији атрибути и намене



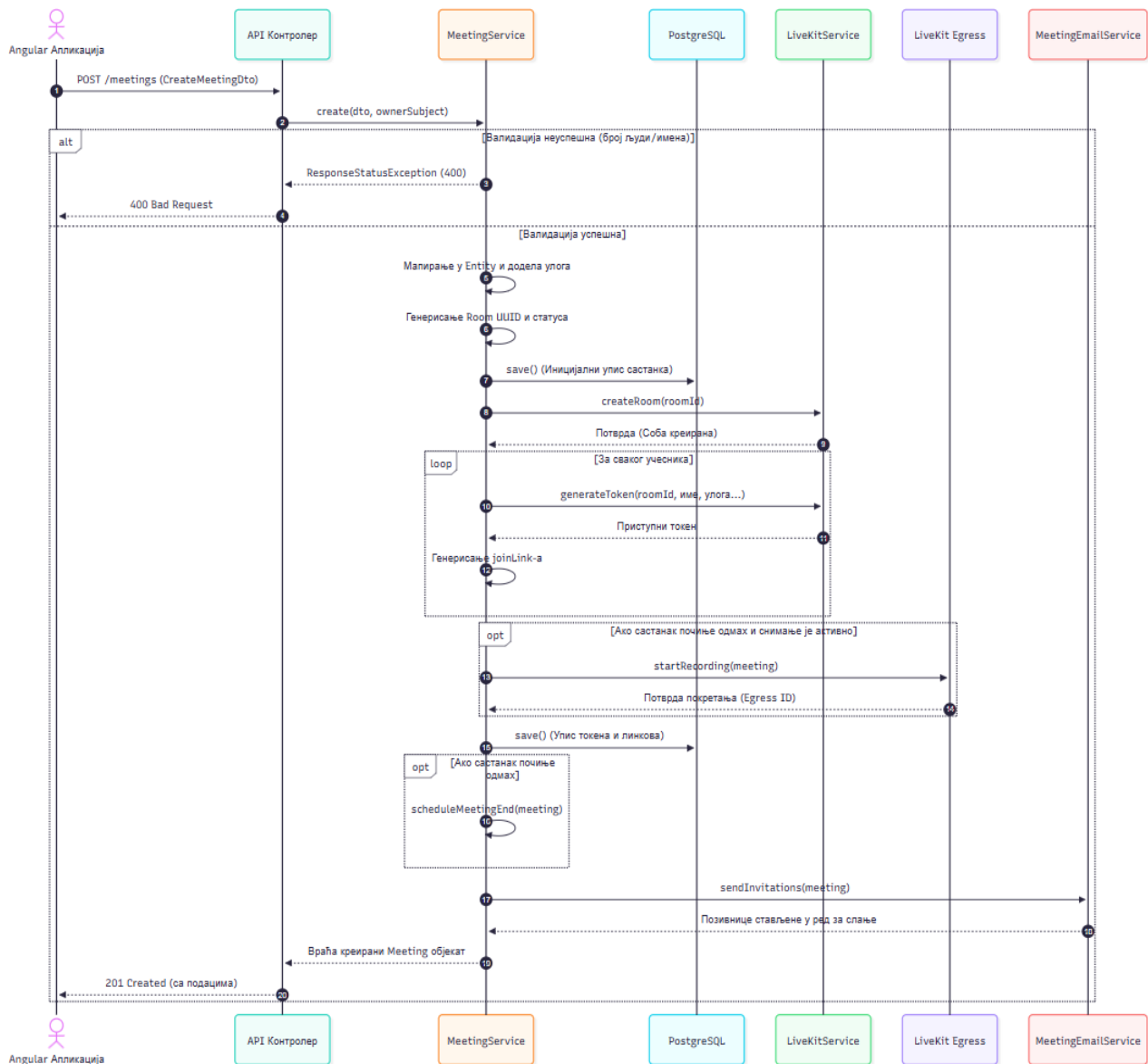
Слика 4.2 — Дијаграм класа

Извор: IntelliJ IDEA алат за генерисање класних дијаграма [55].

4.3. Секвенца креирања састанка

Процес креирања састанка представља комплексан доменски ток који захтева прецизну оркестрацију између клијентске апликације, централног позадинског система и екстерних сервиса, што је моделовано дијаграмом секвенце на слици 4.3. Ток започиње валидацијом клијентског захтева, након чега систем учесницима додељује одговарајуће улоге и генерише јединствени идентификатор комуникационе собе. Ово почетно стање се бележи у релационој бази података ради очувања конзистентности. Одмах затим, систем комуницира са медијским сервером како би инстанцирао изоловану собу. За сваког учесника се итеративно захтева генерисање криптографски потписаног сигурносног токена, који дефинише идентитет и приступна права. На основу ових токена, систем конструише јединствене приступне везе неопходне за ауторизован улазак корисника у састанак.

Након успешне припреме комуникационог слоја, систем евалуира временске параметре и корисничка подешавања како би покренуо пратеће процесе. Уколико се захтева архивирање сесије, систем асинхронно иницира комуникацију са екстерним сервисом за снимање, чиме се избегава блокирање главног тока извршавања. Целокупно стање састанка, обogaћено приступним подацима, затим се трајно ажурира у бази података. Паралелно са тим, систему за обавештавања се делегира задатак дистрибуције позивница путем електронске поште, што се такође изводи у позадини ради оптимизације одзива. Када су сви ресурси успешно алоцирани, апликативни интерфејс клијенту враћа комплетан модел новокреираног састанка, чиме је процес успешно завршен.



Слика 4.3 — Дијаграм секвенце процеса креирања састанка

Извор: Дијаграм израђен помоћу Mermaid софтвера [62].

4.4. Статуси и животни циклус

Састанак може бити `scheduled`, `in_progress`, `completed`, `cancelled`, `failed`, `callback_failed` или `recording_failed`. Прелаз из `scheduled` у `in_progress` дешава се када заказано време наступи или када се састанак креира за тренутни почетак. Завршетак може бити ручни, временски или узрокован

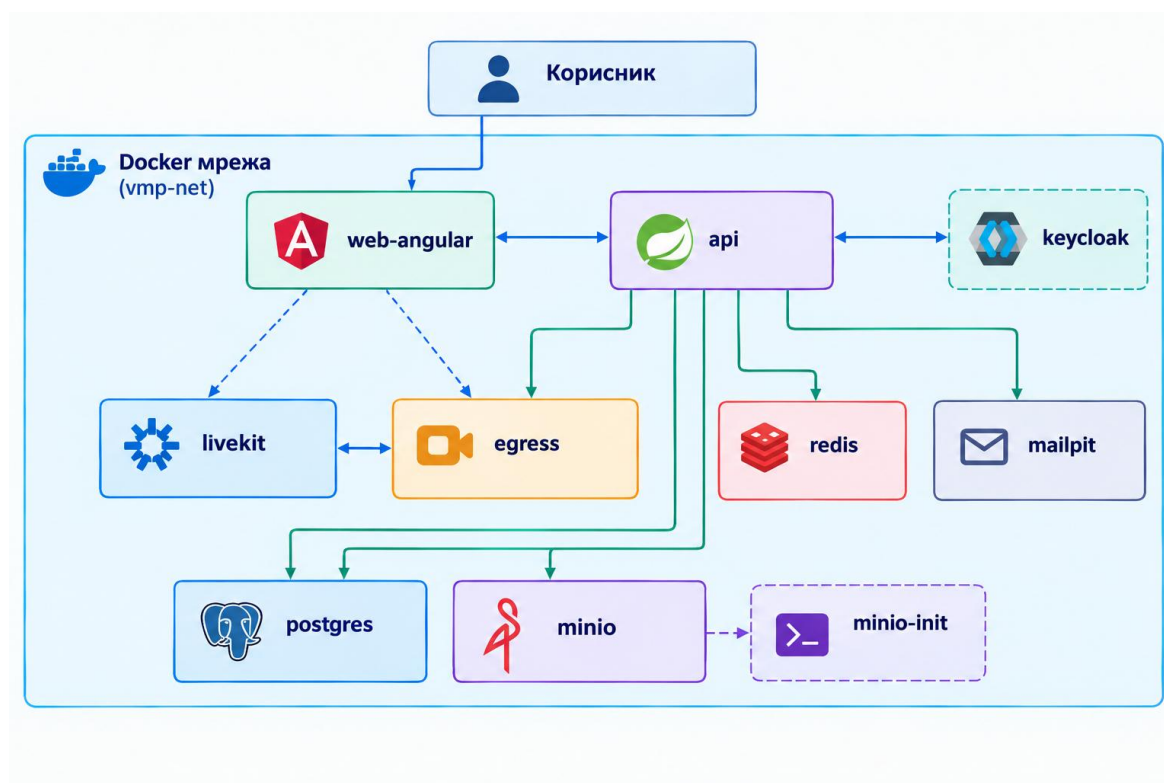
грешком. Статус снимања се разликује од статуса медијске собе јер снимак може још да се обрађује након што је корисник напустио собу.

4.5 Преглед Docker Compose сервиса

Ова табела приказује структуру и међусобне зависности сервиса који чине Docker Compose инфраструктуру система. Као што је приказано у Табели 4.2, сваки сервис има јасно дефинисану намену, портове преко којих комуницира са осталим компонентама и зависности неопходне за његово функционисање. Оваква организација омогућава изоловано покретање појединачних компоненти, али и њихово повезивање у јединствену инфраструктурну целину. На слици 4.4 је визуелни приказ свих сервиса унутар Docker Compose пројекта, као и њихови међусобни односи.

Сервис	Улога	Портови	Зависности
api	Spring Boot backend	5001/5000	PostgreSQL, Redis, LiveKit, MinIO, Mailpit, Keycloak
web-angular	Angular bundle и Nginx	3002/3000	API
livekit	SFU медијски сервер	7880,7881, UDP:50000–50200	Redis
egress	Снимање и export	Интерно	LiveKit, Redis, MinIO
postgres	Релациона база	5434/5432	Нема
redis	Координација и ephemeral state	6379	Нема
minio	S3-compatible object storage	9000, 9001	Нема
minio-init	Креирање bucket-a	Интерно	MinIO
mailpit	Развојни SMTP server/UI	1025, 8025	Нема
keycloak	Identity and access management	8080	Mailpit

Табела 4.2 — Топологија Docker Compose инфраструктуре



Слика 4.4 — Топологија Docker Compose инфраструктуре

Извор: Ауторска дијаграмска слика.

4.6 API модел

Табела 4.3 приказује основни модел REST API интерфејса система и обухвата крајње тачке одговорне за управљање састанцима, аутентикацију и обраду догађаја медијског сервера.

Endpoint	Намена
POST /meetings	Креирање састанка и учесника
GET /meetings/{roomId}	Приказ података о састанку
GET /meetings/{roomId}/access	Провера приступа преко линка
POST /meetings/{roomId}/end	Завршетак састанка
PUT /meetings/{roomId}	Измена заказаног састанка
POST /meetings/{roomId}/cancel	Отказивање заказаног састанка
GET /meetings/mine	Приказ састанака које је направио улоган корисник
POST /webhook/livekit	Пријем LiveKit догађаја
GET/PUT /auth/me	Читање и измена профила
POST /me/password-reset	Измена лозинке путем мејла

Табела 4.3 — Преглед кључних endpoint-а система

4.7 Безбедносни модел

Одсуство обавезног налога не значи одсуство безбедносних механизма. Безбедносни модел имплементиран у систему разликује приступ састанку од приступа функцијама за управљање системом. Идентитет и улога учесника садржани су у ограниченом LiveKit JWT токenu, док су административни API захтеви заштићени Keycloak JWT-ом или API кључем. LiveKit JWT има ограничену намену и не представља исти идентитет као Keycloak налог. API кључ представља интеграцију спољног клијента и у бази се чува као хеш. Ова подела омогућава да гост приступи конкретном састанку без налога, а да пријављени корисник добије додатне функције као што су профил, преглед сопствених састанака и њихово отказивање. [4, 7, 33]

4.8 Детаљна анализа ентитета Meeting

Ентитет Meeting представља централни пословни запис платформе. Он повезује захтев организатора са конкретном LiveKit собом, заказаним временом, ограничењем трајања, учесницима и резултатима снимања. Док медијски сервер води рачуна о тренутној комуникацији, Meeting остаје извор истине о томе шта је састанак, ко га је креирао и у ком се стању налази.

Основни атрибути су UUID идентификатор, roomId, title, description, startTime, duration, status и owner subject. Додатни параметри одређују да ли је снимање укључено, који callback треба обавестити и који су резултати композитног процеса. Веза са Participant записима омогућава да се број и улоге учесника чувају одвојено од општих података о соби.

Животни циклус записа почиње креирањем, наставља се заказаним или активним стањем, а завршава отказивањем, истеком или успешним завршетком. Прелази нису произвољни: отказан састанак не сме поново постати активан, а completed не треба поставити док су релевантни Egress резултати још непознати. Дефинисана стања и дозвољени прелази између њих обезбеђују да пословно стање састанка остане конзистентно и да се не могу извршити нелогичне операције. Статус састанка при томе представља независну целину у односу на статус снимања, јер обрада и чување резултата снимања могу трајати и након завршетка саме комуникационе сесије. [15, 16]

4.9 Детаљна анализа ентитета Participant

Participant описује једну особу у оквиру конкретног Meeting записа. Његова улога није да буде кориснички налог, већ да повеже позивницу, улазак у собу, понашање учесника током сесије и евентуални снимак издвојеног учесника. Зато један гост може постојати у систему иако није регистрован у Keycloak-у.

Запис садржи име и email за приказ и комуникацију, улогу host или guest, join token, join link, време уласка и изласка и идентификатор participant Egress процеса. Join token се користи за ограничен приступ соби и не треба га излагати у одговорима који нису намењени придруживању. Временски подаци омогућавају израчунавање трајања и проверу да ли је учесник заиста био присутан.

Улазак и излазак нису увек упарени једним синхроним захтевом. Корисник може изгубити мрежу, затворити картицу или се поново повезати, па се Participant стање мора повезати са LiveKit догађајима и обрадити идемпотентно. Посебно се разликује гостујућа дозвола за медијску собу од права организатора да мења Meeting.

Учешће у састанку обухвата различите сценарије, укључујући приступ више учесника, различите улоге, привремени прекид везе и поновно повезивање, као и издвојено снимање учесника. Овакво моделовање омогућава да се подаци о учеснику користе како за праћење његовог учешћа током састанка, тако и за повезивање са садржајем који настаје током и након медијске сесије.

4.10 Детаљна анализа ентитета ApiKey

ApiKey представља контролисани идентитет спољне интеграције која позива API без интерактивне Keycloak пријаве. То може бити други сервис или клијент коме су потребне ограничене операције, али не и приступ свим функцијама организатора. Због тога се API кључ моделује као пословни ресурс са сопственим статусом и временом последње употребе.

У запису се чувају назив, BCrypt keyHash, active ознака и lastUsedAt. Оригинална вредност кључа приказује се само у тренутку креирања и не може се накнадно прочитати из базе. При сваком успешном позиву ажурира се време употребе, што омогућава увид у активност и лакше откривање кључева који више нису потребни.

Ротација се спроводи креирањем новог кључа и деактивирањем старог, а не покушајем да се из хеша реконструише претходна вредност. Неважећи, непотпун или деактивиран Bearer кључ мора бити одбијен пре извршавања пословне операције. Поред тога, endpoint-и морају ограничити које радње интеграција уопште сме да позове.

API кључ представља засебан механизам за ауторизацију програмског приступа систему и омогућава контролисано коришћење API интерфејса без интерактивне пријаве корисника. Његов животни циклус обухвата издавање, коришћење, деактивирање и ротацију, при чему се приступ може ограничити на унапред дефинисане операције. Оваква организација омогућава раздвајање приступа намењеног спољним интеграцијама од корисничке аутентификације, уз могућност праћења употребе и благовременог укидања кључева који више нису потребни.

4.11 Доменски инваријанти

Доменски инваријанти су правила која морају важити без обзира на то да ли је захтев дошао из Angular интерфејса, API клијента, scheduler-а или webhook-а. Њихова сврха је да база и сервис не прихвате стање које би касније било немогуће доследно обрадити. Frontend провера побољшава корисничко искуство, али backend остаје коначна граница поверења.

Meeting мора имати јединствени идентификатор собе, важећи статус, позитивно трајање и смислено време почетка. Participant мора припадати конкретном Meeting-у, join подаци морају бити везани за ту собу, а број учесника мора задовољити услове за креирање и покретање састанка. ApiKey мора имати јединствену хеш вредност и јасно active/inactive стање.

Инваријанти се примењују на више места: DTO валидација одбија невалидну структуру, сервис проверава пословни контекст, а база преко ограничења и типова спречава део недозвољених стања. Ова подела не значи да је правило дуплирано без разлога; сваки слој спречава различиту врсту грешке на својој граници.

Примери негативних случајева су празан назив, један учесник када су потребна два, негативна резолуција снимања, невалидан callback URL, измена туђег састанка и покушај поновног активирања отказаног записа. Такви захтеви морају завршити јасним одговором без делимичног уписа података [13, 14, 15].

4.12 Модел грешака

Модел грешака одваја проблеме које може да исправи корисник од проблема који захтевају интервенцију сервиса. Неважећи улаз се представља као 400, неауторизован захтев као 401, недозвољена радња као 403, а конфликт са животним циклусом као 409. Недоступност базе, Keycloak-а, LiveKit-а или складишта припада класи зависничких грешака и не сме се маскирати као грешка форме.

Контролер преводи изузетке у стабилан HTTP и JSON одговор, док се детаљи за дијагностику уписују у серверски лог. Клијент добија поруку довољну за следећу радњу, али не добија stack trace, storage credentials, SQL упит или садржај интерне конфигурације. На тај начин се корисност поруке усклађује са принципом минималног откривања података.

Асинхрони догађаји имају посебан режим обраде. Webhook који је стигао два пута не треба да произведе два снимка, а привремено недоступан Mailpit не сме да поништи успешно креиран Meeting. За сваки случај одређује се да ли је дозвољен retry, да ли је потребан ручни опоравак и који статус остаје уписан у бази.

4.13 Модел претњи и границе поверења

Модел претњи полази од нападача који може да види или измени јавни join линк, пошаље произвољан HTTP захтев, промени JavaScript у сопственом прегледачу или понови пресретнуту поруку. Не претпоставља се да су вредности које долазе из frontend-а поуздане. Сервер зато сам проверава roomId, улогу, статус састанка, потпис токена и право приступа ресурсу.

Постоји више граница поверења: browser-API, API-Keycloak, API-LiveKit, Egress-MiniIO и backend-callback прималац. На свакој граници се мењају формат и значење података. Keycloak JWT није исто што и LiveKit JWT, а presigned URL није исто што и storage credential; мешање ових вредности би проширило последице евентуалног цурења.

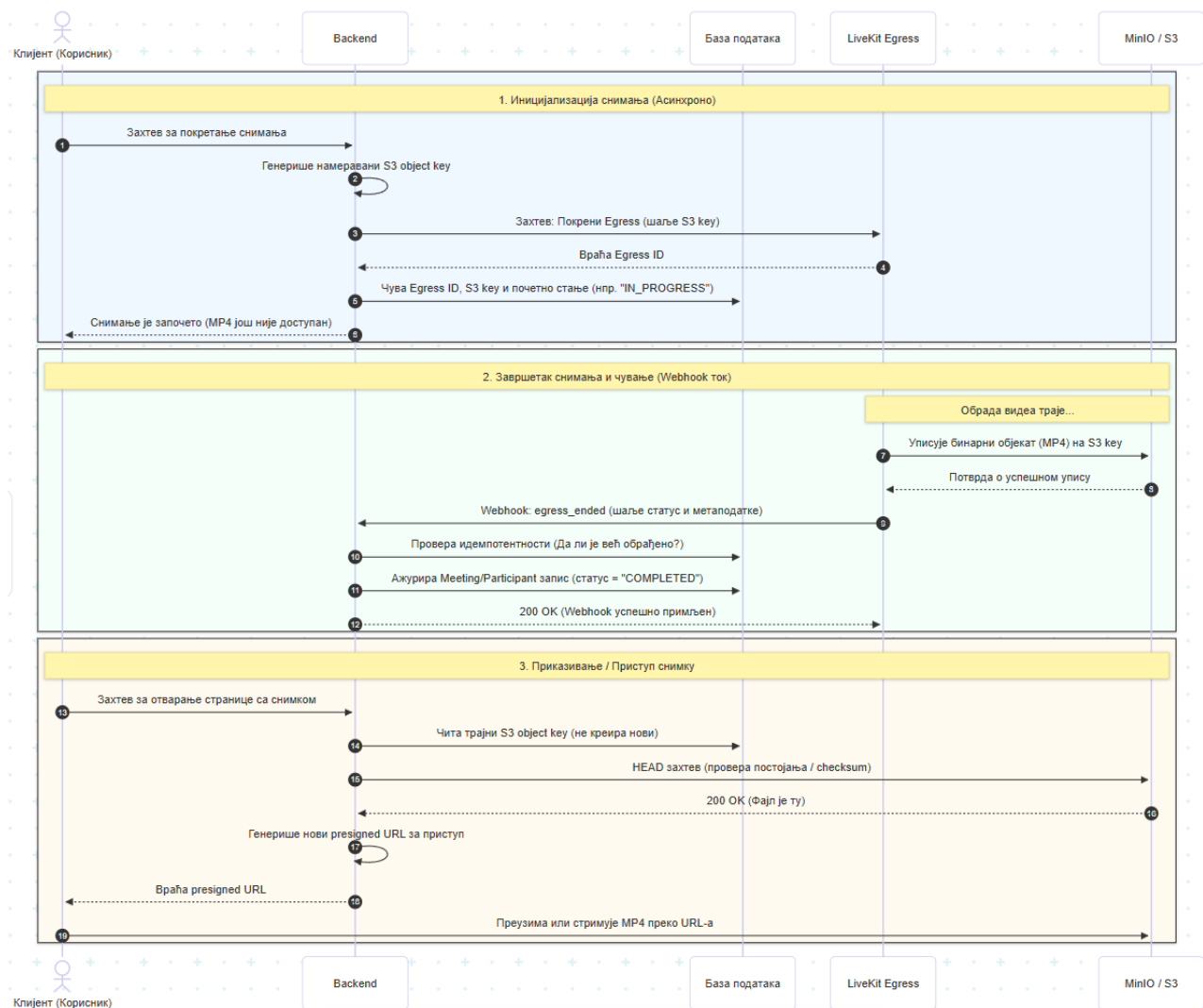
Guest приступ је намерно ограничен на једну собу и њене медијске дозволе. Он не даје увид у листу других Meeting записа, профил, API кључеве или административне endpoint-е. Слично томе, ApiKey не треба аутоматски да добије права Keycloak корисника, већ се његове дозвољене операције морају дефинисати посебном политиком.

Безбедност комуникације између компоненти заснива се на раздвајању различитих облика приступних података и ограничењу њихове намене. Идентитетски токени морају бити проверени у

складу са њиховим потписом, издаваоцем и временом важења, док се механизми као што су РКСЕ и OpenID Connect користе за заштиту процеса аутентификације [4, 5, 6]. Приступ медијским ресурсима и објектном складишту треба ограничити на конкретне ресурсе и временски ограничена права, уз избегавање директног излагања credentials-а клијентима [7, 28]. Овакво раздвајање граница поверења смањује могућност да компромитовање једног приступног механизма доведе до неограниченог приступа осталим деловима система.

4.14 Поузданост снимања и складиштења

Снимање је асинхрони процес и захтев за покретање Egress-а не значи да је MP4 већ доступан. Backend најпре чува идентификатор Egress процеса и почетно стање, затим чека egress_ended webhook и на основу њега повезује исход са Meeting или Participant записом. Овим се разликује започета обрада од успешно завршеног снимка. На слици 4.5 приказан је дијаграм секвенце овог понашања.



Слика 4.5 — Дијаграм секвенце снимања, складиштења и приказивања снимка састанка

Извор: Дијаграм израђен помоћу Mermaid софтвера [62].

S3 object key има улогу трајне везе између пословног записа и бинарног објекта. Он се уписује једном када је позната намеравана путања, а касније се користи за HEAD проверу и генерисање новог presigned URL-а. Не треба креирати нови key сваки пут када се отвори страница, јер би се изгубила веза са претходно сачуваним резултатом.

Морућ је случај у коме је Egress пријавио завршетак, али webhook касни, као и случај у коме је објекат уписан, а пословни запис није ажуриран. Зато продукциона верзија треба да комбинује проверу догађаја са контролисаним retry-јем, HEAD захтевом, checksum-ом и евиденцијом покушаја. Идемпотентност спречава да поновљена порука направи дупли резултат.

MinIO у локалном окружењу омогућава да се овај ток провери без спољног налога, али продукција захтева backup, lifecycle правила, encryption, праћење простора и процедуру опоравка. Поузданост се зато не мери само постојањем MP4 фајла, већ доследношћу између Meeting статуса, webhook-а, S3 key-а и адресе коју корисник може да отвори [28, 29, 30].

5 Имплементација система

У овом поглављу приказана је имплементација пројектованог система, са посебним освртом на серверску и клијентску структуру, креирање и управљање састанцима, интеграцију са LiveKit платформом, снимање и складиштење садржаја, обраду асинхроних догађаја, аутентификацију и ауторизацију, као и управљање временским аспектима система. Целокупан пројекат доступан је на отвореном GitHub налогу аутора. [61]

5.1 Backend структура

Backend је организован у controller, service, repository, entity, dto, mapper, security, config, constants и scheduler пакете. Controller слој дефинише HTTP уговор, service слој оркестрира пословни процес, repository слој комуницира са базом, а mapper претвара ентитете у DTO објекте. Ова подела омогућава да се спољни JSON формат не везује директно за модел персистенције. Структура backend-а је организована тако да контролери остану танки, сервиси носе пословна правила, а репозиторијуми и мапери изолују приступ подацима и трансформацију DTO објеката.

5.2 Сигнализација и медијска веза

Успостављање видео позива састоји се од два повезана, али различита процеса. Сигнализација преноси контролне информације о соби, учесницима и доступним track-овима, док WebRTC преноси звук, слику и податке. У пројекту LiveKit обједињује та два дела: backend креира приступни токен и адресу сервера, а Angular клијент преко LiveKit SDK-а отвара везу и управља медијским учешћем [1, 9].

Када корисник отвори join страницу, browser прво тражи дозволу за камеру и микрофон. Изабрани уређаји се затим прослеђују сервису LiveKitRoomService, који креира локалне track-ове и објављује их у соби. Учеснички track-ови које сервер проследи другим клијентима приказују се у InCallPageComponent-у; тиме је контрола медија одвојена од приказа странице.

Прекид везе не сме одмах да се протумачи као напуштање састанка. LiveKit client разликује стања Reconnecting, Reconnected и Disconnected, а интерфејс их приказује преко overlay-а и статусних порука. Та разлика је важна јер корисник током кратког прекида треба да добије могућност аутоматског повратка, док се после трајног прекида чисте локални ресурси и затвара сесија.

Примена је реализована у web-angular/src/app/core/livekit-room.service.ts и компонентама странице позива, док backend обезбеђује параметре неопходне за повезивање. При провери се посматрају дозволе уређаја, објављивање и гашење track-ова, промена квалитета везе и понашање после реконекције. На тај начин се исправност не своди на то да ли се приказала слика, већ обухвата цео ток од токена до медијског догађаја [9, 10].

5.3 Креирање састанка

Док претходно описана спецификација и дијаграм секвенце приказан у поглављу 4.3 дефинишу пројектовано понашање система на нивоу архитектуре, физичка реализација ове оркестрације приказана је на листингу програмског кода 5.1 приказан је главни део алгорита креирања састанка. MeetingService проверава да ли постоје најмање два учесника и да ли су имена попуњена. Састанак добија UUID собе, статус и временске ознаке. Након уписа у PostgreSQL, позива се LiveKitService за креирање собе и генерисање токена. Токени и join линкови уписују се уз учеснике, а позивнице се шаљу преко MeetingEmailService-а. Креирање састанка је почетна тачка целог доменског тока: један захтев производи Meeting запис, собу, токене, линкове и, када је потребно, обавештења.

```

@Transactional
public Meeting create(CreateMeetingDto dto, String ownerSubject) {
    if (dto.getParticipants() == null || dto.getParticipants().size() < 2) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST,
            "At least 2 participants are required");
    }
    if (dto.getParticipants().stream()
        .skip(ownerSubject == null ? 0 : 1)
        .anyMatch(item -> item.getName() == null
            || item.getName().isBlank())
        || (ownerSubject == null
            && (dto.getParticipants().get(0).getName() == null
            || dto.getParticipants().get(0).getName().isBlank()))) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST,
            "All meeting participants must have names");
    }
    Meeting meeting = meetingMapper.toEntity(dto);
    meeting.setOwnerSubject(ownerSubject);
    if (ownerSubject != null && !meeting.getParticipants().isEmpty()) {
        Participant host = meeting.getParticipants().get(0);
        host.setName(resolveOwnerName(ownerSubject, host.getName()));
        host.setRole(com.connecta.recorder.constants.ParticipantRole.HOST);
        for (int i = 1; i < meeting.getParticipants().size(); i++) {
            meeting.getParticipants().get(i).setRole(
                com.connecta.recorder.constants.ParticipantRole.GUEST);
        }
    }
    meeting.setRoomId(UUID.randomUUID());
    boolean startsImmediately = meeting.getScheduledAt() == null
        || !meeting.getScheduledAt().isAfter(Instant.now());
    meeting.setStatus(startsImmediately
        ? MeetingStatus.IN_PROGRESS
        : MeetingStatus.SCHEDULED);
    if (startsImmediately) {
        meeting.setStartedAt(Instant.now());
    }
    meeting = meetingRepository.save(meeting);
    meetingRepository.flush();
    liveKitService.createRoom(meeting.getRoomId().toString());

    for (Participant participant : meeting.getParticipants()) {
        String token = liveKitService.generateToken(
            meeting.getRoomId().toString(),
            participant.getName(),
            true,
            true,
            participant.getRole().getValue()
        );
        participant.setToken(token);
        participant.setJoinLink(DEFAULT_JOIN_BASE_URL + "/room/" + meeting.getRoomId() +
            "?token=" + token);
    }
    if (startsImmediately && meeting.isRecordingEnabled()) {
        startRecording(meeting);
    }
    meeting = meetingRepository.save(meeting);
    if (startsImmediately) {
        scheduleMeetingEnd(meeting);
    }
    meetingEmailService.sendInvitations(meeting);
    return meeting;
}

```

```
}
```

Листинг 5.1 — Изворни код: креирање састанка, собе и join линкова

Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/service/MeetingService.java`, линије 89–139.

5.4 LiveKit, токени и дозволе

LiveKitService користи API key и secret из configuration properties објекта. AccessToken добија име, identity, metadata улогу и RoomJoin grant. Дозволе за publish и subscribe постављају се у складу са параметрима метода. Тиме се backend не ослања на произвољне податке из URL-а већ креира токен са ограниченим контекстом. Генерисање LiveKit токена намерно је смештено у backend, јер тајна за потписивање не сме стићи у Angular bundle, а дозволе морају бити ограничене на конкретну собу. На листингу програмског кода 5.2 приказан је систем генерисања LiveKit токена, као и комуникација са LiveKit API у сврху управљања процесом снимања. Такође на слици 5.1 је приказана конкретна конфигурација LiveKit SFU и LiveKit Egress сервиса. На слици 5.2 је приказан `livekit.yaml` фајл, који LiveKit користи за своју конфигурацију самог начина рада унутар Docker контејнера.

```
livekit:
  image: livekit/livekit-server:latest
  command: --config /etc/livekit.yaml
  ports:
    - "7880:7880"
    - "7881:7881"
    - "50000-50200:50000-50200/udp"
  environment:
    - TZ=Europe/Belgrade
  volumes:
    - ./livekit.yaml:/etc/livekit.yaml
  depends_on:
    redis:
      condition: service_healthy
  networks:
    - vmp-net
  restart: unless-stopped

egress:
  image: livekit/egress:latest
  cpus: "8"
  mem_limit: 5g
  environment:
    - TZ=Europe/Belgrade
  cap_add:
    - SYS_ADMIN
  shm_size: "3gb"
  depends_on:
    - livekit
    - redis
    - minio
  networks:
    - vmp-net
  restart: unless-stopped
```

Слика 5.1 — снимак кода: LiveKit и Egress конфигурација

Извор: Изворни код: `docker-compose.yml`, линије 77–111.

```

port: 7880
rtc:
  tcp_port: 7881
  port_range_start: 50000
  port_range_end: 50200
  use_external_ip: false

redis:
  address: redis:6379

keys:
  devkey: devsecret12345678901234567890123

room:
  empty_timeout: 300
  max_participants: 6

logging:
  level: info
  sample: true

webhook:
  urls:
    - http://api:5000/webhook/livekit
  api_key: devkey

```

Слика 5.2 — снимак кода: Конфигурација начина рада LiveKit сервиса

Извор: Изворни код: *livekit.yaml*, линије 1–25.

```

public String generateToken(String roomName, String participantName, boolean
canPublish, boolean canSubscribe, String role) {
    validateRoomName(roomName);
    if (participantName == null || participantName.isBlank()) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Participant
name is required");
    }

    AccessToken token = new AccessToken(properties.getLiveKit().getApiKey(),
properties.getLiveKit().getApiSecret());
    token.setName(participantName);
    token.setIdentity(participantName);
    token.setMetadata(role);
    token.addGrants(new RoomJoin(true), new RoomName(roomName));
    if (!canPublish) {
        token.addGrants(new CanPublish(false));
    }
    if (!canSubscribe) {
        token.addGrants(new CanSubscribe(false));
    }
    return token.toJwt();
}

public String startRecording(String roomName, String s3Bucket, String s3Key,
int width, int height) {
    validateRecordingRequest(roomName, s3Bucket, s3Key, width, height);
    try {
        EgressServiceClient client = EgressServiceClient.createClient(

```

```

        properties.getLiveKit().getUrl(),
        properties.getLiveKit().getApiKey(),
        properties.getLiveKit().getApiSecret()
    );

    LivekitEgress.EncodedFileOutput fileOutput =
LivekitEgress.EncodedFileOutput.newBuilder()
        .setFileType(LivekitEgress.EncodedFileType.MP4)
        .setFilepath(s3Key)
        .setS3(buildS3Upload(s3Bucket))
        .build();

    return client.startRoomCompositeEgress(
        roomName,
        fileOutput,
        "grid",
        null,
        buildEncodingOptions(width, height),
        false,
        false
    ).execute().body().getEgressId();
} catch (Exception ex) {
    throw new IllegalStateException("Failed to start room recording for
room: " + roomName, ex);
}
}

```

Листинг 5.2 — Изворни код: LiveKit токен и композитно снимање

Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/service/LiveKitService.java`, линије 57–103.

5.5 Композитно и participant снимање

Композитно снимање користи room composite захтев, grid распоред, MP4 излаз и S3Upload конфигурацију. За учесника се креира посебан template URL који садржи room и identity параметре. Participant template Angular компонента се повезује на собу, чека track-ове и омогућава Egress-у да забележи одговарајући приказ. Подела на композитно и participant снимање покрива два различита начина употребе снимка: преглед целог разговора и накнадну анализу појединачног учесника. На листингу 5.3 приказана је метода која отпрема видео снимке на било које S3-компатибилно складиште користећи S3Upload конфигурацију.

```

private LivekitEgress.S3Upload buildS3Upload(String bucket) {
    return LivekitEgress.S3Upload.newBuilder()
        .setAccessKey(properties.getS3().getAccessKey())
        .setSecret(properties.getS3().getSecretKey())
        .setBucket(bucket)
        .setEndpoint(properties.getS3().getEndpoint())
        .setRegion(properties.getS3().getRegion())
        .setForcePathStyle(true)
        .build();
}

private LivekitEgress.EncodingOptions buildEncodingOptions(int width, int
height) {
    int sanitizedWidth = (width / 2) * 2;
    int sanitizedHeight = (height / 2) * 2;
    return LivekitEgress.EncodingOptions.newBuilder()
        .setWidth(sanitizedWidth)
        .setHeight(sanitizedHeight)
        .setFramerate(properties.getRecordingDefaults().getFramerate())
        .setVideoBitrate(properties.getRecordingDefaults().getVideoBitrate())
        .setAudioBitrate(properties.getRecordingDefaults().getAudioBitrate())

```

```
.setAudioFrequency(properties.getRecordingDefaults().getAudioFrequency())  
.build();
```

Листинг 5.3 — Изворни код: S3 upload и encoding параметри

Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/service/LiveKitService.java`, линије 195–217.

5.6 Webhook обрада

WebhookController приказан у листингу 5.4 прима raw тело и Authorization заглавље, а LiveKitService користи WebhookReceiver за валидацију потписа. Обрађују се egress_ended, participant_joined и participant_left догађаји. Код egress_ended догађаја чувају се величина, трајање и S3 key. Код participant_joined догађаја бележи се присуство и може се покренути participant recording. Webhook обрада повезује спољни Egress догађај са локалним Meeting и Participant записима, па је неопходно прихватити да догађај може стићи са закашњењем или више пута а поузданост испоруке webhook догађаја обезбеђује се механизмом за поновну испоруку који LiveKit примењује у случају неуспешне обраде захтева. Уколико сервер не врати успешан HTTP одговор, LiveKit може поновити слање истог догађаја, чиме се омогућава његова поновна обрада након пролазне грешке.

```
@PostMapping("/livekit")  
public ResponseEntity<Void> livekitWebhook(@RequestBody String body,  
HttpServletRequest request) {  
    String authorization = request.getHeader("Authorization");  
    LivekitWebhook.WebhookEvent event;  
    try {  
        event = liveKitService.parseWebhook(body, authorization == null ? "" :  
authorization);  
    } catch (Exception ex) {  
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();  
    }  
  
    try {  
        if ("egress_ended".equals(event.getEvent()) && event.hasEgressInfo()) {  
            handleEgressEnded(event.getEgressInfo());  
        }  
        if ("participant_joined".equals(event.getEvent()) &&  
event.hasParticipant()) {  
            handleParticipantJoined(event.getParticipant(), event.hasRoom() ?  
event.getRoom().getName() : "");  
        }  
        if ("participant_left".equals(event.getEvent()) &&  
event.hasParticipant()) {  
            handleParticipantLeft(event.getParticipant(), event.hasRoom() ?  
event.getRoom().getName() : "");  
        }  
    } catch (Exception e) {  
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();  
    }  
    return ResponseEntity.ok().build();  
}  
  
void handleEgressEnded(LivekitEgress.EgressInfo egressInfo) {  
    if (meetingService.updateParticipantRecordingInfo(egressInfo)) {  
        return;  
    }  
  
    Meeting meeting =  
meetingService.getByEgressId(egressInfo.getEgressId()).orElse(null);  
    if (meeting == null) {
```

```

        return;
    }

    meetingService.updateRecordingInfo(meeting.getId(), egressInfo);
    Meeting refreshed =
meetingService.getByEgressId(egressInfo.getEgressId()).orElse(null);
    }

    void handleParticipantJoined(LivekitModels.ParticipantInfo participant, String
roomName) {
        if (participant.getIdentity().startsWith("EG_")) {
            return;
        }
        meetingService.setInProgressByRoom(roomName);
        meetingService.updateParticipantTimestamp(participant.getIdentity(),
roomName, true);

        Meeting meeting =
meetingService.getByRoomId(java.util.UUID.fromString(roomName));
        if (!meeting.isRecordingEnabled()) {
            return;
        }

        Participant dbParticipant = meeting.getParticipants().stream()
            .filter(item -> item.getName().equals(participant.getIdentity()))
            .findFirst()
            .orElse(null);
        if (dbParticipant == null) {
            return;
        }

        String s3Key = "recordings/" + meeting.getRoomId() + "/participants/" +
participant.getIdentity() + "-" +
            java.time.format.DateTimeFormatter.ofPattern("yyyyMMdd-
HHmmss").withZone(java.time.ZoneOffset.UTC).format(Instant.now()) + ".mp4";
        String egressId = liveKitService.startParticipantRecording(
            roomName,
            participant.getIdentity(),
            meeting.getRecordingS3Bucket(),
            s3Key,
            meeting.getRecordingWidth(),
            meeting.getRecordingHeight()
        );
        meetingService.saveParticipantEgressId(participant.getIdentity(), roomName,
egressId, s3Key);
    }

    void handleParticipantLeft(LivekitModels.ParticipantInfo participant, String
roomName) {
        if (participant.getIdentity().startsWith("EG_")) {
            return;
        }
        meetingService.updateParticipantTimestamp(participant.getIdentity(),
roomName, false);
    }

```

Листинг 5.4 — изворни код: валидација webhook-а и Egress догађаји

*Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/controller/WebhookController.java`,
линије 30–108.*

5.7 PostgreSQL и JSONB

JPA ентитети се уписују у PostgreSQL, док се metadata и notes чувају у JSONB колонама. ColumnTransformer обезбеђује да се вредност упише као jsonb. Индекси на roomId, status и датум креирања подржавају најчешће претраге. Логичко брисање се реализује пољем deleted, што спречава губитак пословне евиденције. Комбинација релационих колона и JSONB поља омогућава да се стабилни атрибути индексирају, док се променљиви параметри снимања сачувају без непотребног ширења шеме.

5.8 Завршетак и scheduler-и

MeetingService заказује аутоматски завршетак преко TaskScheduler-а. SessionCleanupScheduler периодично проверава активне састанке и завршава оне којима је истекло трајање. MeetingReminderScheduler позива функцију за слање подсетника. Ови процеси чине систем отпорним на ситуацију у којој корисник затвори прегледач без ручног завршетка. Scheduler-и затварају стања која корисник није ручно затворио, али њихова обрада мора бити идемпотентна како истек рока не би поништио каснији легитимни webhook.

5.9 Email обрада

MeetingEmailService креира HTML позивнице, подсетнике, обавештења о изменама, отказивању и завршне извештаје. Mailpit омогућава да се поруке у развоју прегледају без слања стварним примаоцима. Садржај се escape-ује пре уметања у HTML, а presigned URL се генерише за примаоца када је снимак доступан. Email обрада је намерно споредни излаз пословног тока: неуспех SMTP испоруке бележи се и може се поновити, али не сме избрисати већ креирани састанак.

5.10 API и Keycloak безбедност

ApiAuthenticationFilter прихвата Bearer вредност и прво покушава JWT parsing, а затим проверу API кључа. AuthController користи JwtAuthenticationToken за профил и reset лозинке. KeycloakAccountService комуницира са Admin API-јем за читање и измену профила. Администраторски token се не шаље frontend-у. API и Keycloak граница разликују гостујући приступ медијској соби од управљања ресурсима, чиме се непријављеном учеснику не дају права власника. На листинзима програмског кода под бројевима 5.5, 5.6 и 5.7 приказана је имплементација описаног система.

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain)
    throws ServletException, IOException {
    String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        filterChain.doFilter(request, response);
        return;
    }
    String token = authHeader.substring(7).trim();
    boolean authenticated = false;
    String principal = "api-client";
    try {
        principal = jwtService.parse(token).getSubject();
        authenticated = true;
    } catch (JwtException ignored) {
        authenticated = apiKeyService.validateKey(token);
        if (authenticated) {
            principal = "api-key";
        }
    }
}
```

```

        if (!authenticated) {
            filterChain.doFilter(request, response);
            return;
        }
        UsernamePasswordAuthenticationToken authentication = new
UsernamePasswordAuthenticationToken(
            principal,
            null,
            List.of(new SimpleGrantedAuthority("ROLE_API"))
        );
        SecurityContextHolder.getContext().setAuthentication(authentication);
        filterChain.doFilter(request, response);
    }

```

Листинг 5.5 — изворни код: JWT/API key филтер

Извор: Изворни код:

api_java/src/main/java/com/connecta/recorder/security/ApiAuthenticationFilter.java, линије 38–72.

```

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http.csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())

        .sessionManagement(session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/health", "/swagger/**", "/v3/api-
docs/**", "/webhook/**", "/meetings/**").permitAll()
            .anyRequest().permitAll())
        .addFilterBefore(apiAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class)
        .oauth2ResourceServer(oauth -> oauth
            .bearerTokenResolver(this::resolveBearerToken)
            .jwt(Customizer.withDefaults()));

    return http.build();
}

```

Листинг 5.6 — изворни код: Spring Security ланац

Извор: Изворни код:

api_java/src/main/java/com/connecta/recorder/security/SecurityConfiguration.java, линије 28–41.

```

public JsonNode update(String subject, String email, UpdateProfileDto dto) {
    try {
        String adminToken = getAdminToken();
        String userId = findUserId(adminToken, subject, email);
        client.put().uri("/users/" + userId).header(HttpHeaders.AUTHORIZATION,
"Bearer " + adminToken)
            .header(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
            .contentType(MediaType.APPLICATION_JSON)
            .body(objectMapper.createObjectNode().put("firstName",
dto.getFirstName()).put("lastName", dto.getLastName()).put("email",
dto.getEmail()))
            .retrieve().toBodilessEntity();
        return objectMapper.createObjectNode().put("firstName",
dto.getFirstName()).put("lastName", dto.getLastName()).put("email",
dto.getEmail());
    } catch (ResponseStatusException exception) {
        throw exception;
    } catch (RestClientResponseException exception) {
        log.warn("Keycloak profile update rejected: status={}, body={}",
exception.getStatusCode(), exception.getResponseBodyAsString());
        throw new
ResponseStatusException(HttpStatus.valueOf(exception.getStatusCode().value()),
"Keycloak rejected profile update", exception);
    }
}

```

```

        } catch (Exception exception) { throw new
ResponseStatusException(HttpStatus.BAD_GATEWAY, "Could not update profile in
Keycloak", exception); }
    }

    public JsonNode getProfile(String subject, String email) {
        try {
            String adminToken = getAdminToken();
            String userId = findUserId(adminToken, subject, email);
            JsonNode user = client.get().uri("/users/" + userId)
                .header(HttpHeaders.AUTHORIZATION, "Bearer " + adminToken)
                .retrieve().body(JsonNode.class);
            return objectMapper.createObjectNode()
                .put("firstName", user.path("firstName").asText(""))
                .put("lastName", user.path("lastName").asText(""))
                .put("email", user.path("email").asText(""));
        } catch (ResponseStatusException exception) {
            throw exception;
        } catch (RestClientResponseException exception) {
            log.warn("Keycloak profile load rejected: status={}, body={}",
exception.getStatusCode(), exception.getResponseBodyAsString());
            throw new
ResponseStatusException(HttpStatus.valueOf(exception.getStatusCode().value()),
"Keycloak rejected profile load", exception);
        } catch (Exception exception) {
            throw new ResponseStatusException(HttpStatus.BAD_GATEWAY, "Could not
load profile from Keycloak", exception);
        }
    }

    public void sendPasswordResetEmail(String subject, String email) {
        try {
            String adminToken = getAdminToken();
            String userId = findUserId(adminToken, subject, email);
            client.put().uri("/users/" + userId + "/execute-actions-email")
                .header(HttpHeaders.AUTHORIZATION, "Bearer " + adminToken)
                .contentType(MediaType.APPLICATION_JSON)
                .body(List.of("UPDATE_PASSWORD"))
                .retrieve().toBodilessEntity();
        } catch (ResponseStatusException exception) {
            throw exception;
        } catch (RestClientResponseException exception) {
            log.warn("Keycloak password reset email rejected: status={}, body={}",
exception.getStatusCode(), exception.getResponseBodyAsString());
            throw new
ResponseStatusException(HttpStatus.valueOf(exception.getStatusCode().value()),
"Keycloak rejected password reset email", exception);
        } catch (Exception exception) {
            throw new ResponseStatusException(HttpStatus.BAD_GATEWAY, "Could not
send password reset email", exception);
        }
    }

    private String findUserId(String adminToken, String subject, String email) {
        if (email != null && !email.isBlank()) {
            JsonNode users = client.get().uri(uriBuilder ->
uriBuilder.path("/users").queryParam("email", email).queryParam("exact",
true).build())
                .header(HttpHeaders.AUTHORIZATION, "Bearer " +
adminToken).retrieve().body(JsonNode.class);
            if (users != null && users.isArray() && !users.isEmpty()) return
users.get(0).get("id").asText();
        }
        return subject;
    }
}

```

```

    private String getAdminToken() {
        if (properties.getKeycloak().getAdminUsername() == null ||
properties.getKeycloak().getAdminPassword() == null
            || properties.getKeycloak().getAdminUsername().isBlank() ||
properties.getKeycloak().getAdminPassword().isBlank()) {
            throw new ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE,
"Keycloak admin credentials are not configured");
        }
        org.springframework.util.LinkedMultiValueMap<String, String> form = new
org.springframework.util.LinkedMultiValueMap<>();
        form.add("grant_type", "password");
        form.add("client_id", "admin-cli");
        form.add("username", properties.getKeycloak().getAdminUsername());
        form.add("password", properties.getKeycloak().getAdminPassword());
        JsonNode response =
keycloakClient.post().uri("/realms/master/protocol/openid-connect/token")

.contentType(MediaType.APPLICATION_FORM_URLENCODED).body(form).retrieve().body(Json
Node.class);
        return response.get("access_token").asText();
    }

```

Листинг 5.7 — изворни код: Keycloak Admin API

Извор: Изворни код:

api_java/src/main/java/com/connecta/recorder/service/KeycloakAccountService.java, линије 33–112.

5.11 Google Identity Provider

Код Google регистрације Angular не комуницира директно са Google SDK-ом. Keycloak приказује social provider дугме приказано листингом програмског кода 5.8 и управља broker током. Realm мора имати Google provider са одговарајућим Client ID и Secret вредностима. Пошто credentials зависе од конкретне Google Cloud конфигурације, они се не уписују у репозиторијум. Ова могућност је у раду описана као безбедно конфигурационо проширење постојећег Keycloak тока. Google Identity Provider је интеграциони слој иза Keycloak-а, па Angular никада не рукује Google тајном, а систем задржава јединствен облик локалног идентитета.

```

<div id="kc-social-providers"
class="${properties.kcFormSocialAccountSectionClass!}">
    <hr/>
    <h2>${msg("identity-provider-login-label")}</h2>
    <ul class="${properties.kcFormSocialAccountListClass!}">
        <#list social.providers as p>
            <li><a data-once-link data-disabled-
class="${properties.kcFormSocialAccountListButtonDisabledClass!}" id="social-
${p.alias}"
                class="${properties.kcFormSocialAccountListButtonClass!}"
aria-label="Sign in with ${p.displayName!}" type="button" href="${p.loginUrl}">
                <#if p.iconClasses?has_content><i
class="${properties.kcCommonLogoIdP!} ${p.iconClasses!}"
                aria-hidden="true"></i></#if><span
class="${properties.kcFormSocialAccountNameClass!}">${p.displayName!}</span></a></li>
        </#list>
    </ul>
</div>

```

Листинг 5.8 — изворни код: social provider дугме

Извор: Изворни код: keycloak/themes/sessions/login/register.ftl, линије 39–50.

5.12 Angular OAuth и API interceptor

Клијентски AuthService приказан на листингу програмског кода 5.9 креира стање и PKCE code verifier, смешта их у sessionStorage и отвара Keycloak authorization endpoint. После повратка проверава се state, code се мења за токене, а URL се чисти. API interceptor додаје Bearer token само захтевима који припадају заштићеним функцијама. Angular OAuth и interceptor аутоматизују додавање access token-а само за заштићене захтеве, док јавни join ток остаје доступан без непотребног покретања пријаве.

Access token се у клијентској апликацији чува у localStorage-у како би аутентификована сесија остала активна и након поновног учитавања странице, без поновног покретања OAuth поступка. Ово поједностављује SPA архитектуру, јер је токен непосредно доступан AuthService-у и API interceptor-у, који га аутоматски додаје као Bearer token само захтевима ка заштићеним ресурсима. Access token се такође користи за издвајање основних података о тренутном кориснику у методи readProfile. При томе се за добијање токена користи Authorization Code ток са PKCE механизмом, док се вредност state проверава при повратку са Keycloak-а како би се потврдила исправност започете OAuth трансакције.

Избор localStorage-а представља свесни компромис између једноставности имплементације и безбедности. Његова предност је што омогућава једноставно очување клијентске сесије и директан приступ access token-у из Angular апликације, али је истовремено садржај localStorage-а доступан JavaScript коду који се извршава у оквиру истог origin-а. Због тога успешан XSS напад може омогућити нападачу да прочита access token и употреби га за приступ заштићеним ресурсима све док је токен важећи. Овај ризик је посебно значајан у односу на претходно дефинисани модел претњи, па се безбедност ослања и на мере за спречавање XSS напада садржаних унутар Angular радног оквира и одговарајуће ограничење животног века токена.

ID token се у приказаном делу имплементације такође чува у localStorage-у, али се не користи за ауторизацију API захтева нити за читање профила, пошто се те операције заснивају на access token-у. Његово чување је задржано ради расположивости података из процеса аутентификације уколико су потребни другим деловима клијентске апликације.

```
private async initialize(): Promise<void> {  
    const query = new URLSearchParams(window.location.search)  
    const code = query.get('code')  
    if (code) {  
        try {  
            const transaction = this.readOAuthTransaction()  
            if (!transaction || query.get('state') !== transaction.state) throw  
new Error('Invalid OAuth state')  
            const response = await  
fetch(`${this.config.keycloakUrl}/realms/${encodeURIComponent(this.config.keycloakR  
ealm)}/protocol/openid-connect/token`, {  
                method: 'POST',  
                headers: {'Content-Type': 'application/x-www-form-urlencoded'},  
                body: new URLSearchParams({grant_type: 'authorization_code',  
client_id: this.config.keycloakClientId, code, redirect_uri:  
transaction.redirectUri, code_verifier: transaction.codeVerifier})  
            })  
            if (!response.ok) throw new Error('Keycloak token exchange failed')  
            const tokens = await response.json() as {access_token: string;  
id_token?: string}  
            localStorage.setItem('sessions_access_token', tokens.access_token)  
            if (tokens.id_token) localStorage.setItem('sessions_id_token',  
tokens.id_token)  
        } catch {  
            this.clearLocalSession()  
        }  
    }  
}
```

```

        } finally {
            sessionStorage.removeItem('sessions_oauth_transaction')
            window.history.replaceState({}, document.title,
window.location.pathname)
        }
    }
    const token = this.token()
    this.loggedIn.set(!!token)
    this.readProfile(token)
    this.ready.set(true)
}

private async redirectToKeycloak(action?: string): Promise<void> {
    const endpoint =
`${this.config.keycloakUrl}/realms/${encodeURIComponent(this.config.keycloakRealm)}/protocol/openid-connect/auth`
    const redirectUri = window.location.origin
    const state = this.randomUrlValue(32)
    const codeVerifier = this.randomUrlValue(64)
    const codeChallenge = await this.createCodeChallenge(codeVerifier)
    const transaction: OAuthTransaction = {state, codeVerifier, redirectUri}
    sessionStorage.setItem('sessions_oauth_transaction',
JSON.stringify(transaction))
    const params = new URLSearchParams({client_id:
this.config.keycloakClientId, redirect_uri: redirectUri, response_type: 'code',
scope: 'openid profile email', state, code_challenge: codeChallenge,
code_challenge_method: 'S256', prompt: 'login', max_age: '0', theme:
this.theme.isDark() ? 'dark' : 'light'})
    if (action) {
        params.set('kc_action', action)
        params.set('action', action)
    }
    window.location.assign(`${endpoint}?${params}`)
}

private readOAuthTransaction(): OAuthTransaction | undefined {
    const value = sessionStorage.getItem('sessions_oauth_transaction')
    if (!value) return undefined
    try {
        const transaction = JSON.parse(value) as OAuthTransaction
        if (!transaction.state || !transaction.codeVerifier ||
!transaction.redirectUri) return undefined
        return transaction
    } catch {
        return undefined
    }
}

private randomUrlValue(size: number): string {
    const bytes = new Uint8Array(size)
    crypto.getRandomValues(bytes)
    return this.toBase64Url(bytes)
}

private async createCodeChallenge(value: string): Promise<string> {
    const digest = await crypto.subtle.digest('SHA-256', new
TextEncoder().encode(value))
    return this.toBase64Url(new Uint8Array(digest))
}

private toBase64Url(value: Uint8Array): string {
    let binary = ''
    value.forEach(byte => binary += String.fromCharCode(byte))
    return btoa(binary).replace(/\+/g, '-').replace(/\//g, '_').replace(/>

```

```

    }

    private clearLocalSession(): void {
        localStorage.removeItem('sessions_access_token')
        localStorage.removeItem('sessions_id_token')
        this.loggedIn.set(false)
    }

    private readProfile(value = this.token()): void {
        if (!value) return
        const token = JSON.parse(atob(value.split('.')[1])) as Record<string,
string>
        if (!token) return
        this.profile.set({firstName: token['given_name'] || '', lastName:
token['family_name'] || '', email: token['email'] || ''})
    }

```

Листинг 5.9 — изворни код: Authorization Code + PKCE ток

Извор: Изворни код: web-angular/src/app/core/auth.service.ts, линије 54–139.

5.13 LiveKit frontend сервис

LiveKitRoomService креира Room, повезује га са LiveKit URL-ом и региструје догађаје за учеснике, track-ове, квалитет везе и снимање. Reconnecting и Reconnected догађаји ажурирају сигнал reconnecting, као што је приказано на листингу програмског кода 5.10. InCallPageComponent користи то стање за overlay и управља локалним контролама. LiveKit frontend сервис централизује повезивање, објављивање track-ова и реакцију на промене собе, тако да компоненте страница не морају саме да управљају медијским ресурсима.

```

    async connect(token: string, name: string, cameraId?: string, microphoneId?:
string,
        cameraEnabled = true, microphoneEnabled = true): Promise<void> {
        this.disconnect()
        const room = new Room({
            videoCaptureDefaults: cameraId ? {deviceId: cameraId} : undefined,
            audioCaptureDefaults: microphoneId ? {deviceId: microphoneId} :
undefined
        })
        this.activeRoom = room
        this.room.set(room)
        const sync = () => {
            this.localParticipant.set(room.localParticipant)
            this.remoteParticipants.set([...room.remoteParticipants.values()])
        }
        room.on(RoomEvent.ParticipantConnected, sync)
        room.on(RoomEvent.ParticipantDisconnected, sync)
        room.on(RoomEvent.ParticipantNameChanged, sync)
        room.on(RoomEvent.TrackSubscribed, sync)
        room.on(RoomEvent.TrackUnsubscribed, sync)
        room.on(RoomEvent.TrackPublished, sync)
        room.on(RoomEvent.TrackUnpublished, sync)
        room.on(RoomEvent.LocalTrackPublished, sync)
        room.on(RoomEvent.LocalTrackUnpublished, sync)
        room.on(RoomEvent.ActiveSpeakersChanged, sync)
        room.on(RoomEvent.ConnectionQualityChanged, sync)
        room.on(RoomEvent.Reconnecting, () => this.reconnecting.set(true))
        room.on(RoomEvent.Reconnected, () => {
            this.reconnecting.set(false);
            sync()
        })
        room.on(RoomEvent.RecordingStatusChanged, value =>
this.recording.set(value))
        room.on(RoomEvent.Disconnected, () => {

```

```

        this.connected.set(false)
        this.reconnecting.set(false)
        this.remoteParticipants.set([])
        this.localParticipant.set(null)
    })
    try {
        await room.connect(this.config.liveKitUrl, token)
        if (this.activeRoom !== room) {
            room.disconnect();
            return
        }
        if (name.trim() && name.trim() !== room.localParticipant.name) await
room.localParticipant.setName(name.trim())
        await Promise.all([
            room.localParticipant.setCameraEnabled(cameraEnabled),
            room.localParticipant.setMicrophoneEnabled(microphoneEnabled)
        ])
        this.connected.set(room.state === ConnectionState.Connected)
        this.recording.set(room.isRecording)
        sync()
    } catch (error) {
        this.error.set(error instanceof Error ? error.message : 'Unable to
connect to LiveKit.')
        room.disconnect()
    }
}

disconnect(): void {
    this.activeRoom?.disconnect()
    this.activeRoom = null
    this.room.set(null)
}

```

Листинг 5.10 — изворни код: LiveKit client и reconnect

Извор: Изворни код: web-angular/src/app/core/livekit-room.service.ts, линије 17–77.

5.14 Nginx и runtime конфигурација

Frontend се гради у Node Alpine фази, а production bundle се копира у Nginx Alpine слику. env.template.js се на старту попуњава envsubst командом. Nginx прослеђује /api захтеве API сервису, док остале непознате путање враћају index.html како би Angular Router обрадио навигацију. Nginx runtime конфигурација омогућава да се адреса API-ја промени приликом покретања контејнера, без поновног компајлирања Angular апликације.

5.15 Управљање временом и зонама

Временске вредности утичу на заказивање, подсетнике, истек састанка и приказ резултата, па њихова обрада мора бити јединствена кроз цео систем. Backend прихвата и враћа Instant вредности, док се у бази чувају као апсолутне тачке на временској линији. На тај начин исти тренутак не зависи од локалне зоне рачунара на коме је састанак креиран.

Форма за креирање користи локални приказ датума и времена, али се пре слања вредност претвара у облик који API очекује. На backend-у CreateMeetingDto и UpdateMeetingDto проверавају да је scheduledAt у будућности, а durationLimitMinutes у дозвољеном опсегу. Scheduler затим користи Instant.now() за поређење почетка, трајања и timeout-а [13].

Посебан ризик настаје када се локално време помеша са UTC вредношћу. Корисник би могао да види једно време у форми, док би scheduler састанак активирао или затворио у другом тренутку. Зато

се локализација оставља приказном слоју, док се пословна правила и поређења обављају над апсолутним вредностима.

Релевантни делови имплементације су демонстрирани у листингу програмског кода 5.11. DTO класе за креирање и измену и Angular компоненте `create-meeting-page` и `my-meetings-page`. Провера обухвата различите временске зоне, подсетник 15 минута пре почетка, истек дозвољеног трајања и прелазак преко промене летњег рачунања времена.

```
@Scheduled(fixedDelay = 60000)
public void execute() {
    for (Meeting meeting : meetingService.getActiveMeetings()) {
        if (meeting.getStartedAt() == null) {
            continue;
        }
        Instant scheduledEnd =
meeting.getStartedAt().plusSeconds(meeting.getDurationLimitMinutes() * 60L);
        if (!Instant.now().isBefore(scheduledEnd)) {
            try {
                meetingService.endMeeting(meeting.getRoomId(),
EndMeetingReason.TIMEOUT);
            } catch (Exception ignored) {
            }
        }
    }

    for (Meeting meeting : meetingService.getScheduledMeetings()) {
        Instant meetingStart = meeting.getScheduledAt() != null ?
meeting.getScheduledAt() : meeting.getDateCreated();
        Instant timeout =
meetingStart.plusSeconds((meeting.getDurationLimitMinutes() + 15L) * 60L);
        if (!Instant.now().isBefore(timeout)) {
            try {
                meetingService.cancelIfNotStarted(meeting.getId());
            } catch (Exception ignored) {
            }
        }
    }
}
```

Листинг 5.11 — изворни код: `SessionCleanupScheduler`

Извор: Изворни код:

`api_java/src/main/java/com/connecta/recorder/scheduler/SessionCleanupScheduler.java`, линије 19–44.

5.16 Валидација

Валидација представља прву границу између података које шаље корисник или други сервис и пословних правила апликације. Њена сврха није само да провери да ли је поље попуњено, већ да спречи да неисправна вредност стигне до базе, LiveKit-а, Egress-а или система за слање порука. У пројекту се провера спроводи на више нивоа, при чему сваки ниво штити своју границу.

На HTTP граници Spring контролери користе Bean Validation анотације. У `api_java/src/main/java/com/connecta/recorder/controller/MeetingController.java` методе за креирање, измену, листање и завршавање састанка користе `@Valid`, док DTO класе дефинишу правила за поља као што је приказано у примеру листинга 5.12. `CreateMeetingDto` ограничава дужину назива, број учесника, трајање и резолуцију снимања; `CreateParticipantDto` проверава име, улогу, email и текст

позивнице; UpdateMeetingDto проверава назив, будући термин и позитивно трајање; EndMeetingDto ограничава величину белешки.

```
@Size(max = 200)
private String title;
private Instant scheduledAt;
@Min(1)
@Max(480)
private int durationLimitMinutes = 120;
@NotNull
@Size(min = 2, max = 6)
@Valid
private List<CreateParticipantDto> participants = new ArrayList<>();
@NotNull
@Valid
private RecordingConfigDto recording = new RecordingConfigDto();
private String metadata;
@AssertTrue(message = "ScheduledAt must be in the future.")
@JsonProperty(access = JsonProperty.Access.READ_ONLY)
public boolean isScheduledAtValid() {
    return scheduledAt == null || scheduledAt.isAfter(Instant.now());
}
private boolean isValidAbsoluteUrl(String value) {
    try {
        URI uri = URI.create(value);
        return uri.getScheme() != null && uri.getHost() != null;
    } catch (Exception ex) {
        return false;
    }
}
}
```

Листинг 5.12 — изворни код: Валидација помоћу анотација унутар CreateMeetingDto класе

Извор: Изворни код: api_java/src/main/java/com/connecta/recorder/dto/CreateMeetingDto.java, линије 18–52.

Декларативна правила нису довољна за услове који зависе од тренутног стања система. MeetingService чији је део приказан листингом 5.13 зато проверава да ли састанак постоји, да ли је корисник власник, да ли је дозвољен тражени прелаз статуса и да ли се join дешава у дозвољеном временском прозору. Методе getByRoomId, validateJoinAccess и endMeeting у api_java/src/main/java/com/connecta/recorder/service/MeetingService.java представљају пример пословне валидације која се не може изразити само анотацијом над једним пољем.

```
if (dto.getParticipants() == null || dto.getParticipants().size() < 2) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "At least 2
participants are required");
}
if (dto.getParticipants().stream().skip(ownerSubject == null ? 0 :
1).anyMatch(item -> item.getName() == null || item.getName().isBlank())
|| (ownerSubject == null && (dto.getParticipants().get(0).getName()
== null || dto.getParticipants().get(0).getName().isBlank()))) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "All meeting
participants must have names");
}
```

Листинг 5.13 — изворни код: Осигуравање пословне логике унутар create методе класе MeetingService

Извор: Изворни код: api_java/src/main/java/com/connecta/recorder/service/MeetingService.java, линије 90–96.

Провера се наставља и на инфраструктурним границама. LiveKitService из листинга 5.14 проверава назив собе, bucket, object key и позитивну ширину и висину снимка, док ApiAuthenticationFilter и ApiKeyService из листинга 5.15 проверавају Bearer JWT или API кључ. SystemConfigurationProperties је означен са @Validated и захтева непразне вредности за LiveKit и S3 конфигурацију. На frontend-у create-meeting-page.component.ts користи Angular Validators чија је примена показана листингом 5.16 за назив, термин, трајање, димензије и обавезне учеснике, али се те провере увек понављају на backend-у јер клијент није поуздана безбедносна граница [13, 14].

```
private void validateRoomName(String roomName) {
    if (roomName == null || roomName.isBlank()) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Room name is required");
    }
}

private void validateRecordingRequest(String roomName, String s3Bucket, String s3Key, int width, int height) {
    validateRoomName(roomName);
    if (s3Bucket == null || s3Bucket.isBlank()) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "S3 bucket is required");
    }
    if (s3Key == null || s3Key.isBlank()) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "S3 key is required");
    }
    if (width <= 0 || height <= 0) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Recording resolution must be positive");
    }
}
```

Листинг 5.14 — изворни код: Валидационе методе унутар LiveKitService класе

Извор: Изворни код: api_java/src/main/java/com/connecta/recorder/service/LiveKitService.java, линије 176–193.

```
@Transactional
public boolean validateKey(String rawKey) {
    for (ApiKey key : apiKeyRepository.findAllByIsActiveTrueAndDeletedFalse())
    {
        if (BCrypt.checkpw(rawKey, key.getKeyHash())) {
            key.setLastUsedAt(Instant.now());
            apiKeyRepository.save(key);
            return true;
        }
    }
    return false;
}
```

Листинг 5.15 — изворни код: Валидација и ажурирање API кључа унутар ApiKeyService класе

Извор: Изворни код: api_java/src/main/java/com/connecta/recorder/service/ApiKeyService.java, линије 21–31.

```
readonly form = this.fb.group({
    title: ['', [Validators.required, Validators.maxLength(200)]],
    scheduledAt: ['', Validators.required],
    durationLimitMinutes: [60, [Validators.required, Validators.min(1), Validators.max(480)]],
    metadata: [''],
    recordingWidth: [1280, [Validators.min(1)]],
```

```

        recordingHeight: [720, [Validators.min(1)]],
        participants: this.fb.array([this.participant('host'),
this.participant('guest'), this.participant('guest')])
    })

```

Листинг 5.16 — изворни код: Валидација форме са клијентске стране помоћу Angular Validators

Извор: Изворни код: web-angular/src/app/pages/create-meeting-page.component.ts, линије 154-162.

5.17 Идемпотентност webhook обраде

Webhook из LiveKit Egress-a је спољни догађај који може стићи са закашњењем, после привременог прекида или више пута. Сервер зато не сме сваку поруку да третира као нови резултат. Идентификатор Egress процеса служи као кључ за препознавање већ обрађеног догађаја и повезивање са одговарајућим Meeting или Participant записом.

При пријему egress_ended догађаја сервис прво проналази запис и проверава постојећи статус, object key и податке о трајању и величини. Ако су резултати већ уписани, поновљена порука не сме да креира нови запис, пошаље дупли callback или замени исправан резултат празним подацима. Промена стања треба да буде атомична у односу на остале уписе.

Идемпотентност је важна и у случају делимичног успеха. Egress може бити завршен, али upload или callback могу каснити; обрнуто, објекат може постојати пре него што је webhook обрађен. Зато се обрада догађаја комбинује са провером S3 кључа, “retry” политиком и јасним статусима који разликују started, completed и failed исход.

Релевантни код налази се у webhook контролеру, и мапирању статуса Meeting-a. Ручна провера поново шаље исти egress_ended догађај, шаље поруку за непознат Egress ID и симулира прекид између уписа снимка и callback-a. Циљ је да број пословних резултата остане један и после поновљених испорука [10, 11].

5.18 Понашање при прекиду мреже

Прекид мреже у видео позиву мора бити представљен као стање, а не као необрађени изузетак. LiveKit client емитује Reconnecting, Reconnected и Disconnected догађаје, док LiveKitRoomService ажурира сигнале које InCallPageComponent користи за приказ overlay-a. Корисник тако види да ли систем покушава да поврати везу или је сесија коначно прекинута.

Кратак губитак мреже не треба да обрише име учесника, локални избор уређаја или контекст собе. Након повратка, клијент поново успоставља медијске track-ове и освежава приказ учесника. Ако реконекција не успе у предвиђеном року, интерфејс приказује јасну поруку и ослобађа локалне ресурсе како би поновни улазак био могућ.

Серверски део не сме да помеша привремени disconnect са пословним завршетком састанка. О завршетку одлучују експлицитна радња организатора, timeout scheduler или догађај који потврђује да је сесија заиста завршена. Овим се спречава да кратки Wi-Fi прекид покрене прерано заустављање снимања и слање непотпуног извештаја.

Имплементација се налази у web-angular/src/app/core/livekit-room.service.ts чији је део приказан листингом програмског кода 5.17. Провера обухвата искључивање мреже током разговора, прелазак на другу Wi-Fi мрежу, поновни улазак преко истог join линка и случај када сервер више није доступан. Поред визуелне поруке, проверава се да ли су микрофон, камера и број учесника после повратка у исправном стању [9, 19].

```

const room = new Room({
  videoCaptureDefaults: cameraId ? {deviceId: cameraId} : undefined,
  audioCaptureDefaults: microphoneId ? {deviceId: microphoneId} :
undefined
}))
this.activeRoom = room
this.room.set(room)
const sync = () => {
  this.localParticipant.set(room.localParticipant)
  this.remoteParticipants.set([...room.remoteParticipants.values()])
}
room.on(RoomEvent.ParticipantConnected, sync)
room.on(RoomEvent.ParticipantDisconnected, sync)
room.on(RoomEvent.ParticipantNameChanged, sync)
room.on(RoomEvent.TrackSubscribed, sync)
room.on(RoomEvent.TrackUnsubscribed, sync)
room.on(RoomEvent.TrackPublished, sync)
room.on(RoomEvent.TrackUnpublished, sync)
room.on(RoomEvent.LocalTrackPublished, sync)
room.on(RoomEvent.LocalTrackUnpublished, sync)
room.on(RoomEvent.ActiveSpeakersChanged, sync)
room.on(RoomEvent.ConnectionQualityChanged, sync)
room.on(RoomEvent.Reconnecting, () => this.reconnecting.set(true))
room.on(RoomEvent.Reconnected, () => {
  this.reconnecting.set(false);
  sync()
})
})

```

Листинг 5.17 — изворни код: Ажурирање стања учесника унутар LiveKitRoomService

Извор: Изворни код: web-angular/src/app/core/livekit-room.service.ts, линије 20-45.

5.19 Руковање медијским уређајима

Приступ камери и микрофону је предуслов за видео састанак, али није загарантован. Browser може одбити дозволу, уређај може бити заузет, а корисник може имати више камера или микрофона. Због тога се у pre-join кораку прво читају доступни уређаји, приказује preview и тек након избора позива LiveKit веза.

MediaDevicesService, приказан листингом 5.18, комуницира са browser MediaDevices API-јем и представља листу уређаја компоненти [56]. PreJoinPageComponent чува избор и омогућава преглед пре уласка, док LiveKitRoomService користи изабране device ID вредности при повезивању. Ова подела спречава да страница позива сама управља дозволама, избором уређаја и медијским track-овима.

Одбијање дозволе не треба да блокира сваки облик учешћа. Корисник може да уђе са искљученом камером или микрофоном, уз јасну поруку шта недостаје и како се дозвола може накнадно укључити. Слично томе, промена уређаја током разговора мора да заустави стари track и објави нови без рушења целе собе.

Код се налази у web-angular/src/app/core/media-devices.service.ts, pre-join страници са листинга програмског кода 5.19 и сервису за LiveKit собу. Ручна провера укључује празну листу уређаја, одбијену дозволу, искључивање камере током разговора и промену микрофона. На овај начин се корисничко искуство проверава заједно са browser безбедносним ограничењима.

```

async enumerate(): Promise<void> {
  this.loading.set(true)
  this.error.set(null)
  try {
    const permission = await navigator.mediaDevices.getUserMedia({video:
true, audio: true})
    permission.getTracks().forEach(track => track.stop())
  }
}

```

```

        const devices = await navigator.mediaDevices.enumerateDevices()
        this.cameras.set(devices.filter(item => item.kind ===
'videoinput').map(item => ({
            deviceId: item.deviceId,
            label: item.label || 'Camera'
        })))
        this.microphones.set(devices.filter(item => item.kind ===
'audioinput').map(item => ({
            deviceId: item.deviceId,
            label: item.label || 'Microphone'
        })))
    } catch (error) {
        this.error.set(error instanceof Error ? error.message : 'Device access
was denied.')
    } finally {
        this.loading.set(false)
    }
}

```

Листинг 5.18 — изворни код: Добављање доступних медијских уређаја

Извор: Изворни код: web-angular/src/app/core/media-devices.service.ts, линије 15-35.

```

await this.devices.enumerate();
this.cameraId.set(this.devices.cameras()[0]?.deviceId ?? '');
this.microphoneId.set(this.devices.microphones()[0]?.deviceId ?? '');
await this.updatePreview();

```

Листинг 5.19 — изворни код: Руковање добављеним медијским уређајима унутар PreJoinPageComponent

Извор: Изворни код: web-angular/src/app/pages/pre-join-page.component.ts, линије 151-154.

5.20 Управљање белешкама и rich text садржајем

Белешке су додатни пословни садржај састанка који организатор може да унесе пре или после разговора. За њихов унос користи се RichTextEditorComponent, чија имплементација је приказана листингом програмског кода 5.20, док се вредност преко ControlValueAccessor интерфејса уклапа у Angular форме create и my-meetings странице. На backend-у се metadata чува уз Meeting и улази у post-call и callback податке када је то предвиђено.

Компонента не треба да одлучује о пословном формату целог састанка; њена одговорност је да прихвати текст, обавести форму о промени и омогући уређивање. CreateMeetingDto и UpdateMeetingDto примају metadata као ограничену вредност, а MeetingMapper је сериализује и поново чита. Тако се приказ и трајно чување не мешају у једном Angular шаблону.

Rich text отвара ризик од убацивања небезбедног HTML-а. Садржај који се приказује у email-у мора бити escape-ован или очишћен дозвољеним whitelist правилима (санитизован), а кориснички унос не сме да промени структуру поруке. Ограничење дужине и одбијање неочекиваних вредности спречавају и прекомерно увећање базе и mail payload-а.

```

export class RichTextEditorComponent implements ControlValueAccessor, AfterViewInit
{
    @ViewChild('editor') private editor?: ElementRef<HTMLElement>
    readonly Bold = Bold
    readonly Italic = Italic
    readonly Underline = Underline
    readonly List = List
    readonly ListOrdered = ListOrdered
    formatting = {bold: false, italic: false, underline: false}

```

```

private value = ''
private onChange: (value: string) => void = () => undefined
private onTouched: () => void = () => undefined
ngAfterViewInit(): void {
    this.setEditorValue()
}
writeValue(value: string | null): void {
    this.value = value || ''
    this.setEditorValue()
}
registerOnChange(fn: (value: string) => void): void { this.onChange = fn }
registerOnTouched(fn: () => void): void { this.onTouched = fn }
setDisabledState(isDisabled: boolean): void {
this.editor?.nativeElement.toggleAttribute('contenteditable', !isDisabled) }
format(event: MouseEvent, command: string): void {
    event.preventDefault()
    document.execCommand(command, false)
    this.sync()
    this.refreshFormatting()
}
sync(): void {
    const element = this.editor?.nativeElement
    if (!element) return
    this.value = element.textContent?.trim() ? element.innerHTML : ''
    this.onChange(this.value)
    this.onTouched()
    this.refreshFormatting()
}
pasteAsText(event: ClipboardEvent): void {
    event.preventDefault()

document.execCommand('insertText', false, event.clipboardData?.getData('text/plain')
?? '')
    this.sync()
}
refreshFormatting(): void {
    this.formatting = {
        bold: document.queryCommandState('bold'),
        italic: document.queryCommandState('italic'),
        underline: document.queryCommandState('underline')
    }
}
private setEditorValue(): void {
    if (this.editor && this.editor.nativeElement.innerHTML !== this.value)
this.editor.nativeElement.innerHTML = this.value
}
}

```

Листинг 5.20 — изворни код: Имплементација RichTextEditor компоненте

Извор: Изворни код: `web-angular/src/app/shared/rich-text-editor.component.ts`, линије 49-107.

5.21 Email испорука и Mailpit

Email функционалност има улогу обавештавања, а не примарног складишта пословног стања. Backend шаље позивнице, подсетнике и post-call извештаје преко SMTP клијента, док Mailpit у развоју прихвата поруке и приказује их у локалном интерфејсу. На овај начин се цео ток може проверити без слања стварним примаоцима.

Порука се формира на основу Meeting и Participant података. Позивница садржи join линк, а извештај може да садржи трајање, учеснике, белешке и временски ограничен link ка снимку. Пре уметања у HTML шаблон, текстуални и rich text подаци морају бити правилно escape-овани како email не би постао канал за HTML или script injection.

Недоступан SMTP сервер не сме да поништи креирање састанка. Слање се зато посматра као споредни корак који може бити поновљен или означен као неуспешан, док Meeting и Participant записи остају сачувани. Логови треба да садрже класу грешке и идентификатор састанка, али не и лозинку, access token или пун presigned URL.

Релевантне конфигурације су Spring Mail подешавања, сервис за слање порука и Mailpit сервис у docker-compose.yml. Провера обухвата email без адресе, недоступан Mailpit, подсетник и извештај са снимком. У продукцији се Mailpit замењује стварним SMTP провајдером, а credentials се уносе кроз тајни канал [31]. Део имплементације је приказан у листингу програмског кода 5.21.

```
private void sendInvitation(Meeting meeting, Participant participant) throws
MessagingException {
    MimeMessage message = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(message, false, "UTF-8");
    helper.setFrom(properties.getMail().getFrom());
    helper.setTo(participant.getEmail());
    helper.setSubject("Invitation: " + meeting.getTitle());
    helper.setText(buildHtml(meeting, participant), true);
    mailSender.send(message);
}

private String buildHtml(Meeting meeting, Participant participant) {
    String title = escape(meeting.getTitle());
    String invitation = participant.getInvitationText() == null ? "" :
        nl2br(escape(participant.getInvitationText()));
    String summary = richText(meeting.getMetadata());
    String start = meeting.getScheduledAt() == null ? "-" :
DATE_FORMAT.format(meeting.getScheduledAt());
    String end = meeting.getScheduledAt() == null ? "-" :
DATE_FORMAT.format(meeting.getScheduledAt().plusSeconds(meeting.getDurationLimitMin
utes() * 60L));
    String duration = formatDuration(meeting.getDurationLimitMinutes());
    return ""
        <!doctype html><html><body style="margin:0;background:#f4f7fb;font-
family:Arial,sans-serif;color:#172033">
        <html><body>...</body></html>
        """.formatted(title, escape(participant.getName()),
invitationBlock(invitation), start, end, duration,
        summaryBlock(summary), escapeAttribute(participant.getJoinLink()),
escape(participant.getJoinLink()));
}
```

Листинг 5.21 — изворни код: Генерисање HTML структуре и слање мејла за позив на састанак

Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/service/MeetingEmailService.java`, линије 314-357.

HTML код је скраћен између <body> тагова са сврхом боље читљивости

5.22 API уговор и frontend модели

API уговор је заједнички језик између Spring Boot backend-а и Angular клијента. JSON одговори морају имати стабилна имена, типове и правила за nullable вредности, јер се на њих ослањају странице за креирање, позив, post-call преглед и листу састанака. Уговор обухвата и успешне одговоре и структуру грешака.

Backend DTO класе MeetingDto са листинга програмског кода 5.22, ParticipantDto, RecordingDto и CallbackPayloadDto излазе кроз mapper слој, а TypeScript типови у web-angular/src/app/models/meeting.models.ts, чији пример је наведен листингом 5.23, описују оно што клијент заиста очекује. Entity класе се не шаљу директно, чиме се спречава случајно откривање интерних поља, hash вредности или storage детаља.

```
public class MeetingDto extends EntityDto {  
    private UUID roomId;  
    private String title;  
    private String status;  
    private Instant scheduledAt;  
    private int durationLimitMinutes;  
    private boolean recordingEnabled;  
    private Instant startedAt;  
    private Instant endedAt;  
    private String notes;  
    private String metadata;  
    private List<ParticipantDto> participants = new ArrayList<>();  
    private RecordingDto recording;
```

Листинг 5.22 — изворни код: MeetingDto класа са серверске стране

Извор: Изворни код: `api_java/src/main/java/com/connecta/recorder/dto/MeetingDto.java`, линије 8-20.

```
export interface MeetingDto {
  id: number
  roomId: string
  title: string
  status: MeetingStatus
  scheduledAt: string
  durationLimitMinutes: number
  recordingEnabled: boolean
  startedAt: string | null
  endedAt: string | null
  notes: string | null
  metadata: string | null
  recordingWidth?: number
  recordingHeight?: number
  participants: ParticipantDto[]
  recording: RecordingDto | null
}
```

Листинг 5.23 — изворни код: MeetingDto интерфејс са клијетске стране

Извор: Изворни код: web-angular/src/app/models/meeting.models.ts, линије 8-20.

Посебну пажњу захтевају статуси и опционални подаци. Recording може бити null док Egress траје, participant може имати leftAt тек после напуштања, а presigned URL може истећи иако је сам object key и даље важећи. Frontend типови помажу при развоју, али серверска валидација и даље остаје неопходна јер клијентски код није поуздана граница.

Промена API уговора мора бити праћена кроз контролер, DTO, mapper, TypeScript модел и све странице које користе поље. Практична провера укључује промену статуса, одсуство recording објекта, nullable временску вредност и неочекивано поље у одговору. Циљ је да кориснички приказ остане стабилан и када је пословни ток делимично завршен.

5.23 Продукциона употреба и оперативни ризици

Развојна конфигурација има за циљ да се систем покрене једном командом, али продукционо окружење мора имати строже границе. Environment променљиве које садрже лозинке и кључеве треба заменити Docker secrets механизмом, vault системом или менаџером тајни. У истом поступку треба уклонити подразумеване admin и MinIO вредности и ограничити њихову доступност само на неопходне сервисе.

Слике са ознаком latest олакшавају почетак, али отежавају репродукцију и могу неочекивано променити понашање система. Продукциони deployment треба да користи проверене верзије LiveKit-а, Egress-а, MinIO-а, Mailpit-а или његовог production еквивалента, PostgreSQL-а, Redis-а и Keycloak-а. Промене верзија треба тестирати на копији података и проверити webhook и S3 компатибилност.

Мрежа vmp-net омогућава да сервиси користе имена као што су api, livekit и minio, што је практично у локалном окружењу. У продукцији треба одвојити јавни reverse проху од интерне мреже, базу и Redis не излагати јавности, а LiveKit UDP опсег отворити само када је то потребно. TLS треба применити на јавним HTTP и WebSocket адресама, док се интерна комуникација може додатно сегментирати. [3]

Оперативно праћење треба да обухвати број активних соба, трајање Egress процеса, број неуспешних webhook-а, потрошњу CPU и меморије, величину MinIO bucket-а и број послатих email порука. Логови не смеју садржати access token-е, API key вредности, Google Client Secret или целе

presigned URL адресе. За дијагностику је довољно бележити идентификатор састанка, Egress ID и класу грешке.

Backup политика мора обухватити PostgreSQL податке и снимке, али и Keycloak realm конфигурацију. Сама база не може да врати MP4 објекат ако је он изгубљен из MinIO-а, као што ни копија bucket-а није довољна без везе ка Meeting запису. Зато су database backup, object storage replication и периодична провера враћања података део целокупне поузданости система.

Ограничење броја учесника на шест у LiveKit конфигурацији представља пословну и инфраструктурну одлуку, а не само визуелну поставку. Повећање тог броја утиче на bandwidth, CPU, layout и цену снимања. Слично томе, параметри 1280x720, 15 fps и bitrate треба да се посматрају кроз однос квалитета, пропусног опсега, величине објекта и времена обраде.

5.24 Ручна провера отказних сценарија

Ручна провера система треба да прати не само идеалан ток већ и ситуације у којима зависни сервис није доступан. Први сценарио је недоступан PostgreSQL при покретању. Очекује се да healthcheck и depends_on спрече покретање API контејнера пре базе, док се у логовима може видети разлог неуспеха. Други сценарио је недоступан LiveKit: креирање састанка не сме тихо да врати линкове за собу која не постоји.

Посебно се проверава случај у коме је други учесник одбио дозволу за камеру или микрофон. Систем не треба да прекине целу сесију, већ да прикаже разумљиву поруку и омогући улазак са искљученим уређајем. Ово је важно за једноставност јер корисник не треба да буде приморан да разуме разлику између browser дозволе, оперативног система и физичког уређаја.

При провери приступа снимку користе се три сценарија: валидан presigned URL, URL након истека и промењен key у query параметрима. Први треба да врати MP4, други треба да буде одбијен, а трећи не сме да омогући приступ другом објекту. На тај начин се проверава да ли потпис обухвата стварни bucket, key и рок, а не само постојање неке URL адресе.

За Keycloak се проверава повратак након локалне регистрације, повратак након Google broker тока, отказивање пријаве, невалидан state и истекла сесија. Корисник који откаже Google дијалог треба да се врати у апликацију без креираног налога. Корисник који се врати са неочекиваним state параметром не сме добити access token. Ово је суштина PKCE и state заштите у јавном frontend клијенту.

За account linking се проверава постојећи локални корисник са истом email адресом као Google налог. Систем не треба да споји налог без потврде власништва и јасног Keycloak flow-а. Ово правило је важније од привидно једноставнијег корисничког искуства, јер само поклапање текстуалне email адресе није довољан доказ да исто лице контролише оба начина пријаве.

Провера Egress-а обухвата покретање снимања када су присутна најмање два учесника, завршетак након напуштања собе, појаву egress_ended догађаја, упис трајања и величине, као и доступност објекта у MinIO-у. Ако је Egress процес прекинут, састанак не треба представити као да има исправан снимак. Уместо тога, корисник треба да види стање recording_failed или опис ограничења.

Приватност снимака зависи и од корисничког процеса, а не само од техничког потписа. Позивница може да садржи join линк, а извештај може да садржи presigned URL. Зато се у продукцији мора дефинисати ко сме да прими email, колико дуго URL важи и да ли се снимци аутоматски бришу

након истека пословног рока. Ова правила нису у потпуности одређена развојном конфигурацијом и представљају предмет будућег hardening-a.

Архитектура је погодна за постепено проширење. LiveKit може да се издвоји на посебан хост, MinIO може да се замени AWS S3 системом, а Mailpit production SMTP провајдером. Услов је да се не мењају доменски уговори: Meeting и Participant и даље чувају идентификаторе, Egress шаље догађај, а backend генерише приступну адресу. Ово показује вредност употребе стандардних протокола уместо везивања за један производ.

Будуће проширење може увести организације и tenant границу. Тада ownerSubject више не би био довољан за изолацију података, већ би се увео organizationId, правила видљивости и одвојена политика за снимке. Исто важи за API кључеве: тренутна имплементација проверава да ли су активни, док би напреднија верзија могла да уведе scope-ове као што су meetings:create, meetings:read и meetings:end.

На овај начин ручна провера није само формални списак кликова, већ веза између захтева, кода и оперативног ризика. Сваки сценарио има очекивано стање у интерфејсу, промену у бази или складишту и границу која не сме бити нарушена. Оваква провера је пример примене системског приступа: кориснички доживљај, безбедност и инфраструктура посматрају се као једна целина.

5.25 Приватност података

Видео снимак садржи личне податке и може да открије идентитет, глас, лице, разговор и пословни контекст учесника. Зато се питање приватности не завршава избором S3 складишта. Потребно је одредити ко покреће снимање, ко добија линк, колико дуго се снимак чува и како се поступа са захтевом за брисање. У развојној верзији ови параметри су поједностављени, али су у раду издвојени као обавезне продукционе политике.

Callback URL представља још један излаз података из система. Пошто payload може да садржи идентификатор састанка, време, учеснике, S3 key и presigned URL, адреса примаоца мора бити контролисана. Продукциона верзија треба да користи HTTPS, потписивање callback поруке, ограничење броја покушаја и евиденцију резултата. Ако спољни систем није доступан, то не би требало да обрише локално сачуване податке о састанку.

Прелаз са локалног MinIO-а на AWS S3 или други S3-компатибилан систем може се извести без промене корисничког интерфејса ако се задржи исти bucket/key модел и конфигурациони слој. Разлика се појављује у endpoint-у, credentials-има, path-style подршци, политици encryption-a и начину управљања lifecycle правилима. Ово потврђује да је S3 API коришћен као апстракција, а не као директна зависност од једног локалног контејнера.

Keycloak realm треба мигрирати из dev-file режима у продукциону базу, а Google Identity Provider credentials треба уносити кроз тајни канал. Realm export може бити основ за конфигурацију, али не сме садржати Client Secret у репозиторијуму. При миграцији се посебно проверавају redirect URI, web origins, PKCE подешавање, email provider, theme и администраторски налози.

Комбинација једноставног гостујућег приступа и строгог управљања налогом захтева јасан UX. Екран не треба да тражи регистрацију када корисник само отвара join линк, али треба да објасни када је пријава потребна за моје састанке, профил или измену заказане сесије. Овако се безбедносна правила не скривају од корисника, већ се представљају у складу са њиховом стварном потребом.

Коначна процена система зато обухвата три нивоа: употребљивост, функционалну исправност и безбедност. Употребљивост се види у броју корака до састанка, функционална исправност у доследности података између API-ја, LiveKit-а и MinIO-а, а безбедност у исправном одбијању неважећих токена, URL-ова и callback захтева. Тек заједничко посматрање ова три нивоа даје реалну слику квалитета платформе.

6 Приказ имплементираног система

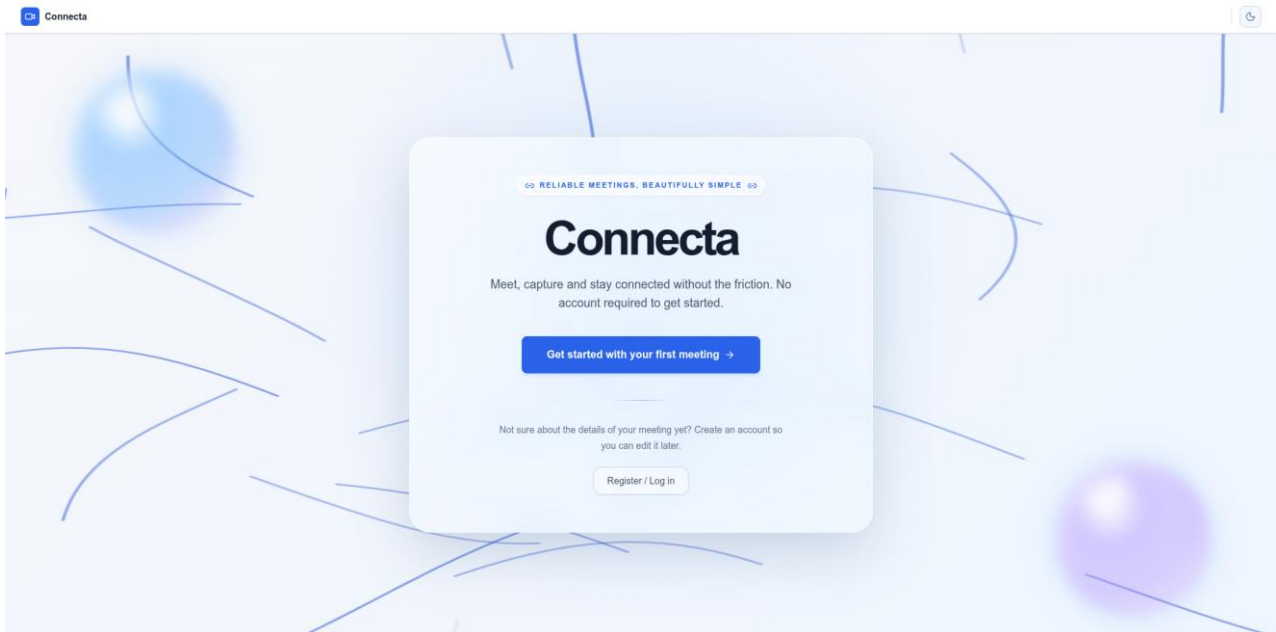
У овом поглављу приказане су најзначајније функционалности и кориснички интерфејс имплементираног система за реализацију видео конференција. Приказ обухвата кључне делове система са којима корисник непосредно ступа у интеракцију, од почетне странице и процеса креирања састанка, преко приступа и управљања видео конференцијом, до прегледа и управљања снимљеним садржајем. Циљ овог поглавља је да се на конкретним примерима прикаже начин на који су претходно описани захтеви и пројектована решења реализовани у оквиру имплементираног система.

На слици 6.1 приказана је почетна страница имплементираног система за видео конференције. Почетна страница представља тачку уласка у апликацију и дизајнирана је тако да омогући брз приступ кључним функционалностима без непотребних корака.

У централном делу екрана позициониран је главни панел који садржи назив апликације и кратак опис. Опис експлицитно наводи да за покретање састанка није неопходно креирање корисничког налога, чиме се умањује баријера за почетак коришћења система. Примарна акција на страници реализована је кроз дугме *Get started with your first meeting*. Активацијом овог дугмета, корисник се директно преусмерава на процес креирања новог видео састанка без претходне аутентификације.

Испод примарне акције налази се опција за регистрацију и пријаву на систем, представљена дугметом *Register / Log in*. Ова функционалност је намењена корисницима који желе да креирају налог како би сачували податке о састанцима за накнадну употребу или измену.

Иза централног панела налазе се украсни елементи у виду *36 анимираних линија*. Линије прате предефинисане путање али су њихова појављивања насумична тако да дају кориснику осећај непрекидности.



Слика 6.1 — Почетна страница платформе

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/landing-page.component.ts`.

На слици 6.2 приказана је екранска форма *New Meeting*, која служи за конфигурацију параметара и креирање новог видео састанка у систему. Форма је логички сегментирана у неколико целина како би се олакшао унос неопходних података.

У првом сегменту, под називом *BASIC INFO*, омогућен је унос наслова састанка кроз текстуално поље *Title*. Следећа секција, *SCHEDULE*, намењена је дефинисању термина одржавања састанка. У оквиру ње налазе се контрола за одабир датума и времена почетка, као и поље за унос предвиђеног трајања састанка у минутима. Опциони опис састанка може се унети у оквиру секције *ADDITIONAL INFORMATION*. За ову намену имплементиран је едитор текста у пољу *Description (optional)* који садржи основне алате за форматирање, попут подебљавања, искошавања текста и креирања набрајања. Управљање списком учесника врши се у секцији *PARTICIPANTS*. Омогућено је динамичко додавање нових корисника притиском на акцију *Add participant*. За сваког додатог учесника приказује се текстуално поље за унос имена, дугме за додељивање улоге домаћина (*HOST*), као и икона за уклањање тог учесника са листе.

На крају, у оквиру сегмента *RECORDING* имплементиран је прекидач *Enable recording*, којим се дефинише да ли ће систем аутоматски снимати састанак. Целокупан процес се финализује притиском на акционо дугме *Create Meeting* које се налази на дну форме, чиме се потврђују унети подаци и иницира креирање састанка у систему.

New Meeting

Configure your meeting settings and participants.

BASIC INFO

Title

Q3 Product Review

SCHEDULE

Scheduled at

mm/dd/yyyy, --:--



Duration (minutes)

60

ADDITIONAL INFORMATION

Description (optional)

B *I* U | ☰ ☷

Provide a short description of the meeting

PARTICIPANTS

+ Add participant



Participant name

👤 HOST



Participant name

👤 HOST



Participant name

👤 HOST



RECORDING

Enable recording



Create Meeting

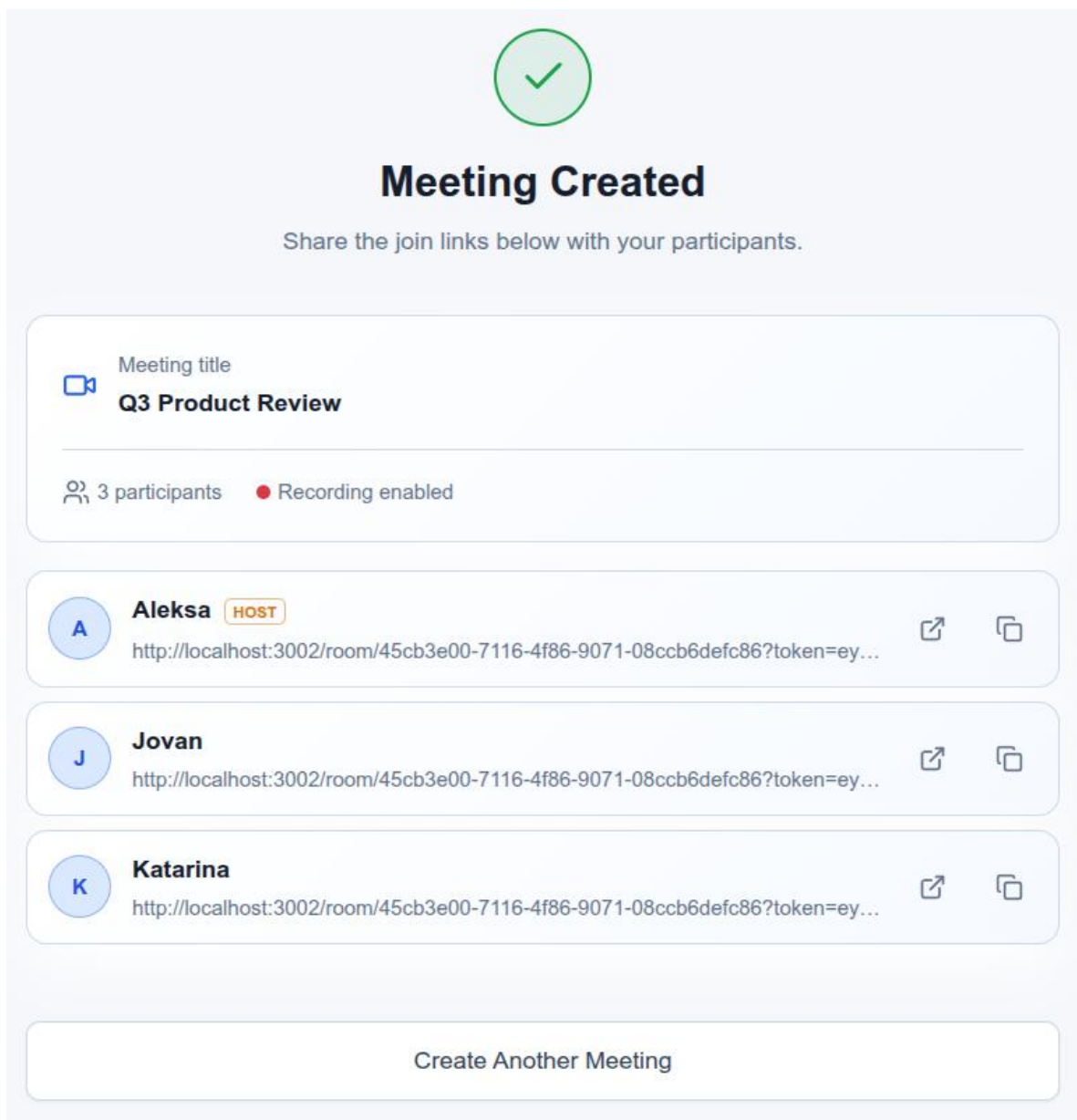
Слика 6.2 — Форма за креирање састанка

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/create-meeting-page.component.ts`.

На слици 6.3 приказана је екранска форма која кориснику пружа повратну информацију о успешном креирању видео састанка, насловљена поруком *Meeting Created*. Основна сврха овог приказа је дистрибуција генерисаних приступних параметара претходно дефинисаним учесницима.

У горњем делу приказа налази се панел са сумарним информацијама, у оквиру којег је исписан назив састанка под ознаком *Meeting title*. У истом панелу приказан је укупан број пријављених учесника, као и статус опције за аутоматско снимање, који је визуелно наглашен индикатором *Recording enabled*.

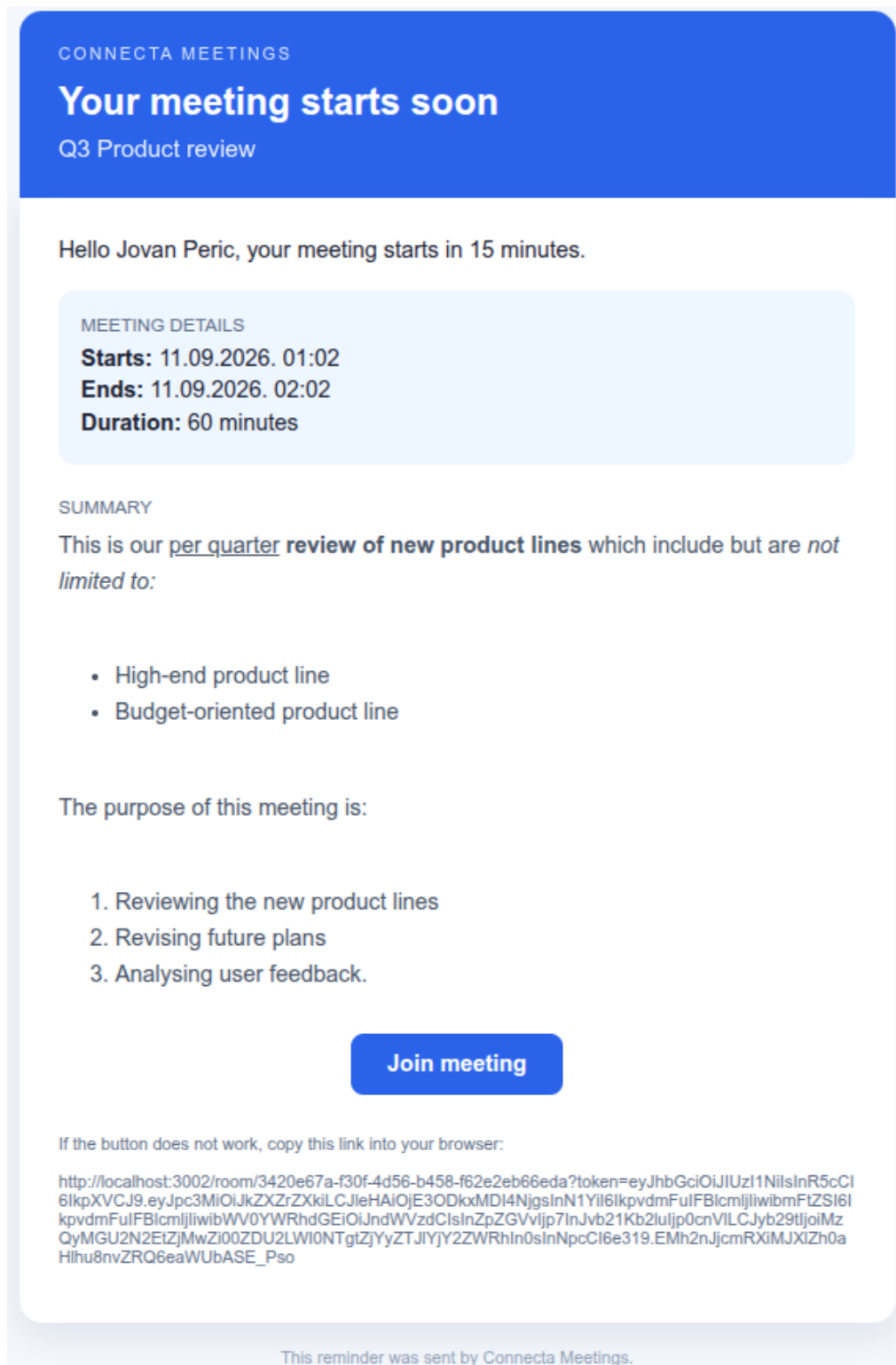
Централни део екрана заузима листа учесника. За сваког унетог корисника систем је генерисао јединствени линк за приступ састанку. За сваког учесника на листи обезбеђене су контроле за директно отварање приступног линка у веб прегледачу, као и за његово копирање у привремену меморију рачунара ради даљег дељења.



Слика 6.3 — Приказ успешно креираног састанка и линкова

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/meeting-created-page.component.ts`.

Систем сваких 60 секунди проверава који састанци почињу ускоро, и за сваки састанак чији је почетак у року од 15 минута свим учесницима са доступном е-mail адресом шаље поруку подсећања за надалазећи састанак. На слици 6.4 приказан је овај подсетник, који је у форми сличан поруци позивнице на састанак са слике 6.5.



Слика 6.4 — е-mail за подсећање корисника о надалазећем састанку

Извор: Снимак екрана мејла-подсетника.

Приликом креирања састанка, кориснику је пружена могућност да за сваког учесника унесе његову e-mail адресу као и текст саме позивнице. Ова поља нису обавезна, али уколико се корисник одлучи да их попуни, систем свим учесницима за које су ови подаци доступни шаље позивницу на састанак путем електронске поште. На слици 6.5 приказана је email порука позивнице, која садржи основне податке о састанку, као и дугме на које корисник може да кликне да се директно из e-mail поруке прикључи састанку. Уколико учесников e-mail клиент не подржава HTML поруке, кориснику је дата и резервна опција у виду целог приступног линка који корисник затим копира у свој прегледач и тако се придружује састанку.

You're invited to Q3 Product Review

Hello Aleksa, you have been invited to join this meeting.

Aleksa, you will be the meeting host.

MEETING DETAILS

Starts: 15.09.2026. 09:30

Ends: 15.09.2026. 10:30

Duration: 60 minutes

SUMMARY

This is our per quarter **review of new product lines** which include but *are not limited to*:

- High-end product line
- Budget-oriented product line

The purpose of this meeting is:

1. Reviewing the new product lines
2. Revising future plans
3. Analysing user feedback.

[Join meeting](#)

If the button does not work, copy this link into your browser:

http://localhost:3002/room/45cb3e00-7116-4f86-9071-08ccb6defc86?token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJkZXZrZXkiLCJleHAiOiJlODkwMTQzMjQsInN1YiI6IkkFsZWtZYSIsIm5hbWUiOiJBbGVrc2EiLCJtZXRhZGF0YSI6Imhvc3QiLCJ2aWRiOiJl6eyJyb29tSm9pbil6dHJ1ZSwicm9vbSI6IjQ1Y2IzZTAwLTcxMTYtNGY4Ni05MDcxLTA4Y2NiNmRlZmM4NiJ9LCJzaXAiOiJ0nt9fQ.V327625eRYQ_qYyDF1vXHbwlXS5igKdEzLNUEOxWODs

This invitation was sent by Connecta Meetings.

Слика 6.5 — Е-mail позивница на састанак

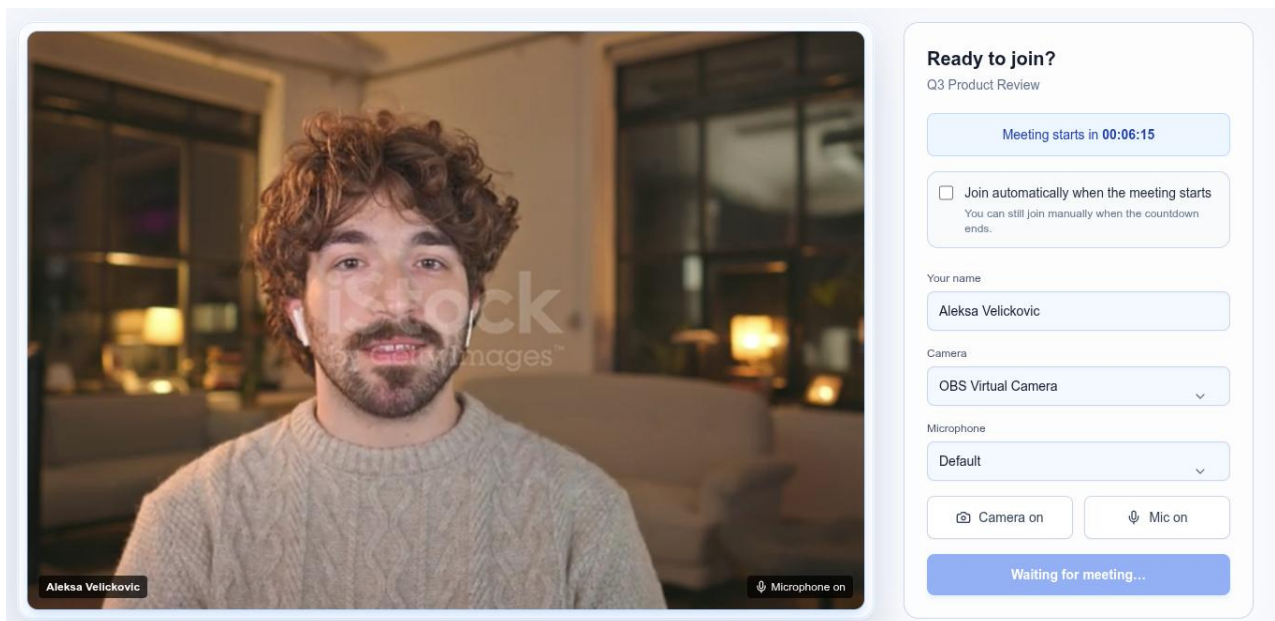
Извор: Снимак екрана е-mail позивнице.

На слици 6.6 приказана је припремна соба, којој корисник приступа пре самог уласка на састанак. Основна намена ове екранске форме је провера и конфигурација аудио-визуелних уређаја, као и приказ статуса предстојећег догађаја.

Лева страна корисничког интерфејса резервисана је за преглед видео-сигнала са изабране камере. У доњим угловима овог приказа налазе се информације о имену корисника и тренутном статусу микрофона, што омогућава визуелну потврду да ли се видео и звук успешно емитују пре прикључења сесији.

На десној страни интерфејса имплементиран је панел са контролама за подешавање параметара. Испод наслова *Ready to Join?* приказан је назив састанка, након чега следи тајмер који одбројава преостало време до заказаног почетка догађаја. Кориснику је омогућено да одабере опцију аутоматског придруживања састанку по истеку времена, означавањем поља *Join automatically when the meeting starts*.

У доњем делу контролног панела омогућен је унос или измена приказног имена корисника кроз текстуално поље *Your name*. За избор улазних хардверских компоненти реализоване су падајуће листе за одабир камере и микрофона. Брзо укључивање и искључивање ових уређаја пре уласка на састанак омогућено је притиском на контролна дугмад *Camera* и *Mic*, као и пречицама на тастатури Ctrl+V за микрофон и Ctrl+C за камеру. На самом дну форме позиционирано је главно акционо дугме које тренутно приказује статус *Waiting for meeting...* и које је у неактивном стању све док се не стекну неопходни услови за приступ састанку.

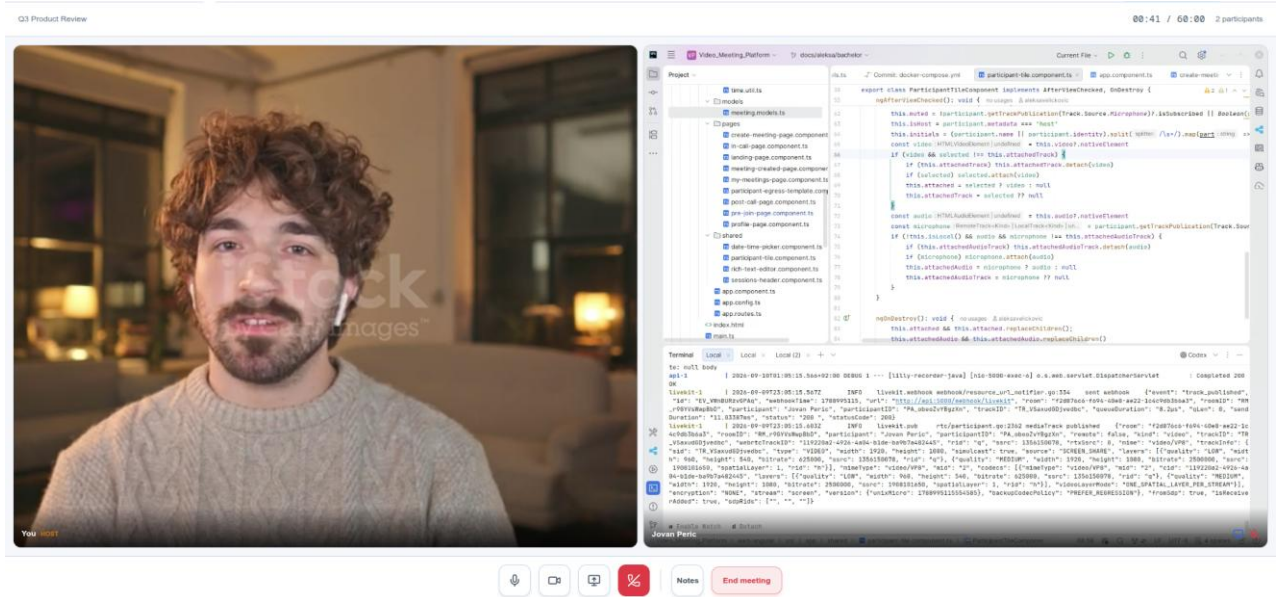


Слика 6.6 — Провера камере, микрофона и избор уређаја

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/pre-join-page.component.ts`.

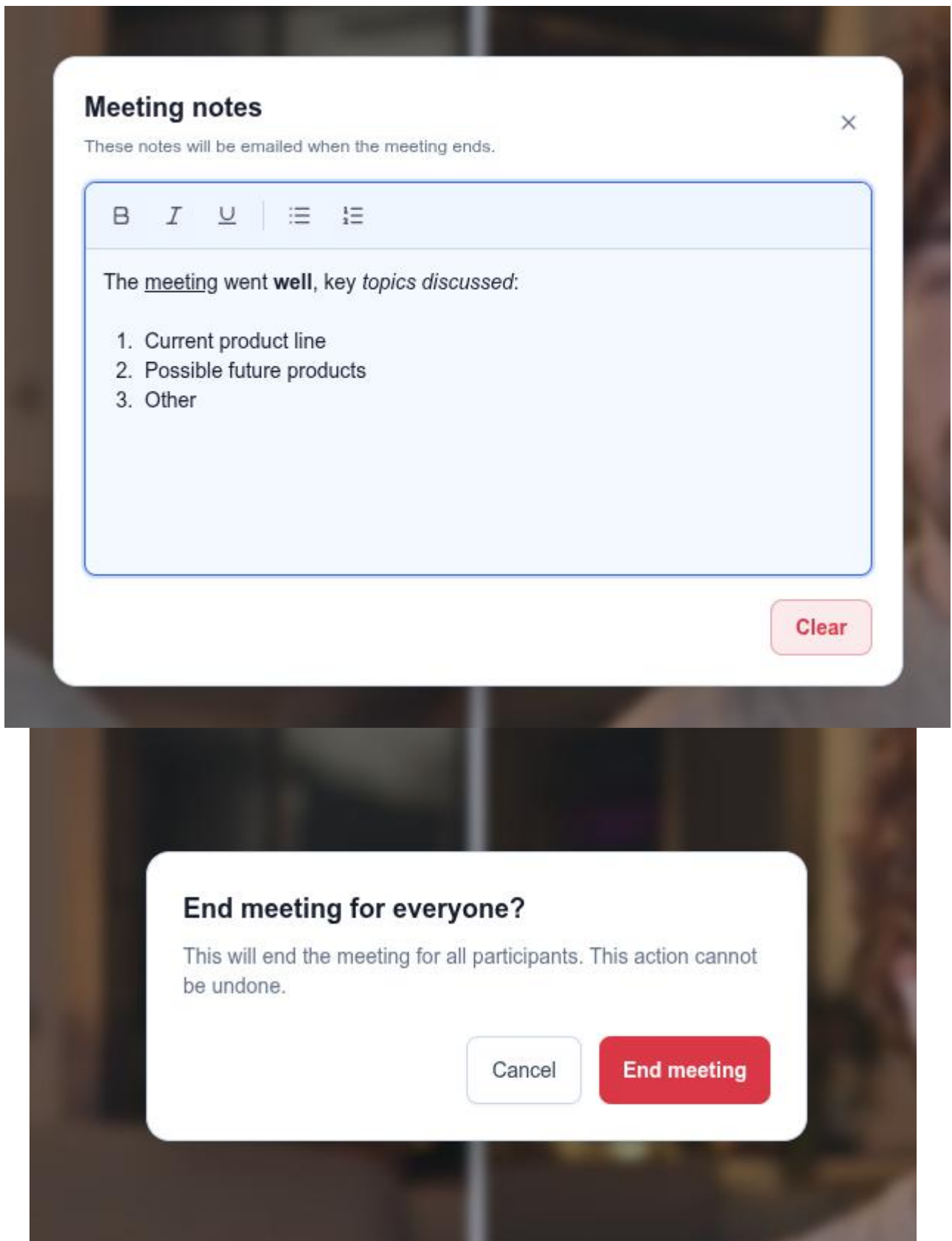
На слици 6.7 приказан је кориснички интерфејс активне видео-конференцијске сесије из перспективе host-а састанка, на којем је приказан и паралелни видео-сигнал другог учесника који дели свој екран. Главни део екрана заузимају два видео-приказа један поред другог, при чему се у доњем левом углу host-овог видео-квадрата налази ознака *HOST*.

У доњем делу екрана позиционирана је контролна трака са стандардним алатима за управљање аудио и видео уређајима. На десном делу контролне траке налазе се специјализована дугмад намењена искључиво host-у сесије, а то су опција за вођење белешки *Notes* и акција за прекидање састанка *End meeting*. Белешке се након састанка шаљу свим учесницима путем електронске поште, ако је иста унесена. Функционалност ових дугмади је приказана на слици 6.8.



Слика 6.7 — Видео-састанак са два учесника, од којих један дели свој екран

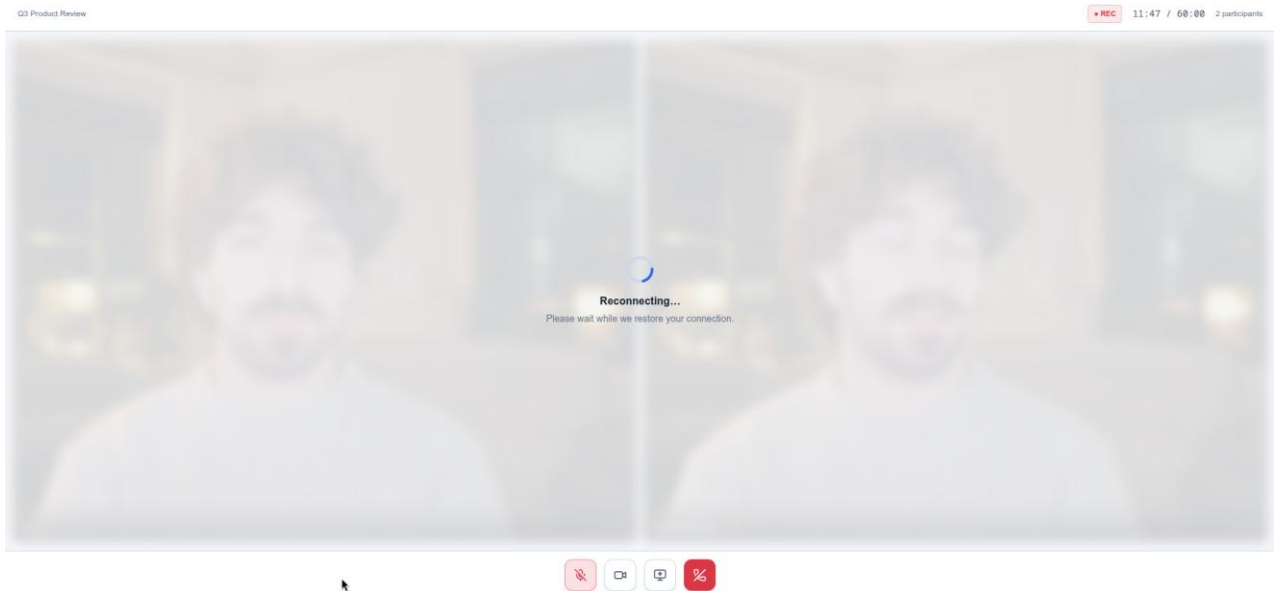
Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/in-call-page.component.ts`



Слика 6.8 — Специјализоване контроле за host-а састанка

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/in-call-page.component.ts`

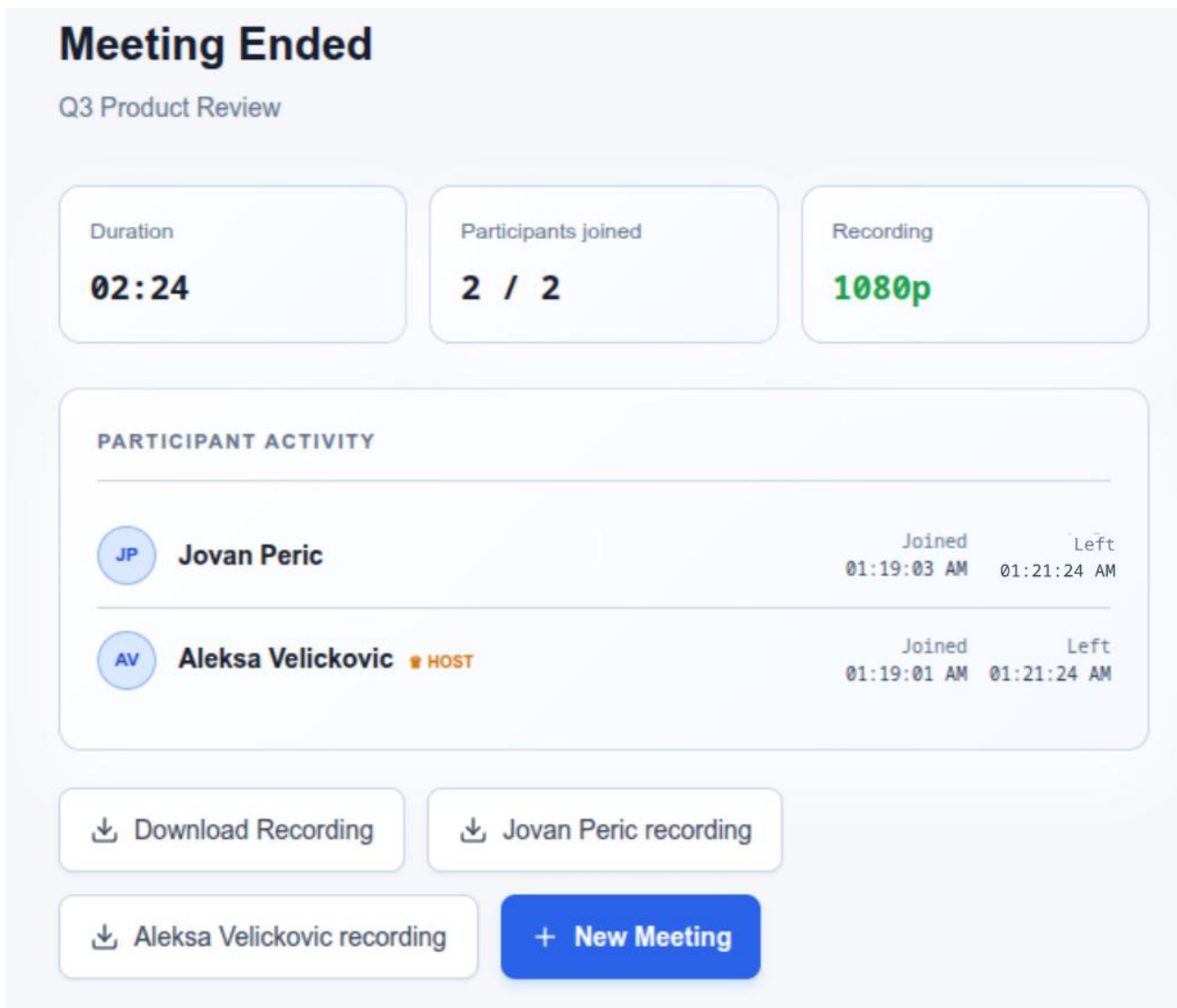
Overlay за реконекцију са слике 6.9 обавештава учесника да је медијска веза прекинута и да клијент покушава да је успостави. Његова сврха је да спречи погрешно тумачење празног или замрзнутог приказа као успешног наставка разговора, док LiveKit сервис обрађује Reconnecting и Reconnected догађаје.



Слика 6.9 — Overlay за поновно повезивање

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/core/livekit-room.service.ts`.

Post-call страница на слици 6.10 сумира завршену сесију кроз трајање разговора, учеснике и расположиве резултате. Екран приказује основне податке о састанку, као што су трајање, резолуција којом је састанак сниман, као и информације о томе када се који учесник придружио састанку и изашао из истог. Екран је из угла host-а састанка, па као такав приказује и додатну дугмад за преузимање композитног снимка целог састанка, као и засебних снимака видео и аудио feed-а сваког учесника.



Слика 6.10 — Post-call преглед информација о састанку

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/post-call-page.component.ts`.

Након завршетка састанка, свим корисницима за које је унета адреса електронске поште приликом креирања састанка, систем шаље поруку о завршеном састанку. На слици 6.11 је приказана ова порука. Порука садржи основне информације о састанку као и о учесницима, а домаћину састанка се још додатно и шаљу линкови за преузимање снимака састанка, обезбеђених помоћу S3 Pre-signed URL.

Meeting ended

Q3 Product review

Hello Aleksa Velickovic, here is your meeting summary.

Duration

1:09

Participants joined

2 / 2

Recording

1080p

Started: 11.09.2026. 01:02 - Ended: 11.09.2026. 01:03

Summary

This is our per quarter review of new product lines which include but are *not limited to*:

- High-end product line
- Budget-oriented product line

The purpose of this meeting is:

1. Reviewing the new product lines
2. Revising future plans
3. Analysing user feedback.

Meeting notes

The meeting went well key topics discussed:

- Current product line
- Possible future products
- Other

Participant activity

Aleksa Velickovic HOSTJoined: 01:02:01 AM - Left: 01:03:10 AM**Jovan Peric**Joined: 01:02:01 AM - Left: 01:03:10 AM

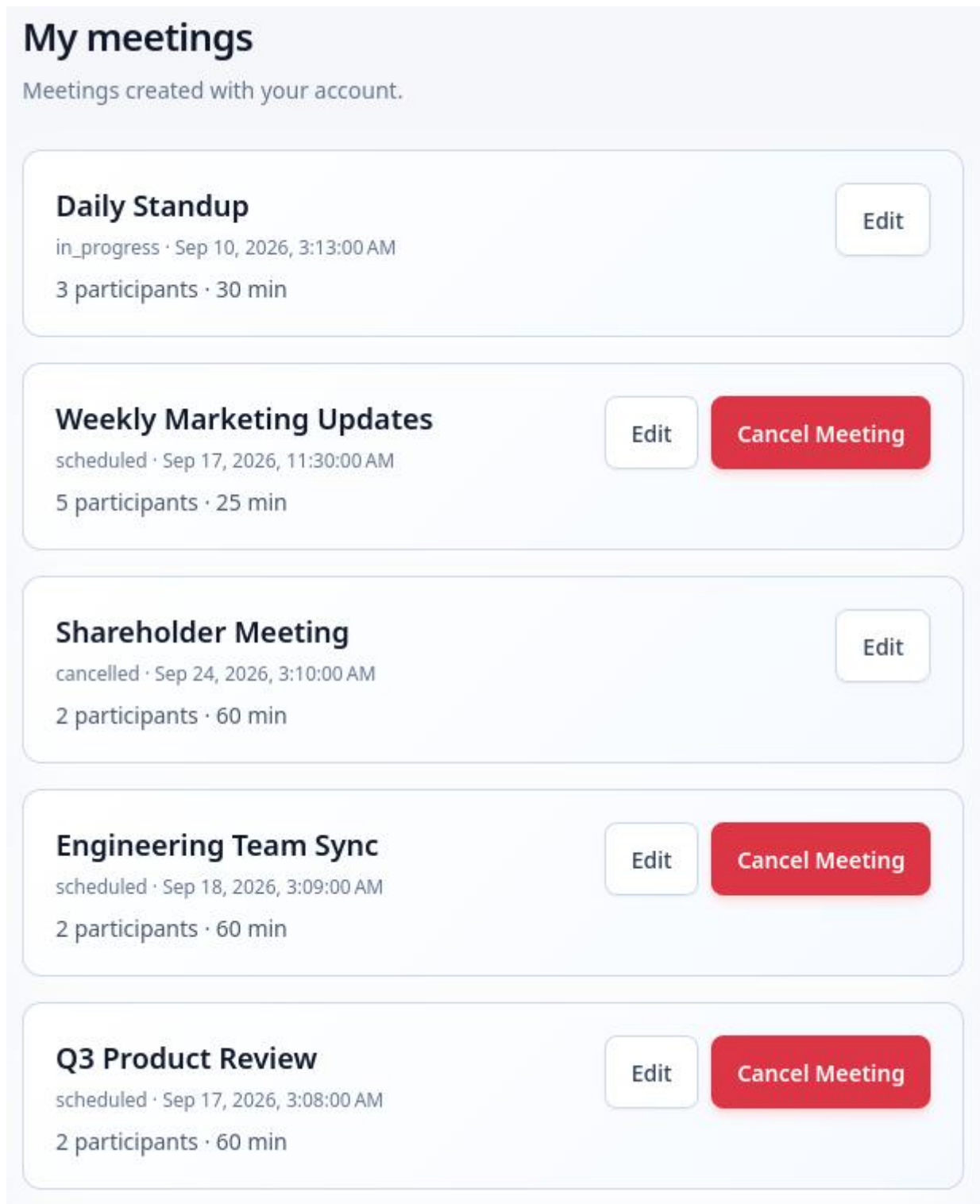
Recordings

[Download full meeting recording](#)[Download Aleksa Velickovic recording](#)[Download Jovan Peric recording](#)

Слика 6.11 — е-mail намењен сумирању протеклог састанка

Извор: Снимак екрана поруке електронске поште.

Страница на слици 6.12 са мојим састанцима приказује састанке повезане са профилем пријављеног организатора. Свака ставка садржи основне податке о састанку који описује, као што су име, заказан датум и време, број учесника и трајање, као и тренутни статус састанка. Такође, екран нуди корисницима могућност да измене или откажу састанак уколико је то могуће.



Слика 6.12 — Страница са мојим састанцима

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/my-meetings-page.component.ts`.

На слици 6.13 приказана је модална форма *Edit meeting*, чија је основна сврха накнадна измена и ажурирање дефинисаних параметара постојећег видео састанка пре потврде промена. Кориснички интерфејс је структуриран кроз логичке целине које омогућавају прецизну контролу над свим аспектима заказаног догађаја.

У горњем делу форме, у оквиру секције *BASIC INFORMATION*, налази се текстуално поље *Title* намењено измени назива састанка. Централни леви сегмент, *SCHEDULE*, обухвата контроле за дефинисање термина одржавања, где поље *Scheduled at* омогућава измену датума и времена почетка, док поље *Duration (minutes)* дефинише очекивано трајање сесије у минутима.

Десно од секције за заказивање налази се панел *RECORDING*, који је посвећен подешавањима видео-записивања. Укључивањем или искључивањем прекидача *Enable recording* дефинише се да ли ће се сесија снимати, док се кроз падајућу листу *Resolution preset* и пратећа поља за ширину (*Width*) и висину (*Height*) подешава жељена резолуција излазног видео-записа. Такође, назначено је да се снимање врши у MP4 формату.

У доњем делу форме смештена је опциона секција *DESCRIPTION* са напредним текстуалним едитором. Она омогућава унос и форматирање белешки или дневног реда састанка коришћењем основних алата за уређивање текста, попут подебљавања, искошавања, подвлачења и креирања листи са набрајањима.

Процес измене се закључује у дну форме, где су понуђене опције за одустајање од промена путем дугмета *Cancel*, односно трајна потврда унетих измена кликом на акционо дугме *Confirm changes*.

На слици 6.14 приказан је дијалог прозор за потврду отказивања састанка, чија је основна сврха спречавање ненамерног отказивања састанка. Интерфејс кориснику јасно предочава последице одустајања од догађаја, уз могућност одустајања од ове акције путем дугмета *No, keep it* или коначне потврде брисања помоћу дугмета *Yes, cancel meeting*.

MEETING SETTINGS

✕

Edit meeting

Update the details before confirming your changes.

BASIC INFORMATION

Title

Engineering Team Sync

SCHEDULE

Scheduled at

Fri, Sep 18, 2026 · 17:15

Duration (minutes)

60

RECORDING

Enable recording

Resolution preset

HD (1280 × 720)

Width

1280

Height

720

MP4 recording

DESCRIPTION (optional)

B I U | ☰ ☷

- In this meeting we will discuss what further steps we should take regarding the **project**

✕ Cancel

✓ Confirm changes

Cancel meeting?

Are you sure you want to cancel **Weekly Marketing Updates**? All participants will be notified by email and will no longer be able to join.

No, keep it

Yes, cancel meeting

Слике 6.13 и 6.14 — Измена и отказивање закажаног састанка

Извор: Снимак екрана из апликације; локација: `web-angular/src/app/pages/my-meetings-page.component.ts`;

Кориснички профил са слике 6.15 приказује податке које Keycloak и backend користе за идентификацију организатора. На истом месту су издвојене радње измене профила и покретања поступка за ресет лозинке, чиме се види граница између података профила и управљања акредитивима.

The screenshot shows a 'Profile' page with the following elements:

- Profile** (Section Header)
- Change your name and email without leaving the application. (Text)
- First name (Label):
- Last name (Label):
- Email (Label):
- Profile updated. (Green confirmation message)
- Save changes (Blue button)
- Change password by email (White button)
- Back to meetings (White button)

Слика 6.15 — Кориснички профил након ажурирања података

Извор: Снимак екрана из апликације; локација: web-angular/src/app/pages/profile-page.component.ts.

Keycloak login и register странице приказане сликом 6.16 приказују спољну границу аутентификације апликације. Слика обухвата странице за пријаву и регистрацију корисничког налога. Интерфејс садржи поља за унос корисничког имена или електронске поште, као и поље за унос лозинке, уз пратећу икону за прикривање или откривање унетог текста лозинке. Испод ових поља налази се опција за опоравак заборављене лозинке Forgot Password?, док се порврда пријаве врши активацијом дугмета Sign In. Такође, форма омогућава алтернативне начине пријаве путем Google

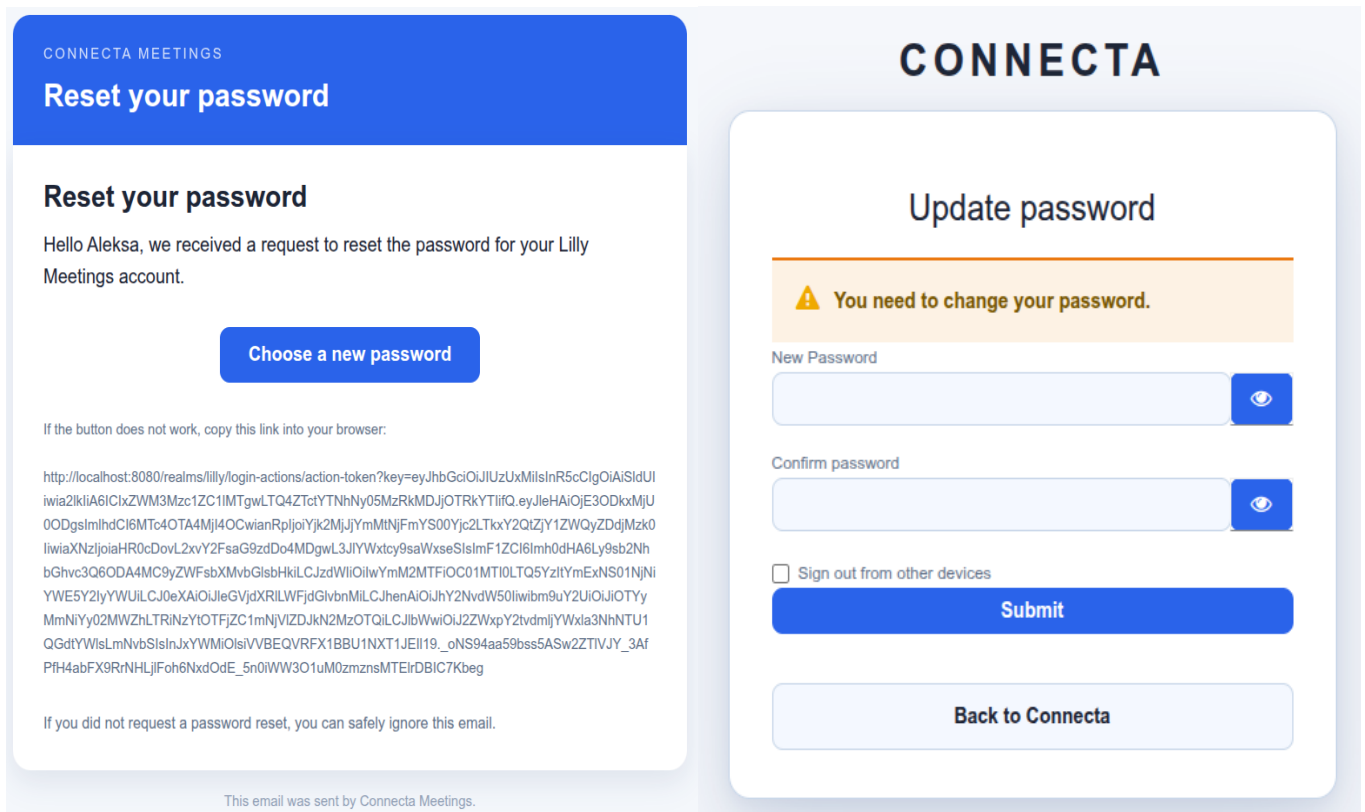
креденцијала, као и брз прелазак на форму за регистрацију нових корисника и повратка на почетну страницу. Форма за регистрацију садржи поља за унос корисничког имена, лозинке, електронске поште, као и имена и презимена. Корисник је приморан да унесе лозинку два пута, да би се избегао случајни унос погрешне лозинке. У коришћеној конфигурацији, при пријави преко Google Identity Provider-а додатна локална верификација адресе електронске поште није захтевана. Приликом коришћења класичног начина регистрације, уз пријаву путем адресе електронске поште и лозинке, након регистрације корисник добија мејл са верификационим линком, и може да приступи налогу тек након што исти кликне.

The image displays two side-by-side screenshots of the Keycloak Connecta theme interface. The left screenshot shows the 'Sign in to your account' page, featuring input fields for 'Username or email' and 'Password', a 'Forgot Password?' link, a blue 'Sign In' button, a 'New user? Register' link, and a Google sign-in option. The right screenshot shows the 'Register' page, which includes a '* Required fields' note, input fields for 'Username', 'Password', 'Confirm password', 'Email', 'First name', and 'Last name', a 'Back to Login' link, a blue 'Register' button, and a Google sign-in option. Both pages have a 'Back to Connecta' button at the bottom.

Слика 6.16 — Keycloak login и register странице

Извор: Снимак екрана из апликације; локација: `keycloak/themes/sessions/login/register.ftl`.

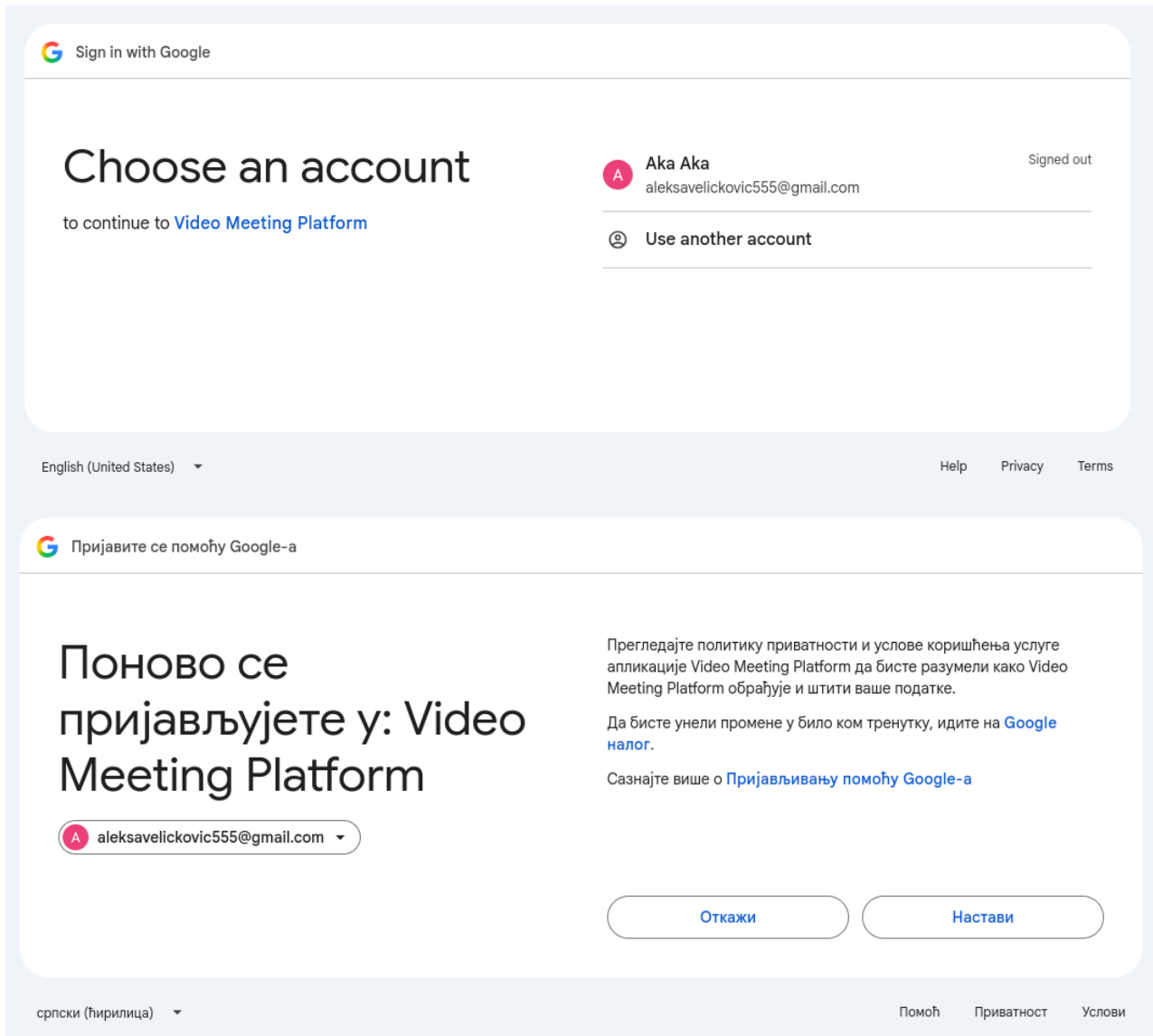
У случају да је корисник заборавио лозинку, на login страници може да унесе своје корисничко име или адресу електронске поште, након чега ће му бити послата електронска порука са линком који омогућава промену лозинке. На слици 6.17 је приказана форма за промену лозинке, као и порука коју корисник добија на e-mail.



Слика 6.17 — е-mail за ресетовање лозинке и форма за унос нове лозинке

Извор: Снимак екрана поруке за ресетовање лозинке.

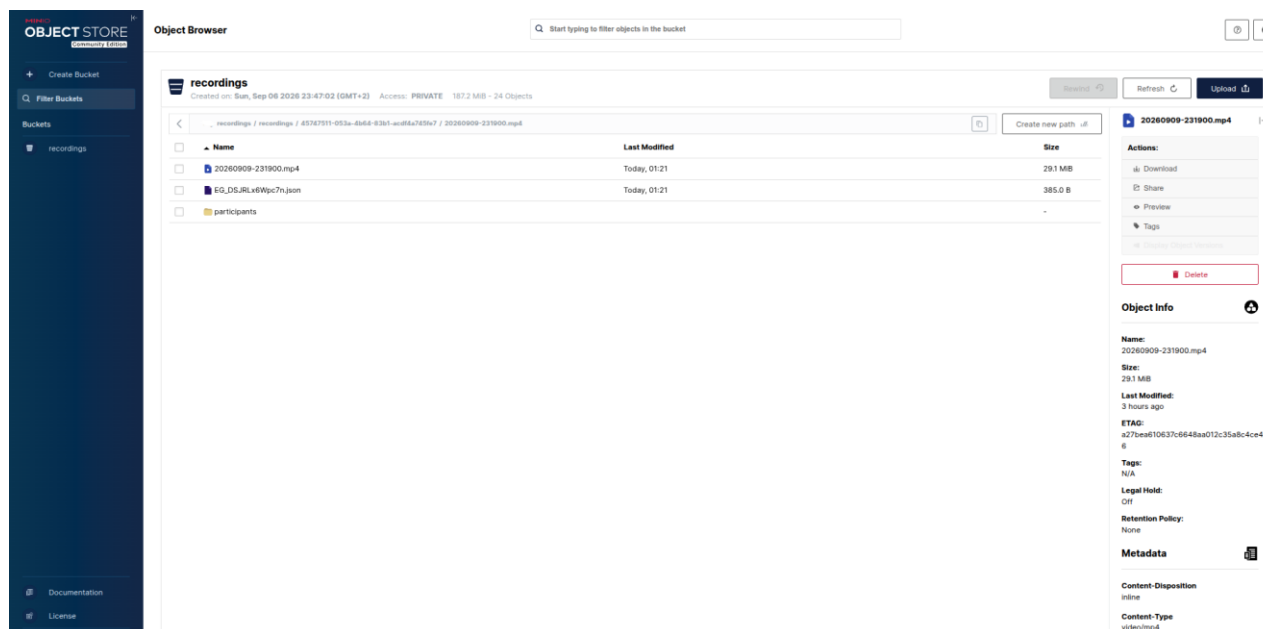
Google „Sign in with Google“ дугме приказује опциони Identity Provider ток у Keycloak-у. Када је Google провајдер подешен, корисник избором дугмета напушта локалну login форму, врши аутентификацију на Google OAuth 2.0 страници, и враћа се у исти OIDC client; Client Secret остаје у Keycloak конфигурацији и није део Angular апликације.



Слика 6.18 — Google login ток.

Извор: Снимак екрана, Google Account Manager

MinIO на слици 6.19 приказује bucket и MP4 објекат који је настао после Egress обраде. Назив object key-а повезује снимак са Meeting записом, док административни приказ потврђује да је бинарни резултат заиста уписан у S3-компатибилно складиште, док се кориснички download у апликацији реализује преко presigned URL-а. У фолдеру participants налазе се појединачни снимци сваког учесника који је приступио састанку, за које исто важи горенаведено.



Слика 6.19 — MinIO bucket са MP4 снимком

Извор: Снимак екрана из апликације MinIO; локација: <http://localhost:9001>.

6.1 Ручна провера функционалности

Ручна провера обухвата позитивне, негативне и граничне сценарије који покривају кључне функционалности, безбедносне механизме и понашање система при отказима зависних сервиса. Провера укључује креирање састанка без налога, приступ госта, рад са аудио и видео уређајима, контролу медија, реконекцију, аутентикацију преко Keycloak-а и Google Identity Provider-а, приступ заштићеним функцијама, покретање и завршетак снимања, појаву објекта у MinIO-у, presigned URL-ове, обраду webhook догађаја, слање извештаја преко Mailpit-а и реакцију на недоступност зависних сервиса. За сваки сценарио наведени су очекивани и добијени резултат, као и исход провере, чиме се омогућава директно поређење пројектованог понашања са понашањем реализованог система. У табели 6.1 представљени су резултати ручне провере свих кључних делова система.

Сценарио	Очекивани резултат	Добијени резултат	Исход провере
Креирање без налога	Систем креира састанак и генерише потребне join линкове без претходне аутентификације.	Састанак је успешно креиран, а генерисани су join линк за учеснике и подаци о састанку.	Успешно
Минималан број учесника	Захтев са мање од два учесника се одбија и не долази до креирања невалидног састанка.	При уносу једног учесника сервер је одбио захтев и састанак није креиран.	Успешно - одбијање
Приступ госта	Гост приступа састанку путем join линка без регистрације и добија приступ медијској сесији.	Гост је приступио састанку без налога, а медијска сесија је успостављена.	Успешно

Сценарио	Очекивани резултат	Добијени резултат	Исход провере
Дозвола за камеру и микрофон	Након одобравања browser дозвола доступни су изабрани аудио и видео уређаји.	Уређаји су препознати, приказан је preview и изабрани уређаји су коришћени у сесији.	Успешно
Одбијена дозвола за камеру или микрофон	Одбијање дозволе не прекида целу сесију; корисник може да уђе са искљученим уређајем и добија одговарајућу поруку.	Након одбијања дозволе корисник је могао да настави без одбијеног уређаја, уз приказ стања у интерфејсу.	Успешно – одбијање обрађено
Контрола медија	Корисник може да укључи и искључи камеру и микрофон, као и да промени изабрани уређај током разговора.	Искључивање камере и микрофона и промена микрофона извршени су без прекида сесије.	Успешно
Прекид и реконекција	Краткотрајни прекид мреже се приказује као стање реконекције и не доводи до губитка контекста састанка.	Након прекида мреже приказано је стање реконекције, а после повратка везе учесник и медијски track-ови су поново били доступни.	Успешно
Keycloak login	Након успешне локалне пријаве корисник се враћа у Angular апликацију са валидним токеном.	Корисник је успешно враћен у апликацију и добио је валидан JWT који је коришћен за приступ заштићеним функцијама.	Успешно
Истекао или неважећи JWT	Захтев са истеклим или невалидним токеном не сме да приступи заштићеном ресурсу.	Захтев без важећег токена је одбијен, а заштићени ресурс није био доступан.	Успешно – одбијање
Неауторизован приступ заштићеној функцији	Корисник без одговарајућег права не може да изврши заштићену операцију, чак и ако позив директно пошаље ка API-ју.	Директан захтев ка заштићеној функцији без потребне ауторизације је одбијен и није извршена пословна промена.	Успешно – одбијање
Неактиван API кључ	Неактиван или неважећи API кључ доводи до одбијања захтева без извршавања пословне операције.	Захтев са неактивним API кључем је одбијен и није извршен упис података.	Успешно – одбијање
Google регистрација	Када је Google Identity Provider правилно подешен, Keycloak broker креира или повезује корисника и враћа га у апликацију.	Након успешне Google аутентикације корисник је враћен у Angular, а Keycloak је креирао/повезао корисника.	Успешно
Снимање састанка	Након покретања снимања Egress креира процес, а по завршетку се добија MP4 објекат повезан са састанком.	Egress процес је покренут и након његовог завршетка MP4 снимак је био доступан у MinIO складишту.	Успешно
Неуспех Egress процеса	Прекинут или неуспешан Egress не сме бити приказан као исправно завршен снимак.	При неуспеху Egress-а резултат није представљен као исправан снимак, већ је стање обрађено као неуспешно снимање.	Успешно – грешка обрађена
Важећи presigned URL	Важећи потписани URL омогућава приступ конкретном снимку у ограниченом временском интервалу.	Помоћу важећег presigned URL-а MP4 снимак је успешно преузет/приказан у оквиру предвиђеног рока.	Успешно
Истекао или измењен presigned URL	URL након истека рока или са измењеним параметрима не сме омогућити приступ снимку.	Након истека рока приступ је одбијен, а измена key параметра није омогућила приступ другом објекту.	Успешно – одбијање

Сценарио	Очекивани резултат	Добијени резултат	Исход провере
Поновљени egress_ended webhook	Поновљена испорука истог webhook догађаја не сме да креира дупли пословни резултат, запис или callback.	Исти egress_ended догађај је поново послат, али је постојећи резултат препознат и није креиран дупли запис.	Успешно – идемпотентно
Непознат Egress ID	Webhook са Egress идентификатором који није познат систему не сме изменити постојеће записе.	Догађај са непознатим Egress ID-јем није повезао снимак ни са једним састанком и није изазвао нежељену измену података.	Успешно – одбијање
Webhook након кашњења или прекида	Кашњење webhook-а или прекид између уписа снимка и callback-а не сме довести до губитка везе између Egress-а и Meeting записа.	Након симулираног кашњења обрада догађаја је настављена на основу Egress ID-ја и снимак је повезан са одговарајућим записом.	Успешно
Завршетак састанка	По завршетку састанка соба се затвара, подаци о исходу се уписују, а извештај се прослеђује преко конфигурисаног email сервиса.	Соба је затворена, исход је уписан, а извештај је успешно прослеђен и видљив у Mailpit-у.	Успешно
Неуспешно слање email-а	Грешка email сервиса не сме поништити већ успешно креиран састанак или обрисати локално сачуване податке.	При недоступности email сервиса подаци о састанку су остали сачувани, док је неуспешно слање евидентирано као грешка зависног сервиса.	Успешно – грешка обрађена

Табела 6.1 — Резултати ручне провере кључних делова софтверског система

7 Закључак

У раду је приказано пројектовање и развој само-хостоване платформе за видео састанке у реалном времену. Систем повезује Angular frontend, Spring Boot backend, LiveKit SFU, LiveKit Egress, PostgreSQL, Redis, MinIO, Mailpit и Keycloak у једну Docker Compose целину.

Једна од значајних карактеристика развијеног система јесте једноставан приступ видео састанцима. За креирање састанка и приступ госта није неопходан налог, а учеснику није потребно преузимање посебног софтвера. Довољни су веб-прегледач и линк. Истовремено, корисници који желе историју састанака, измену профила или напредно управљање могу да користе налог обезбеђен помоћу Keycloak-а.

LiveKit омогућава да се WebRTC комуникација организује као серверска SFU услуга, док Egress издваја медијску обраду од пословне логике. MinIO омогућава локално S3-компатибилно складиштење, а presigned URL механизам смањује потребу за директним излагањем приступних кључева.

У оквиру безбедносног модела раздвојене су улоге guest join токена, API кључа, Keycloak JWT-а и LiveKit серверског токена. Keycloak омогућава класичну регистрацију, PKCE за frontend и могућност Google Identity Provider интеракције. Google credentials остају изван frontend-а и репозиторијума.

Функционалност развијеног система проверена је кроз скуп позитивних, негативних и граничних сценарија који обухватају основне корисничке токове, контролу приступа, обраду медијских уређаја, снимање и складиштење садржаја, обраду webhook догађаја и понашање при отказу појединих зависних сервиса. У спроведеним ручним проверама систем је остварио очекивано понашање у свим приказаним сценаријима, укључујући одбијање неважећих и неауторизованих захтева и правилну обраду репрезентативних грешака и поновљених догађаја.

Систем има и ограничења. Развојна Docker конфигурација користи подразумеване вредности, MinIO bucket се иницијално поставља као јавни, а Google Provider захтева спољно ручно подешавање. За продукцију би било потребно увести тајне, TLS, private bucket политику, ротацију кључева, детаљнију евиденцију callback покушаја и праћење рада сервиса.

Као будућа унапређења имплементираног система могу се издвојити подршка за рад са више организација, напреднији систем корисничких улога и дозвола, ограничење броја захтева ка API интерфејсу (енгл. *rate limiting*), централизовано логовање, унапређен мониторинг, шифровано складиштење објеката (енгл. *encrypted object storage*), TURN инфраструктура за поузданије успостављање WebRTC комуникације, као и аутоматизована конфигурација Google Identity Provider-а у оквиру безбедног процеса постављања система.

Подршка за више организација омогућила би да једна инстанца система буде коришћена од стране више независних организација, уз логичко раздвајање њихових корисника, састанака и података. Ово би систем учинило погоднијим за употребу у већим организационим структурама. Додатно, напредније корисничке улоге и дозволе омогућиле би прецизнију контролу приступа, на пример кроз разликовање администратора организације, модератора и обичних корисника. Увођењем механизма *rate limiting* било би могуће ограничити број захтева које појединачни корисник или IP адреса може да упути систему у одређеном временском периоду, чиме би се смањила могућност злоупотребе API интерфејса и непотребног оптерећења система.

Централизовано логовање и напреднији мониторинг омогућили би обједињено праћење рада различитих компоненти система и лакше откривање проблема. Поред основних системских метрика,

могле би се пратити информације као што су број активних састанака, број учесника, искоришћеност ресурса и стање процеса задужених за снимање. Са друге стране, увођењем шифрованог складиштења обезбедила би се додатна заштита снимљених видео материјала, нарочито у случају неовлашћеног приступа систему за складиштење.

Увођење TURN инфраструктуре представљало би додатно унапређење WebRTC дела система, јер би омогућило успостављање комуникације и у мрежним окружењима у којима директна веза између учесника није могућа због NAT или firewall ограничења. На крају, аутоматизацијом конфигурације Google Identity Provider-а у оквиру deployment процеса смањила би се потреба за ручним подешавањем, уз могућност да се осетљиви параметри безбедно задају кроз конфигурацију окружења или механизам за управљање тајнама. На овај начин би се систем могао постепено унапредити у погледу скабилности, безбедности, поузданости и одржавања.

Наведена унапређења не представљају предуслов за коришћење тренутне имплементације, већ могуће правце њеног даљег развоја. Њиховом постепеном имплементацијом систем би могао да подржи већи број корисника и организација, обезбеди прецизнију контролу приступа и виши ниво безбедности, као и да омогући детаљнији надзор над радом инфраструктуре и поузданију реализацију видео комуникације у различитим мрежним окружењима.

8 Литература

- [1] W3C, WebRTC 1.0: Real-Time Communication Between Browsers, <https://www.w3.org/TR/webrtc/>,
приступљено 08.09.2026.
- [2] IETF, RFC 8825: Overview: Real-Time Protocols for Browser-Based Applications, <https://www.rfc-editor.org/rfc/rfc8825>, приступљено 08.09.2026.
- [3] IETF, RFC 6455: The WebSocket Protocol, <https://www.rfc-editor.org/rfc/rfc6455>, приступљено 08.09.2026.
- [4] IETF, RFC 7519: JSON Web Token, <https://www.rfc-editor.org/rfc/rfc7519>, приступљено 08.09.2026.
- [5] IETF, RFC 7636: Proof Key for Code Exchange by OAuth Public Clients, <https://www.rfc-editor.org/rfc/rfc7636>, приступљено 08.09.2026.
- [6] OpenID Foundation, OpenID Connect Core 1.0, https://openid.net/specs/openid-connect-core-1_0.html, приступљено 08.09.2026.
- [7] Keycloak Documentation, Server Administration Guide, https://www.keycloak.org/docs/latest/server_admin/, приступљено 08.09.2026.
- [8] Keycloak Documentation, Securing Applications and Services Guide, https://www.keycloak.org/docs/latest/securing_apps/, приступљено 08.09.2026.
- [9] LiveKit Documentation, Rooms, Participants and Tracks, <https://docs.livekit.io/intro/basics/rooms-participants-tracks/>, приступљено 08.09.2026.
- [10] LiveKit Documentation, Webhooks and Events, <https://docs.livekit.io/intro/basics/rooms-participants-tracks/webhooks-events/>, приступљено 08.09.2026.
- [11] LiveKit Documentation, Egress Overview, <https://docs.livekit.io/transport/media/ingress-egress/egress/>, приступљено 08.09.2026.
- [12] Oracle, Java Platform, Standard Edition Documentation, <https://docs.oracle.com/en/java/javase/21/>,
приступљено 08.09.2026.
- [13] Spring, Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/reference/>,
приступљено 08.09.2026.
- [14] Spring, Spring Framework Web MVC Documentation, <https://docs.spring.io/spring-framework/reference/web/webmvc.html>, приступљено 08.09.2026.
- [15] Jakarta Persistence, Jakarta Persistence Specification, <https://jakarta.ee/specifications/persistence/>,
приступљено 08.09.2026.
- [16] Hibernate ORM Documentation, <https://hibernate.org/orm/documentation/>, приступљено 08.09.2026.
- [17] PostgreSQL Global Development Group, PostgreSQL Documentation, <https://www.postgresql.org/docs/>, приступљено 08.09.2026.
- [18] Redis Documentation, <https://redis.io/docs/latest/>, приступљено 08.09.2026.
- [19] Angular Documentation, <https://angular.dev/docs>, приступљено 08.09.2026.

- [20] TypeScript Documentation, <https://www.typescriptlang.org/docs/>, приступљено 08.09.2026.
- [21] RxJS Documentation, <https://rxjs.dev/guide/overview>, приступљено 08.09.2026.
- [22] Tailwind CSS Documentation, <https://tailwindcss.com/docs>, приступљено 08.09.2026.
- [23] Lucide Documentation, <https://lucide.dev/guide/packages/lucide-angular>, приступљено 08.09.2026.
- [24] Docker Documentation, Docker Compose, <https://docs.docker.com/compose/>, приступљено 08.09.2026.
- [25] Docker Documentation, Compose File Reference, <https://docs.docker.com/compose/compose-file/>, приступљено 08.09.2026.
- [26] Nginx Documentation, <https://nginx.org/en/docs/>, приступљено 08.09.2026.
- [27] Google Identity, OpenID Connect, <https://developers.google.com/identity/openid-connect/openid-connect>, приступљено 08.09.2026.
- [28] Amazon Web Services, Amazon S3 User Guide, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>, приступљено 08.09.2026.
- [29] Amazon Web Services, Download and Upload Objects with Presigned URLs, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>, приступљено 08.09.2026.
- [30] MinIO Documentation, S3 API Compatibility, <https://min.io/docs/minio/linux/developers/s3-api-compatibility.html>, приступљено 08.09.2026.
- [31] Mailpit Documentation, <https://mailpit.axllent.org/docs/>, приступљено 08.09.2026.
- [32] Apache Maven Project, Maven Documentation, <https://maven.apache.org/guides/>, приступљено 08.09.2026.
- [33] Spring Security Documentation, OAuth 2.0 Resource Server JWT, <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>, приступљено 08.09.2026.
- [34] AWS SDK for Java 2.x Developer Guide, Amazon S3, <https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/home.html>, приступљено 08.09.2026.
- [35] Integrating Sign In with Google into your web app, <https://developers.google.com/identity/gsi/web/guides/overview>, приступљено 08.09.2026.
- [36] TURN and STUN Servers, <https://trueconf.com/blog/wiki/turn-and-stun-servers>, приступљено 08.09.2026.
- [37] Zoom, <https://www.zoom.com/>, приступљено 08.09.2026.
- [38] Google Meet, <https://meet.google.com>, приступљено 08.09.2026.
- [39] Microsoft Teams, <https://www.microsoft.com/microsoft-teams>, приступљено 08.09.2026.
- [40] Jitsi Meet, <https://meet.jit.si>, приступљено 08.09.2026.
- [41] Cisco Webex, <https://www.webex.com>, приступљено 08.09.2026.
- [42] Whereby, How to Choose a Free Conference Call Platform, <https://whereby.com/blog/how-to-choose-a-free-conference-call-platform/>, приступљено 08.09.2026.

- [43] The Verge, Zoom accessibility features, <https://www.theverge.com/2020/9/23/21452400/zoom-accessibility-features-pin-spotlight-multiple-videos-during-call>, приступљено 08.09.2026.
- [44] Android Authority, Google Meet dynamic layout, <https://www.androidauthority.com/google-meet-dynamic-layout-3539680/>, приступљено 08.09.2026.
- [45] Palssere, Customize Jitsi Meet UI, <https://palssere.weebly.com/blog/customize-jitsi-meet-ui>, приступљено 08.09.2026.
- [46] ZDNET, Cisco debuts new Webex Suite for hybrid work, <https://www.zdnet.com/article/cisco-debuts-new-webex-suite-for-hybrid-work-adds-hundreds-of-new-features/>, приступљено 08.09.2026.
- [47] Efficient and scalable video conferences with selective forwarding units and webRTC, <https://theses.hal.science/tel-02917712/>, приступљено 08.09.2026.
- [48] Hypertext Transfer Protocol, <https://www.rfc-editor.org/info/rfc2616/>, приступљено 08.09.2026.
- [49] WebRTC SFU: the complete Guide., <https://medium.com/@jamesbordane57/webrtc-sfu-the-complete-guide-3589be4daa54>, приступљено 08.09.2026.
- [50] Cataloging dependency injection anti-patterns in software systems., https://www.researchgate.net/publication/355564988_Cataloging_dependency_injection_anti-patterns_in_software_systems, приступљено 08.09.2026.
- [51] Architectural Styles and the Design of Network-based Software Architectures, https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf, приступљено 08.09.2026.
- [52] Proof Key for Code Exchange (PKCE), <https://oauth.net/2/pkce/>, приступљено 08.09.2026.
- [53] Thesis Analysis and Design of a Hashing Algorithm, https://www.researchgate.net/publication/380728197_Thesis_Analysis_and_Design_of_a_Hashing_Algorithm_p_Hashing_Algorithm, приступљено 08.09.2026.
- [55] DrawIO, <https://app.diagrams.net/>, приступљено 08.09.2026.
- [55] UML class diagrams, <https://www.jetbrains.com/help/idea/class-diagram.html>, приступљено 08.09.2026.
- [56] Media Devices, <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/>, приступљено 08.09.2026.
- [57] What is software as a service (SaaS)?, <https://www.ibm.com/think/topics/saas>, приступљено 08.09.2026.
- [58] Graph API Client, <https://learn.microsoft.com/en-us/microsoftteams/platform/teams-sdk/essentials/graph?pivots=csharp>, приступљено 08.09.2026.
- [59] Formal Analysis of DTLS-SRTP Combined Protocol Based on Logic of Events, <https://www.mdpi.com/2076-3417/14/5/1804>, приступљено 08.09.2026.

[60] A Practical Look at Network Address Translation,

https://www.csd.uoc.gr/~hy435/material/whitepaper_technicalnetworkaddresstranslation.pdf,

приступљено 08.09.2026.

[61] Video Meeting Platform Thesis, <https://github.com/aleksavelickovic/VideoMeetingPlatformThesis>,

приступљено 08.09.2026.

[62] Mermaid, <https://mermaid.ai/>, приступљено 08.09.2026.

9 Биографија

Алекса Величковић је рођен 29. Новембра 2004. године у Београду. Завршио је средњу електротехничку школу “Михајло Пупин” у Новом Саду, смер Електротехничар рачунара. Студије на Факултету техничких наука Универзитета у Новом Саду уписао је 2023. Године, смер Софтверске и Информационе Технологије. У оквиру завршног рада на студијама развио је платформу за видео састанке засновану на WebRTC технологији, са посебним нагласком на једноставност употребе, снимање, безбедност и контролу над подацима.